



LPIC-3 CERTIFICATION | 305-300

Learning LPIC-3 305-300

Virtualization and Containerization

Study guide and practical laboratory reference

English edition | Marcos Silvestrini | 2026-09-15

Contents

📖 About Project

🚀 Getting Started

🔧 Prerequisites

💾 Installation

📝 Usage

⬆️⬆️ For up and down

↺️ For reboot VMs

💻 Use PowerShell for up and down

🗺️ Infrastructure Schema Topic 351

🗺️ Roadmap

🗽 Four Essential Freedoms

🔍 Inspect commands

💻 Topic 351: Full Virtualization

🧠 351.1 Virtualization Concepts and Theory

🐧 351.2 Xen

💻 351.3 QEMU

🏢 351.4 Libvirt Virtual Machine Management

💾 351.5 Virtual Machine Disk Image Management

📦 Topic 352: container Virtualization

🧠 352.1 container Virtualization Concepts

📦 352.2 LXC

🐳 352.3 Docker

📁 352.4 container Orchestration Platforms

☁️ Topic 353: VM Deployment and Provisioning

☁️ 353.1 Cloud Management Tools

📦 353.2 Packer

☁️ 353.3 cloud-init

Understanding cloud-init

📦 353.4 Vagrant

 Contributing

 License

 Contact

 Acknowledgments



About Project

This project aims to help students or professionals to learn the main concepts of GNU/Linux and free software. Some GNU/Linux distributions like Debian and RPM will be covered. Installation and configuration of some packages will also be covered. By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this. Use **Vagrant** for up machines and execute labs and practice content in this article. I have published in folder Vagrant a Vagrantfile with what is necessary for you to upload an environment for studies.



Getting Started

For starting the learning, see the documentation above.



Prerequisites

- Git
- VMware Workstation
- []
- Vagrant VMWare Utility
- Vagrant



Installation

Clone the repo

```
git clone https://github.com/marcossilvestrini/learning-lpic-3-305-300.git
cd learning-lpic-3-305-300
```

Customize a template *Vagrantfile-topic-XXX*. This file contains a vms configuration for labs.

Example:

- File Vagrantfile-topic-351
 - `vm.clone_directory = "<your_driver_letter>:\<folder>\<to_machine>\#{VM_NAME}-instance-1"`
Example: `vm.clone_directory = "E:\Servers\VMWare\#{VM_NAME}-instance-1"`
 - `vm.vmx["memsize"] = ""`
 - `vm.vmx["numvcpus"] = ""`
 - `vm.vmx["cpuid.coresPerSocket"] = ""`

Customize network configuration in files `configs/network`.



Usage

Use this repository for get learning about LPIC-3 305-300 exam



For up and down

Switch a *Vagrantfile-topic-xxx* template and copy for a new file with name *Vagrantfile*

```
cd vagrant && vagrant up  
cd vagrant && vagrant destroy -f
```



For reboot VMs

```
cd vagrant && vagrant reload
```

Important: *If you reboot vms without vagrant, shared folder not mount after boot.*

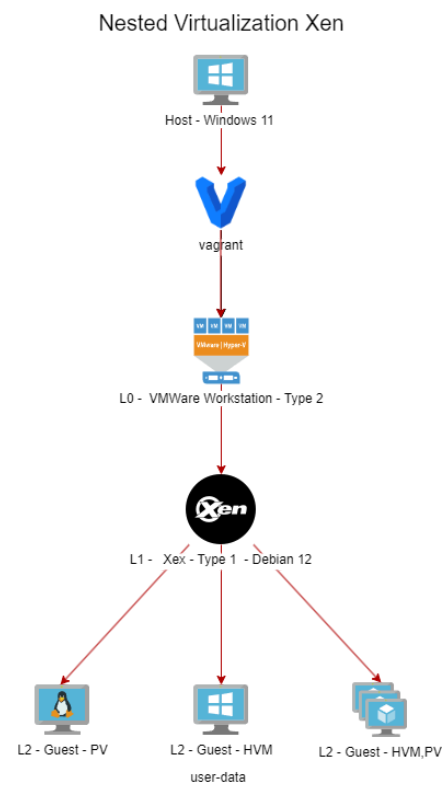


Use PowerShell for up and down

If you use Windows platform, I create a powershell script for up and down vms.

```
vagrant/up.ps1  
vagrant/destroy.ps1
```

Infrastructure Schema Topic 351





Roadmap

- ☒ Create repository
- ☒ Create scripts for provisioning labs
- ☒ Create examples about Topic 351
- ☒ Create examples about Topic 352
- ☒ Create examples about Topic 353
- ☐ Upload simulated itexam



Four Essential Freedoms

- 0:** The freedom to run the program as you wish, for any purpose (freedom 0).
- 1:** The freedom to study how the program works, and change it so it does your computing as you wish (freedom 1). Access to the source code is a precondition for this.
- 2:** The freedom to redistribute copies so you can help others (freedom 2).
- 3:** The freedom to distribute copies of your modified versions to others (freedom 3).



Inspect commands

```
type COMMAND # check if command is an alias, function, builtin or file
apropos COMMAND # search for COMMAND in manual page descriptions
whatis COMMAND --long # search for COMMAND in manual page descriptions with long output
whereis COMMAND # locate the binary, source and manual page files for COMMAND
COMMAND --help, --h # display help information for COMMAND
man COMMAND # display the manual page for COMMAND
```



Topic 351: Full Virtualization



The difference between type-1 and type-2 hypervisors



351.1 Virtualization Concepts and Theory

Weight: 6

Description: Candidates should know and understand the general concepts, theory and terminology of virtualization. This includes Xen, QEMU and libvirt terminology.

Key Knowledge Areas:

-  Understand virtualization terminology
-  Understand the pros and cons of virtualization
-  Understand the various variations of Hypervisors and Virtual Machine Monitors
-  Understand the major aspects of migrating physical to virtual machines
-  Understand the major aspects of migrating virtual machines between host systems
-  Understand the features and implications of virtualization for a virtual machine, such as snapshotting, pausing, cloning and resource limits
-  Awareness of oVirt, Proxmox, systemd-machined and VirtualBox
-  Awareness of Open vSwitch

351.1 Cited Objects

Hypervisor
Hardware Virtual Machine (HVM)
Paravirtualization (PV)
Emulation and Simulation
CPU flags
/proc/cpuinfo
Migration (P2V, V2V)

Hypervisors

Type 1 Hypervisor (Bare-Metal Hypervisor)

Type 1 Definition

Runs directly on the host's physical hardware, providing a base layer to manage VMs without the need for a host operating system.

Type 1 Characteristics

-  High performance and efficiency.
-  Lower latency and overhead.
-  Often used in enterprise environments and data centers.

Type 1 Examples

- VMware ESXi: A robust and widely used hypervisor in enterprise settings.
- Microsoft Hyper-V: Integrated with Windows Server, offering strong performance and management features.
- Xen: An open-source hypervisor used by many cloud service providers.
- KVM (Kernel-based Virtual Machine): Integrated into the Linux kernel, providing high performance for Linux-based systems.

Type 2 Hypervisor (Hosted Hypervisor)

Type 2 Definition

Runs on top of a conventional operating system, relying on the host OS for resource management and device support.

Type 2 Characteristics

-  Easier to set up and use, especially on personal computers.
-  More flexible for development, testing, and smaller-scale deployments.
-  Typically less efficient than Type 1 hypervisors due to additional overhead from the host OS.

Type 2 Examples

- VMware Workstation: A powerful hypervisor for running multiple operating systems on a single desktop.
- Oracle VirtualBox: An open-source hypervisor known for its flexibility and ease of use.
- Parallels Desktop: Designed for Mac users to run Windows and other operating systems alongside macOS.
- QEMU (Quick EMUlator): An open-source emulator and virtualizer, often used in conjunction with KVM.

Key Differences Between Type 1 and Type 2 Hypervisors

- Deployment Environment:
 - Type 1 hypervisors are commonly deployed in data centers and enterprise environments due to their direct interaction with hardware and high performance.
 - Type 2 hypervisors are more suitable for personal use, development, testing, and small-scale virtualization tasks.
- Performance:
 - Type 1 hypervisors generally offer better performance and lower latency because they do not rely on a host OS.
 - Type 2 hypervisors may experience some performance degradation due to the overhead of running on top of a host OS.
- Management and Ease of Use:
 - Type 1 hypervisors require more complex setup and management but provide advanced features and scalability for large-scale deployments.
 - Type 2 hypervisors are easier to install and use, making them ideal for individual users and smaller projects.

Migration Types

In the context of hypervisors, which are technologies used to create and manage virtual machines, the terms P2V migration and V2V migration are common in virtualization environments. They refer to processes of migrating systems between different types of platforms.

P2V - Physical to Virtual Migration

P2V migration refers to the process of migrating a physical server to a virtual machine. In other words, an operating system and its applications, running on dedicated physical hardware, are “converted” and moved to a virtual machine that runs on a hypervisor (such as VMware, Hyper-V, KVM, etc.).

- Example: You have a physical server running a Windows or Linux system, and you want to move it to a virtual environment, like a cloud infrastructure or an internal virtualization server. The process involves copying the entire system state, including the operating system, drivers, and data, to create an equivalent virtual machine that can run as if it were on the physical hardware.

V2V - Virtual to Virtual Migration

V2V migration refers to the process of migrating a virtual machine from one hypervisor to another. In this case, you already have a virtual machine running in a virtualized environment (like VMware), and you want to move it to another virtualized environment (for example, to Hyper-V or to a new VMware server).

- **Example:** You have a virtual machine running on a VMware virtualization server, but you decide to migrate it to a Hyper-V platform. In this case, the V2V migration converts the virtual machine from one format or hypervisor to another, ensuring it can continue running correctly.

HVM and Paravirtualization

Hardware-assisted Virtualization (HVM)

HVM Definition

HVM leverages hardware extensions provided by modern CPUs to virtualize hardware, enabling the creation and management of VMs with minimal performance overhead.

HVM Key Characteristics

-  **Hardware Support:** Requires CPU support for virtualization extensions such as Intel VT-x or AMD-V.
-  **Full Virtualization:** VMs can run unmodified guest operating systems, as the hypervisor provides a complete emulation of the hardware environment.
-  **Performance:** Typically offers near-native performance because of direct execution of guest code on the CPU.
-  **Isolation:** Provides strong isolation between VMs since each VM operates as if it has its own dedicated hardware.

HVM Examples

VMware ESXi, Microsoft Hyper-V, KVM (Kernel-based Virtual Machine).

HVM Advantages

-  **Compatibility:** Can run any operating system without modification.
-  **Performance:** High performance due to hardware support.
-  **Security:** Enhanced isolation and security features provided by hardware.

HVM Disadvantages

-  **Hardware Dependency:** Requires specific hardware features, limiting compatibility with older systems.
-  **Complexity:** May involve more complex configuration and management.

Paravirtualization

Paravirtualization Definition

Paravirtualization involves modifying the guest operating system to be aware of the virtual environment, allowing it to interact more efficiently with the hypervisor.

Paravirtualization Key Characteristics

-  **Guest Modification:** Requires changes to the guest operating system to communicate directly with the hypervisor using hypercalls.
-  **Performance:** Can be more efficient than traditional full virtualization because it reduces the overhead associated with emulating hardware.
-  **Compatibility:** Limited to operating systems that have been modified for paravirtualization.

Paravirtualization Examples

Xen with paravirtualized guests, VMware tools in certain configurations, and some KVM configurations.

Paravirtualization Advantages

-  **Efficiency:** Reduces the overhead of virtualizing hardware, potentially offering better performance for certain workloads.
-  **Resource Utilization:** More efficient use of system resources due to direct communication between the guest OS and hypervisor.

Paravirtualization Disadvantages

-  **Guest OS Modification:** Requires modifications to the guest OS, limiting compatibility to supported operating systems.
-  **Complexity:** Requires additional complexity in the guest OS for hypercall implementations.

Key Differences

Guest OS Requirements

- **HVM:** Can run unmodified guest operating systems.
- **Paravirtualization:** Requires guest operating systems to be modified to work with the hypervisor.

Performance

- **HVM:** Typically provides near-native performance due to hardware-assisted execution.
- **Paravirtualization:** Can offer efficient performance by reducing the overhead of hardware emulation, but relies on modified guest OS.

Hardware Dependency

- **HVM:** Requires specific CPU features (Intel VT-x, AMD-V).

- **Paravirtualization:** Does not require specific CPU features but needs modified guest OS.

Isolation

- **HVM:** Provides strong isolation using hardware features.
- **Paravirtualization:** Relies on software-based isolation, which may not be as robust as hardware-based isolation.

Complexity

- **HVM:** Generally more straightforward to deploy since it supports unmodified OS.
- **Paravirtualization:** Requires additional setup and modifications to the guest OS, increasing complexity.

NUMA (Non-Uniform Memory Access)

NUMA (Non-Uniform Memory Access) is a memory architecture used in multiprocessor systems to optimize memory access by processors. In a NUMA system, memory is distributed unevenly among processors, meaning that each processor has faster access to a portion of memory (its “local memory”) than to memory that is physically further away (referred to as “remote memory”) and associated with other processors.

Key Features of NUMA Architecture

1. **Local and Remote Memory:** Each processor has its own local memory, which it can access more quickly. However, it can also access the memory of other processors, although this takes longer.
2. **Differentiated Latency:** The latency of memory access varies depending on whether the processor is accessing its local memory or the memory of another node. Local memory access is faster, while accessing another node’s memory (remote) is slower.
3. **Scalability:** NUMA architecture is designed to improve scalability in systems with many processors. As more processors are added, memory is also distributed, avoiding the bottleneck that would occur in a uniform memory access (UMA) architecture.

Advantages of NUMA

-  **Better Performance in Large Systems:** Since each processor has local memory, it can work more efficiently without competing as much with other processors for memory access.
-  **Scalability:** NUMA allows systems with many processors and large amounts of memory to scale more effectively compared to a UMA architecture.

Disadvantages

-  **Programming Complexity:** Programmers need to be aware of which regions of memory are local or remote, optimizing the use of local memory to achieve better performance.
-  **Potential Performance Penalties:** If a processor frequently accesses remote memory, performance may suffer due to higher latency. This architecture is common in high-performance multiprocessor systems, such as servers and supercomputers, where scalability and memory optimization are critical.

Opensource Solutions

-  oVirt: <https://www.ovirt.org/>
-  Proxmox: <https://www.proxmox.com/en/proxmox-virtual-environment/overview>
-  Oracle VirtualBox: <https://www.virtualbox.org/>
-  Open vSwitch: <https://www.openvswitch.org/>

Types of Virtualization

Hardware Virtualization (Server Virtualization)

HV Definition

Abstracts physical hardware to create virtual machines (VMs) that run separate operating systems and applications.

HV Use Cases

Data centers, cloud computing, server consolidation.

HV Examples

VMware ESXi, Microsoft Hyper-V, KVM.

Operating System Virtualization (containerization)

containerization Definition

Allows multiple isolated user-space instances (containers) to run on a single OS kernel.

containerization Use Cases

Microservices architecture, development and testing environments.

containerization Examples

Docker, Kubernetes, LXC.

Network Virtualization

Network Virtualization Definition

Combines hardware and software network resources into a single, software-based administrative entity.

Network Virtualization Use Cases

Software-defined networking (SDN), network function virtualization (NFV).

Network Virtualization Examples

VMware NSX, Cisco ACI, OpenStack Neutron.

Storage Virtualization

Storage Virtualization Definition

Pools physical storage from multiple devices into a single virtual storage unit that can be managed centrally.

Storage Virtualization Use Cases

Data management, storage optimization, disaster recovery.

Storage Virtualization Examples

IBM SAN Volume Controller, VMware vSAN, NetApp ONTAP.

Desktop Virtualization

Desktop Virtualization Definition

Allows a desktop operating system to run on a virtual machine hosted on a server.

Desktop Virtualization Use Cases

Virtual desktop infrastructure (VDI), remote work solutions.

Desktop Virtualization Examples

Citrix Virtual Apps and Desktops, VMware Horizon, Microsoft Remote Desktop Services.

Application Virtualization

Application Virtualization Definition

Separates applications from the underlying hardware and operating system, allowing them to run in isolated environments.

Application Virtualization Use Cases

Simplified application deployment, compatibility testing.

Application Virtualization Examples

VMware ThinApp, Microsoft App-V, Citrix XenApp.

Data Virtualization

Data Virtualization Definition

Integrates data from various sources without physically consolidating it, providing a unified view for analysis and reporting.

Data Virtualization Use Cases

Business intelligence, real-time data integration.

Data Virtualization Examples

Denodo, Red Hat JBoss Data Virtualization, IBM InfoSphere.

Benefits of Virtualization

-  **Resource Efficiency:** Better utilization of physical resources.
-  **Cost Savings:** Reduced hardware and operational costs.
-  **Scalability:** Easy to scale up or down according to demand.
-  **Flexibility:** Supports a variety of workloads and applications.
-  **Disaster Recovery:** Simplified backup and recovery processes.
-  **Isolation:** Improved security through isolation of environments.

Emulation

Emulation involves simulating the behavior of hardware or software on a different platform than originally intended.

This process allows software designed for one system to run on another system that may have different architecture or operating environment.

While emulation provides versatility by enabling the execution of unmodified guest operating systems or applications, it often comes with performance overhead.

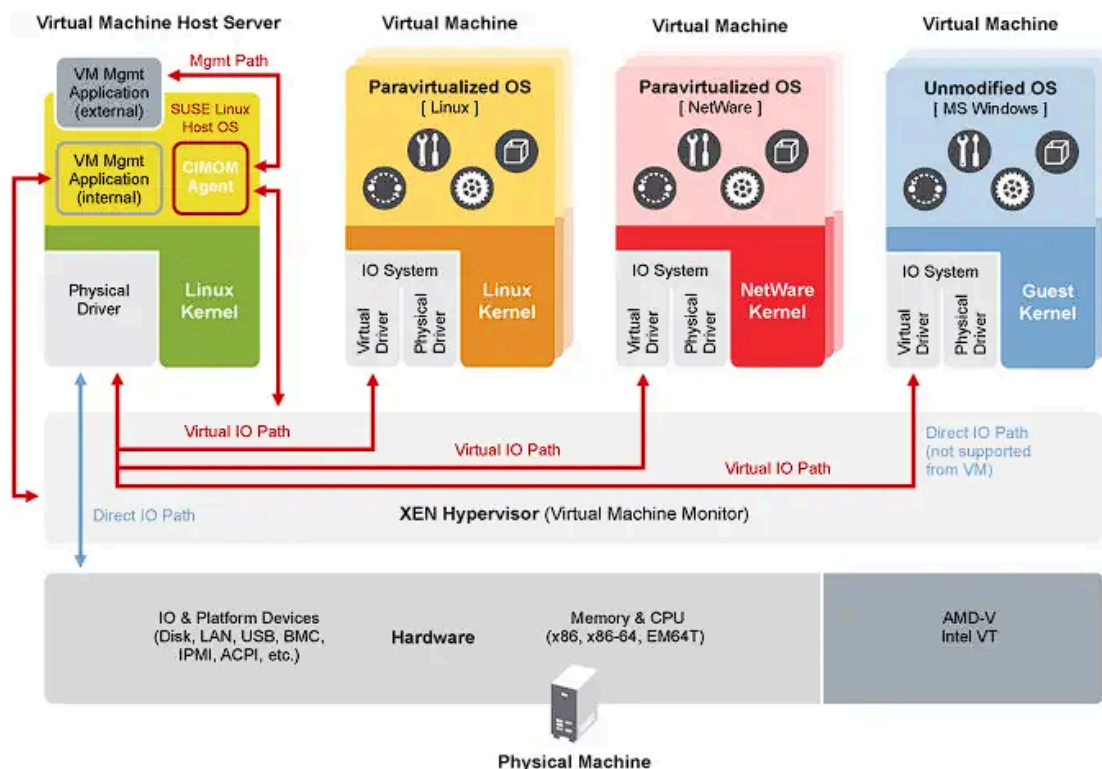
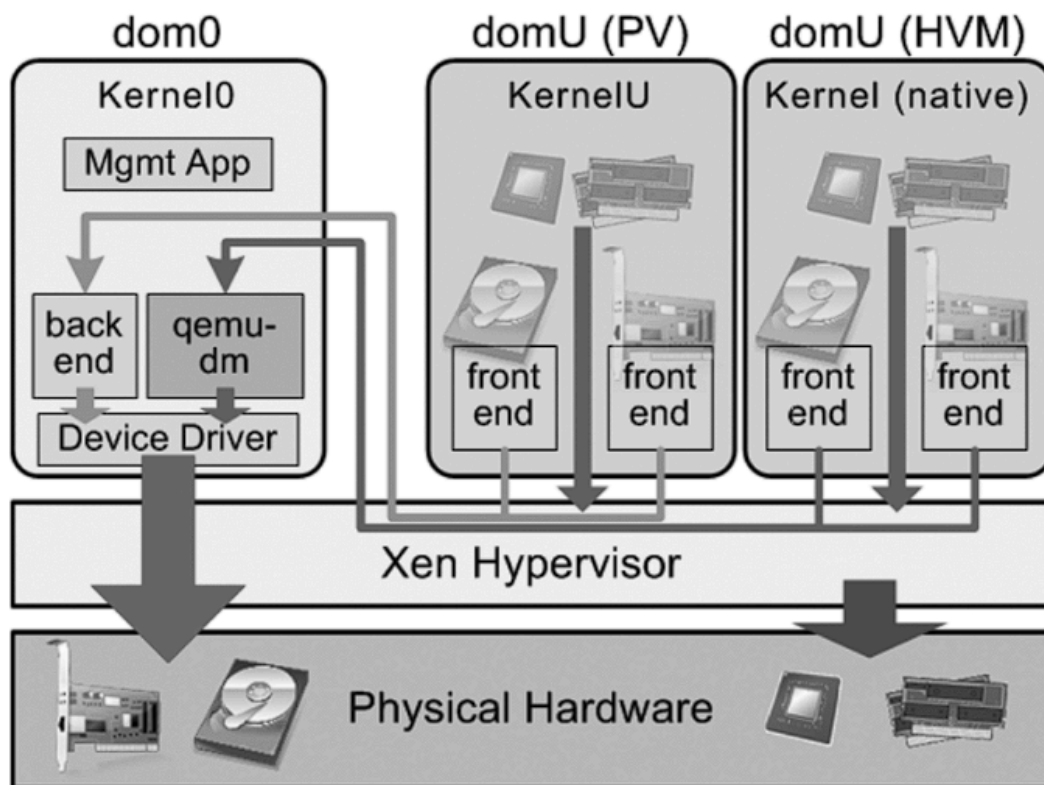
This overhead arises because the emulated system needs to interpret and translate instructions meant for the original system into instructions compatible with the host system. As a result, emulation can be slower than native execution, making it less efficient for resource-intensive tasks.

Despite this drawback, emulation remains valuable for running legacy software, testing applications across different platforms, and facilitating cross-platform development.

systemd-machined

The systemd-machined service is dedicated to managing virtual machines and containers within the systemd ecosystem. It provides essential functionalities for controlling, monitoring, and maintaining virtual instances, offering robust integration and efficiency within Linux environments.

351.2 Xen



Weight: 3

Description: Candidates should be able to install, configure, maintain, migrate and troubleshoot Xen installations. The focus is on Xen version 4.x.

Key Knowledge Areas:

- Understand architecture of Xen, including networking and storage
- Basic configuration of Xen nodes and domains
- Basic management of Xen nodes and domains
- Basic troubleshooting of Xen installations
- Awareness of XAPI
- Awareness of XenStore
- Awareness of Xen Boot Parameters
- Awareness of the xm utility

Xen



Xen is an open-source type-1 (bare-metal) hypervisor, which allows multiple operating systems to run concurrently on the same physical hardware. Xen provides a layer between the physical hardware and virtual machines (VMs), enabling efficient resource sharing and isolation.

- **Architecture:** Xen operates with a two-tier system where Domain 0 (Dom0) is the privileged domain with direct hardware access and manages the hypervisor. Other virtual machines, called Domain U (DomU), run guest operating systems and are managed by Dom0.
- **Types of Virtualization:** Xen supports both paravirtualization (PV), which requires modified guest OS, and hardware-assisted virtualization (HVM), which uses hardware extensions (e.g., Intel VT-x or AMD-V) to run unmodified guest operating systems. Xen is widely used in cloud environments, notably by Amazon Web Services (AWS) and other large-scale cloud providers.

XenSource

XenSource was the company founded by the original developers of the Xen hypervisor at the University of Cambridge to commercialize Xen. The company provided enterprise solutions based on Xen and offered additional tools and support to enhance Xen's capabilities for enterprise use.

- **Acquisition by Citrix:** In 2007, XenSource was acquired by Citrix Systems, Inc. Citrix used Xen technology as the foundation for its Citrix XenServer product, which became a popular enterprise-grade virtualization platform

based on Xen.

- **Transition:** After the acquisition, the Xen project continued as an open-source project, while Citrix focused on commercial offerings like XenServer, leveraging XenSource technology.

Xen Project

Xen Project refers to the open-source community and initiative responsible for developing and maintaining the Xen hypervisor after its commercialization. The Xen Project operates under the Linux Foundation, with a focus on building, improving, and supporting Xen as a collaborative, community-driven effort.

- **Goals:** The Xen Project aims to advance the hypervisor by improving its performance, security, and feature set for a wide range of use cases, including cloud computing, security-focused virtualization (e.g., Qubes OS), and embedded systems.
- **Contributors:** The project includes contributors from various organizations, including major cloud providers, hardware vendors, and independent developers.
- **XAPI and XenTools:** The Xen Project also includes tools such as XAPI (XenAPI), which is used for managing Xen hypervisor installations, and various other utilities for system management and optimization.

XenStore

Xen Store is a critical component of the Xen Hypervisor. Essentially, Xen Store is a distributed key-value database used for communication and information sharing between the Xen hypervisor and the virtual machines (also known as domains) it manages.

Here are some key aspects of Xen Store:

- **Inter-Domain Communication:** Xen Store enables communication between domains, such as Dom0 (the privileged domain that controls hardware resources) and DomUs (user domains, which are the VMs). This is done through key-value entries, where each domain can read or write information.
- **Configuration Management:** It is used to store and access configuration information, such as virtual devices, networking, and boot parameters. This facilitates the dynamic management and configuration of VMs.
- **Events and Notifications:** Xen Store also supports event notifications. When a particular key or value in the Xen Store is modified, interested domains can be notified to react to these changes. This is useful for monitoring and managing resources.
- **Simple API:** Xen Store provides a simple API for reading and writing data, making it easy for developers to integrate their applications with the Xen virtualization system.

XAPI

XAPI, or XenAPI, is the application programming interface (API) used to manage the Xen Hypervisor and its virtual machines (VMs). XAPI is a key component of XenServer (now known as Citrix Hypervisor) and provides a standardized way to interact with the Xen hypervisor to perform operations such as creating, configuring, monitoring, and controlling VMs.

Here are some important aspects of XAPI:

- **VM Management:** XAPI allows administrators to programmatically create, delete, start, and stop virtual machines.
- **Automation:** With XAPI, it's possible to automate the management of virtual resources, including networking, storage, and computing, which is crucial for large cloud environments.
- **Integration:** XAPI can be integrated with other tools and scripts to provide more efficient and customized administration of the Xen environment.
- **Access Control:** XAPI also provides access control mechanisms to ensure that only authorized users can perform specific operations in the virtual environment.

XAPI is the interface that enables control and automation of the Xen Hypervisor, making it easier to manage virtualized environments.



Xen Summary

- **Xen:** The core hypervisor technology enabling virtual machines to run on physical hardware.
- **XenSource:** The company that commercialized Xen, later acquired by Citrix, leading to the development of Citrix XenServer.
- **Xen Project:** The open-source initiative and community that continues to develop and maintain the Xen hypervisor under the Linux Foundation.
- **XenStore:** Xen Store acts as a communication and configuration intermediary between the Xen hypervisor and the VMs, streamlining the operation and management of virtualized environments.
- **XAPI** is the interface that enables control and automation of the Xen Hypervisor, making it easier to manage virtualized environments.



Domain0 (Dom0)

Domain0, or Dom0, is the control domain in a Xen architecture. It manages other domains (DomUs) and has direct access to hardware. Dom0 runs device drivers, allowing DomUs, which lack direct hardware access, to communicate with devices. Typically, it is a full instance of an operating system, like Linux, and is essential for Xen hypervisor operation.



DomainU (DomU)

DomUs are non-privileged domains that run virtual machines. They are managed by Dom0 and do not have direct access to hardware. DomUs can be configured to run different operating systems and are used for various purposes, such as application servers and development environments. They rely on Dom0 for hardware interaction.



PV-DomU (Paravirtualized DomainU)

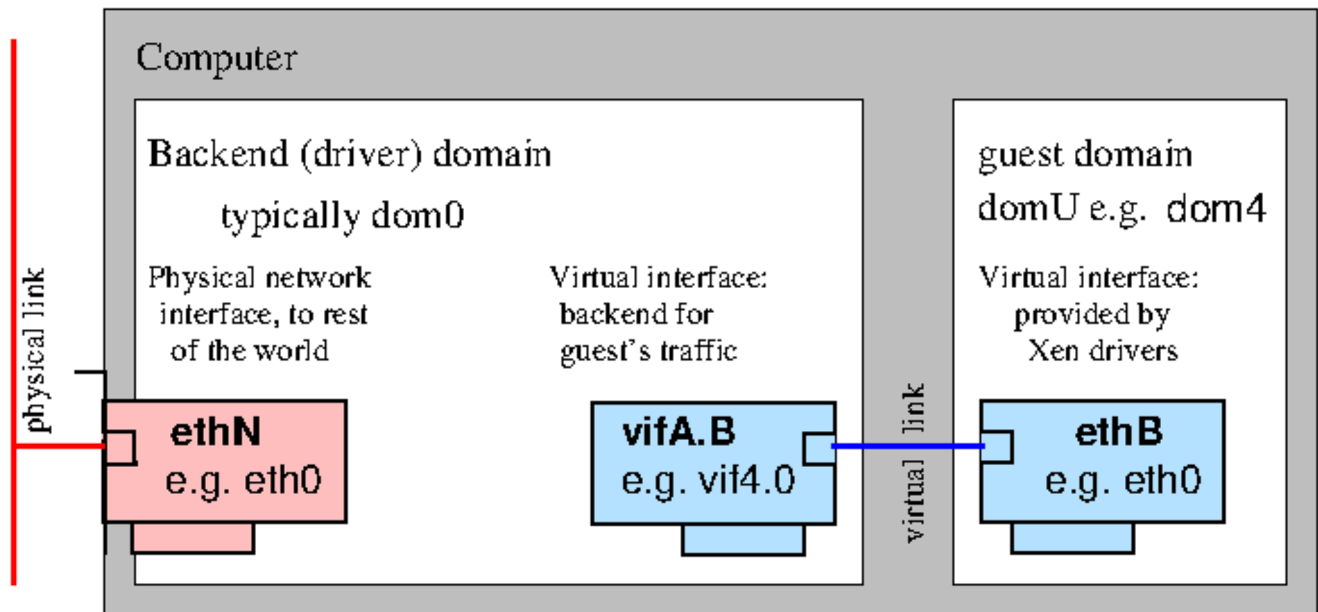
PV-DomUs use a technique called paravirtualization. In this model, the DomU operating system is modified to be aware that it runs in a virtualized environment, allowing it to communicate directly with the hypervisor for optimized performance. This results in lower overhead and better efficiency compared to full virtualization.

⚙️ HVM-DomU (Hardware Virtual Machine DomainU)

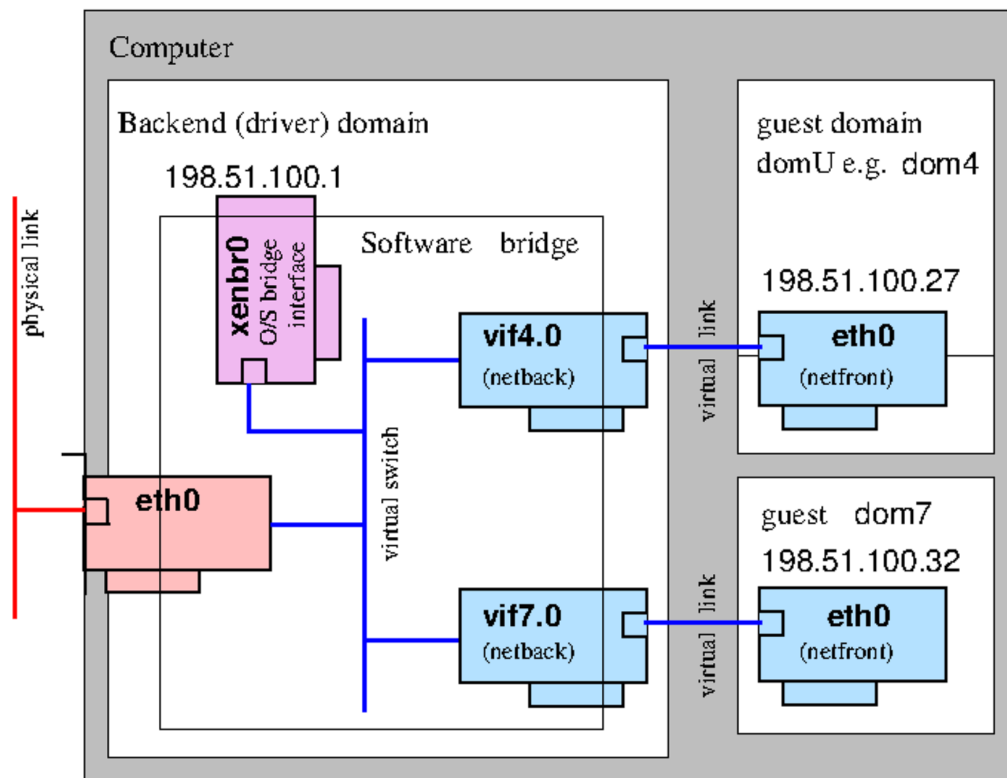
HVM-DomUs are virtual machines that utilize full virtualization, allowing unmodified operating systems to run. The Xen hypervisor provides hardware emulation for these DomUs, enabling them to run any operating system that supports the underlying hardware architecture. While this offers greater flexibility, it can result in higher overhead compared to PV-DomUs.

🌐 Xen Network

Paravirtualized Network Devices



Bridging



351.2 Cited Objects

```
Domain0 (Dom0), DomainU (DomU)
PV-DomU, HVM-DomU
/etc/xen/
xl
xl.cfg
xl.conf # Xen global configurations
xentop
oxenstored # Xenstore configurations
```

351.2 Notes

```
# Xen Settings
/etc/xen/
/etc/xen/xl.conf - Main general configuration file for Xen
/etc/xen/oxenstored.conf - Xenstore configurations

# VM Configurations
/etc/xen/xlexample.pvlinux
/etc/xen/xlexample.hvm

# Service Configurations
/etc/default/xen
/etc/default/xendomains

# xen-tools configurations
/etc/xen-tools/
/usr/share/xen-tools/

# docs
xl(1)
xl.conf(5)
xlcpupool.cfg(5)
xl-disk-configuration(5)
xl-network-configuration(5)
xen-tscmode(7)

# initialized domains auto
/etc/default/xendomains
    XENDOMAINS_AUTO=/etc/xen/auto

/etc/xen/auto/

# set domain for up after xen reboot
## create folder auto
cd /etc/xen && mkdir -p auto && cd auto

# create symbolic link
ln -s /etc/xen/lpic3-pv-guest /etc/xen/auto/lpic3-pv-guest
```

vif

In Xen, “vif” stands for Virtual Interface and is used to configure networking for virtual machines (domains).

By specifying “vif” directives in the domain configuration files, administrators can define network interfaces, assign IP addresses, set up VLANs, and configure other networking parameters for virtual machines running on Xen hosts. For example: `vif = ['bridge=xenbr0']`, in this case, it connects the VM’s network interface to the Xen bridge named “xenbr0”.

Xen Lab

Use this script for lab provisioning: `xen.sh`

351.2 Important Commands

xen-create-image

```
# create a pv image
xen-create-image \
  --hostname=lpic3-pv-guest \
  --memory=1gb \
  --vcpus=2 \
  --lvm=vg_xen \
  --bridge=xenbr0 \
  --dhcp \
  --pygrub \
  --password=vagrant \
  --dist=bookworm
```

xen-list-images

```
# list image
xen-list-images
```

xen-delete-image

```
# delete a pv image
xen-delete-image lpic3-pv-guest --lvm=vg_xen
```

xenstore-ls

```
# list xenstore infos
sudo xenstore-ls
```

```
# xl requires privileged access to Xen control interfaces (Dom0)
# run all xl operations with sudo

# view xen information
sudo xl info

# list Domains
sudo xl list
sudo xl list lpic3-hvm-guest
sudo xl list lpic3-hvm-guest -l

# create DomainU - virtual machine
sudo xl create /etc/xen/lpic3-pv-guest.cfg

# create DomainU virtual machine and connect to guest
sudo xl create -c /etc/xen/lpic3-pv-guest.cfg

# uptime Domains
sudo xl uptime

# pause Domain
sudo xl pause 2
sudo xl pause lpic3-hvm-guest

# save state Domains
sudo xl -v save lpic3-hvm-guest ~root/image-lpic3-hvm-guest.save

# restore Domain
sudo xl restore /root/image-lpic3-hvm-guest.save

# get Domain name
sudo xl domname 2

# view dmesg information
sudo xl dmesg

# monitoring domain
sudo xl top
xentop

# Limit mem Dom0
sudo xl mem-set 0 2048

# Limit cpu (not permanent after boot)
sudo xl vcpu-set 0 2

##-----
# create DomainU virtual machine HVM

## create logical volume
lvcreate -l +20%FREE -n lpic3-hvm-guest-disk vg_xen

## optional: create a ssh tunnel for VNC (secondary view only)
ssh -l vagrant -L 5900:localhost:5900 192.168.0.130

## extract installer kernel/initrd from ISO (one-time)
mkdir -p /var/lib/xen/boot/debian12 /mnt/debian-iso
mount -o loop /home/vagrant/isos/debian/debian-12.8.0-amd64-DVD-1.iso /mnt/debian-iso
cp -f /mnt/debian-iso/install.amd/vmlinuz /var/lib/xen/boot/debian12/
```

```

cp -f /mnt/debian-iso/install.amd/initrd.gz /var/lib/xen/boot/debian12/
umount /mnt/debian-iso

## start HTTP server to publish preseed.cfg
cd configs/xen/hvm/debian
python3 -m http.server 8000

## optional: validate that preseed.cfg is reachable
curl http://192.168.0.130:8000/preseed.cfg

## create HVM domain and attach to the Xen serial console
sudo xl create configs/xen/hvm/debian/lpic3-hvm-guest-debian-auto.cfg -c

## reconnect later, if needed
sudo xl console <id_or_name>

## note:
## the unattended installer was configured for text mode on ttyS0
## so the active installation appears in xl console, not in VNC
## VNC may show only the guest VGA framebuffer or early boot messages

## after installation finished, destroy domain if still running:
sudo xl destroy <id_or_name>

## create HVM domain from installed disk
1776302514356

## access runtime domain
sudo xl console <id_or_name>
##-----

# connect in domain guest
sudo xl console <id>|<name> (press enter)
sudo xl console 1
sudo xl console lpic3-pv-guest

#How do I exit domU "xl console" session
#Press ctrl+] or if you're using Putty press ctrl+5.

# Poweroff domain
sudo xl shutdown lpic3-pv-guest

# destroy domain
sudo xl destroy lpic3-pv-guest

# reboot domain
sudo xl reboot lpic3-pv-guest

# list block devices
sudo xl block-list 1
sudo xl block-list lpic3-pv-guest

# detach block devices
sudo xl block-detach lpic3-hvm-guest hdc
sudo xl block-detach 2 xvdc

# attach block devices

## hard disk devices
sudo xl block-attach lpic3-hvm-guest-ubuntu 'phy:/dev/vg_xen/lpic3-hvm-guest-disk2,xvde,w'

## cdrom

```

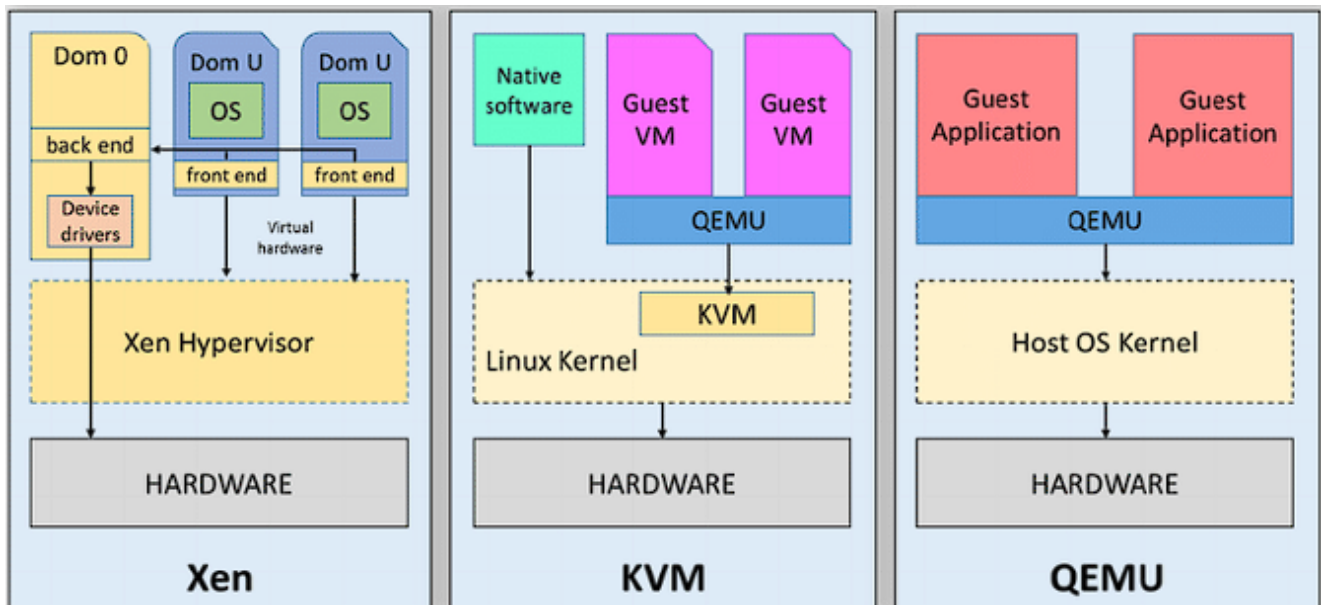
```

sudo xl block-attach lpic3-hvm-guest 'file:/home/vagrant/isos/ubuntu/seed.iso,xvdc:cdrom,r'
sudo xl block-attach 2 'file:/home/vagrant/isos/ubuntu/seed.iso,xvdc:cdrom,r'

# insert and eject cdrom devices
sudo xl cd-insert lpic3-hvm-guest-ubuntu xvdb /home/vagrant/isos/ubuntu/ubuntu-24.04.1-live-server-amd64.iso
sudo xl cd-eject lpic3-hvm-guest-ubuntu xvdb

```

351.3 QEMU



Weight: 4

Description: Candidates should be able to install, configure, maintain, migrate and troubleshoot QEMU installations.

Key Knowledge Areas:

- Understand the architecture of QEMU, including KVM, networking and storage
- Start QEMU instances from the command line
- Manage snapshots using the QEMU monitor
- Install the QEMU Guest Agent and VirtIO device drivers
- Troubleshoot QEMU installations, including networking and storage
- Awareness of important QEMU configuration parameters

351.3 Cited Objects

```
Kernel modules: kvm, kvm-intel and kvm-amd
/dev/kvm
QEMU monitor
qemu
qemu-system-x86_64
ip
brctl
tunctl
```

Understanding KVM and QEMU

In this section, we study virtualization on Linux by understanding the role of each component: what QEMU does, what KVM does, and why both are typically used together in production.

What is QEMU?

QEMU (Quick Emulator) is a userspace virtual machine monitor and hardware emulator. It is responsible for creating and managing the virtual machine process and exposing virtual devices to the guest.

Key responsibilities of QEMU:

- Emulates or virtualizes CPU execution for guest systems.
- Provides virtual hardware (disk controllers, NICs, display, USB, PCI devices, etc.).
- Loads guest firmware/boot media and starts guest boot.
- Handles VM resources and runtime controls (snapshots, monitor commands, migration hooks).
- Supports many machine architectures (x86_64, aarch64, ppc64, and others).

When QEMU runs without hardware acceleration, guest CPU instructions are translated in software (TCG), which is flexible but slower.

What is KVM?

KVM (Kernel-based Virtual Machine) is a Linux kernel virtualization framework. It turns the Linux kernel into a hypervisor by exposing virtualization capabilities through `/dev/kvm`.

Key responsibilities of KVM:

- Uses CPU virtualization extensions (Intel VT-x / AMD-V) for accelerated guest execution. This characteristic is often referred to as hardware-assisted virtualization (HVM).
- Provides kernel-level support for guest privilege transitions (guest mode vs host mode).
- Handles low-level virtualization mechanisms such as vCPU execution and interrupt virtualization.
- Exposes an API to userspace VMMs (like QEMU) via ioctls on `/dev/kvm`.

KVM by itself is not a full VM manager with rich device models; it is the acceleration/hypervisor layer.

How QEMU and KVM Work Together

In the common Linux stack, QEMU is the userspace VMM and KVM is the kernel acceleration backend:

- QEMU creates the VM and virtual devices.
- QEMU asks KVM to run vCPUs with hardware acceleration.
- KVM executes guest CPU code near-native speed.
- QEMU still handles emulated I/O and device models.

Practical result:

- `qemu-system-x86_64` without `-enable-kvm` works as emulation (slower, but useful for cross-arch/labs).
- `qemu-system-x86_64 -enable-kvm` uses hardware acceleration (faster, typical for real workloads).

KVM vs QEMU (Quick Comparison)

Aspect	QEMU	KVM
Type	Userspace VMM / emulator	Kernel virtualization framework
Runs in	User space	Linux kernel
Main role	VM lifecycle + device emulation	CPU/memory virtualization acceleration
Depends on	Can run alone (TCG emulation)	Needs a userspace VMM (commonly QEMU)
Hardware emulation	Yes	No (not a device emulator)
Performance impact	Higher overhead alone	Near-native CPU execution with VT-x/AMD-V
Typical usage	<code>qemu-system-*</code> commands	Enabled through <code>/dev/kvm</code> and <code>-enable-kvm</code>

In short: QEMU gives the VM its virtual machine “shape” (devices and runtime), while KVM gives the VM fast hardware-assisted execution.

351.3 Important Commands

351.3 Others Commands

check kvm module

```
# check if kvm is enabled
egrep -o '(vmx|svm)' /proc/cpuinfo
lscpu |grep Virtualization
lsmod|grep kvm
ls -l /dev/kvm
hostnamectl
systemd-detect-virt
```

```
# check kernel infos
uname -a

# check root device
findmnt /

# mount a qcow2 image
## Example 1:
mkdir -p /mnt/qemu
guestmount -a os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 -i /mnt/qemu/

## Example 2:
sudo guestfish --rw -a os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2
run
list-fileSystems

# run commands in qcow2 images
## Example 1:
virt-customize -a os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 --run-command 'echo hello
>/root/hello.txt'
## Example 2:
sudo virt-customize -a os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 \
--run-command 'echo -e "auto ens3\niface ens3 inet dhcp" >
/etc/network/interfaces.d/ens3.cfg'

# generate mac
printf 'DE:AD:BE:EF:%02X:%02X\n' $((RANDOM%256)) $((RANDOM%256))
```



```
# list links
ip link show

# create bridge
ip link add br0 type bridge
```



```
# list bridges
brctl show

# create bridge
brctl addbr br0
ip link set br0 up

# add interface to bridge
brctl addif br0 eth0

# remove interface / remove bridge
brctl delif br0 eth0
ip link set br0 down
brctl delbr br0
```

qemu-img

```
# create image
qemu-img create -f qcow2 vm-disk-debian-12.qcow2 20G

# convert vmdk to qcow2 image
qemu-img convert \
  -f vmdk \
  -O qcow2 os-images/Debian_12.0.0_VMM/Debian_12.0.0_VMM_LinuxVMImages.COM.vmdk os-
images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 \
  -p \
  -m16

# check image
qemu-img info os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2
```

```
# create vm with ISO
qemu-system-x86_64 \
  -name lpic3-debian-12 \
  -enable-kvm -hda vm-disk-debian-12.qcow2 \
  -cdrom /home/vagrant/isos/debian/debian-12.8.0-amd64-DVD-1.iso \
  -boot d \
  -m 2048 \
  -smp cpus=2 \
  -k pt-br

# create vm with ISO using vnc in no gui servers \ ssh connections

## create ssh tunnel in host - for use VNC viewer in local machine with -vnc :2
ssh -l vagrant -L 5902:localhost:5902 192.168.0.131

## create vm
qemu-system-x86_64 \
  -name lpic3-debian-12 \
  -enable-kvm \
  -m 2048 \
  -smp cpus=2 \
  -k pt-br \
  -vnc :2 \
  -device qemu-xhci \
  -device usb-tablet \
  -device ide-cd,bus=ide.1,drive=cdrom,bootindex=1 \
  -drive id=cdrom,media=cdrom,if=none,file=/home/vagrant/isos/debian/debian-12.8.0-amd64-DVD-1.iso \
  -hda vm-disk-debian-12.qcow2 \
  -boot order=d \
  -vga std \
  -display none \
  -monitor stdio

# create vm with OS Image - qcow2

## create vm
qemu-system-x86_64 \
  -name lpic3-debian-12 \
  -enable-kvm \
  -m 2048 \
  -smp cpus=2 \
  -k pt-br \
  -vnc :2 \
  -hda os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2

## create vm with custom kernel params
qemu-system-x86_64 \
  -name lpic3-debian-12 \
  -kernel /vmlinuz \
  -initrd /initrd.img \
  -append "root=/dev/mapper/debian--vg-root ro fastboot console=ttyS0" \
  -enable-kvm \
  -m 2048 \
  -smp cpus=2 \
  -k pt-br \
  -vnc :2 \
  -hda os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2
```

```

## create vm with and attach disk
qemu-system-x86_64 \
-name lpic3-debian-12 \
-enable-kvm \
-m 2048 \
-smp cpus=2 \
-vnc :2 \
-hda os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 \
-hdb vmdisk-debian12.qcow2 \
-drive file=vmdisk-extra-debian12.qcow2,index=2,media=disk,if=ide \
-netdev bridge,id=net0,br=qemubr0 \
-device virtio-net-pci,netdev=net0

## create vm network netdev user
qemu-system-x86_64 \
-name lpic3-debian-12 \
-enable-kvm \
-m 2048 \
-smp cpus=2 \
-vnc :2 \
-hda os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 \
-netdev user,id=mynet0,net=192.168.0.150/24,dhcpstart=192.168.0.155,hostfwd=tcp::2222-:22 \
-device virtio-net-pci,netdev=mynet0

## create vm network netdev tap (Private Network)
ip link add br0 type bridge ; ifconfig br0 up
qemu-system-x86_64 \
-name lpic3-debian-12 \
-enable-kvm \
-m 2048 \
-smp cpus=2 \
-vnc :2 \
-hda os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 \
-netdev tap,id=br0 \
-device e1000,netdev=br0,mac=DE:AD:BE:EF:1A:24

## create vm with public bridge
#create a public bridge : https://www.linux-kvm.org/page/Networking

qemu-system-x86_64 \
-name lpic3-debian-12 \
-enable-kvm \
-m 2048 \
-smp cpus=2 \
-hda os-images/Debian_12.0.0_VMM/Debian_12.0.0.qcow2 \
-k pt-br \
-vnc :2 \
-device qemu-xhci \
-device usb-tablet \
-vga std \
-display none \
-netdev bridge,id=net0,br=qemubr0 \
-device virtio-net-pci,netdev=net0

## get a ipv4 ip - open ssh in vm and:
dhcpcd ens4

```

QEMU Monitor

For initiate QEMU monitor in commandline use **-monitor stdio** param in **qemu-system-x86_64**

```
qemu-system-x86_64 -monitor stdio
```

Exit qemu-monitor:

```
ctrl+alt+2
```

```
# Management
info status # vm info
info cpus # cpu information
info network # network informations
stop # pause vm
cont # start vm in status pause
system_powerdown # poweroff vm
system_reset # restart monitor

# Blocks
info block # block info
boot_set d # force boot iso
change id1-cd0 /home/vagrant/isos/debian/debian-12.8.0-amd64-DVD-1.iso # attach cdrom
eject id1-cd0 # detach cdrom

# Snapshots
info snapshots # list snapshots
savevm snapshot-01 # create snapshot
loadvm snapshot-01 # restore snapshot
delvm snapshot-01
```

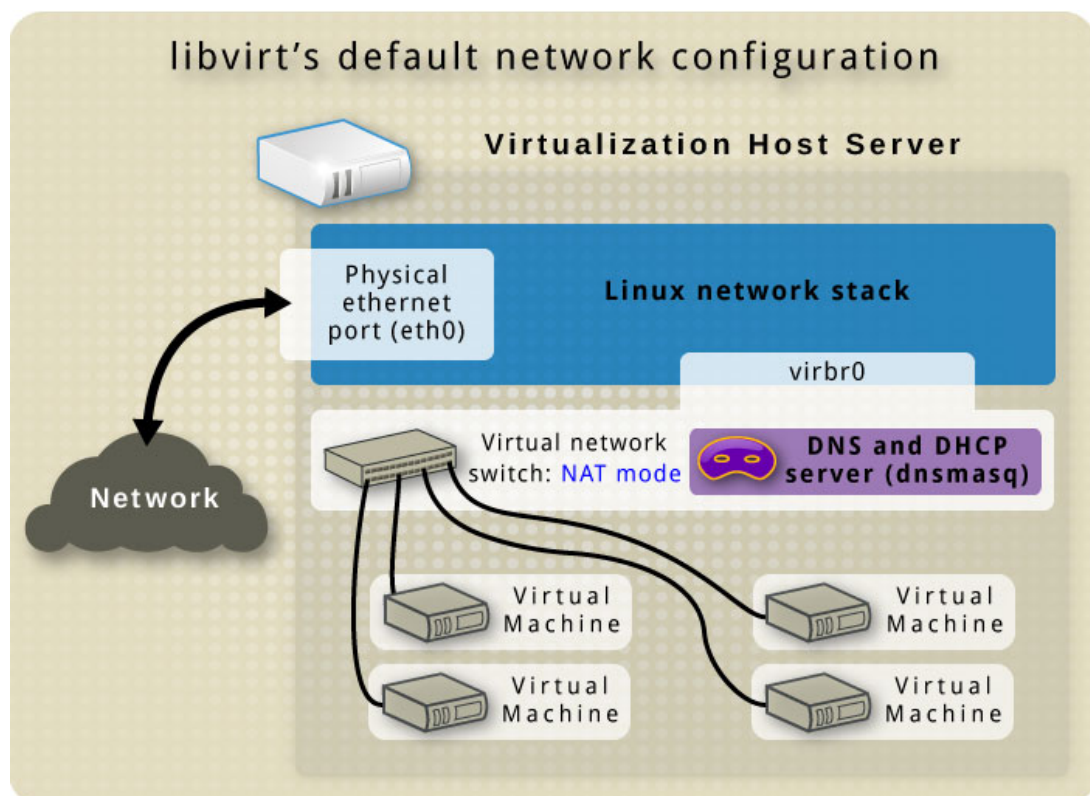
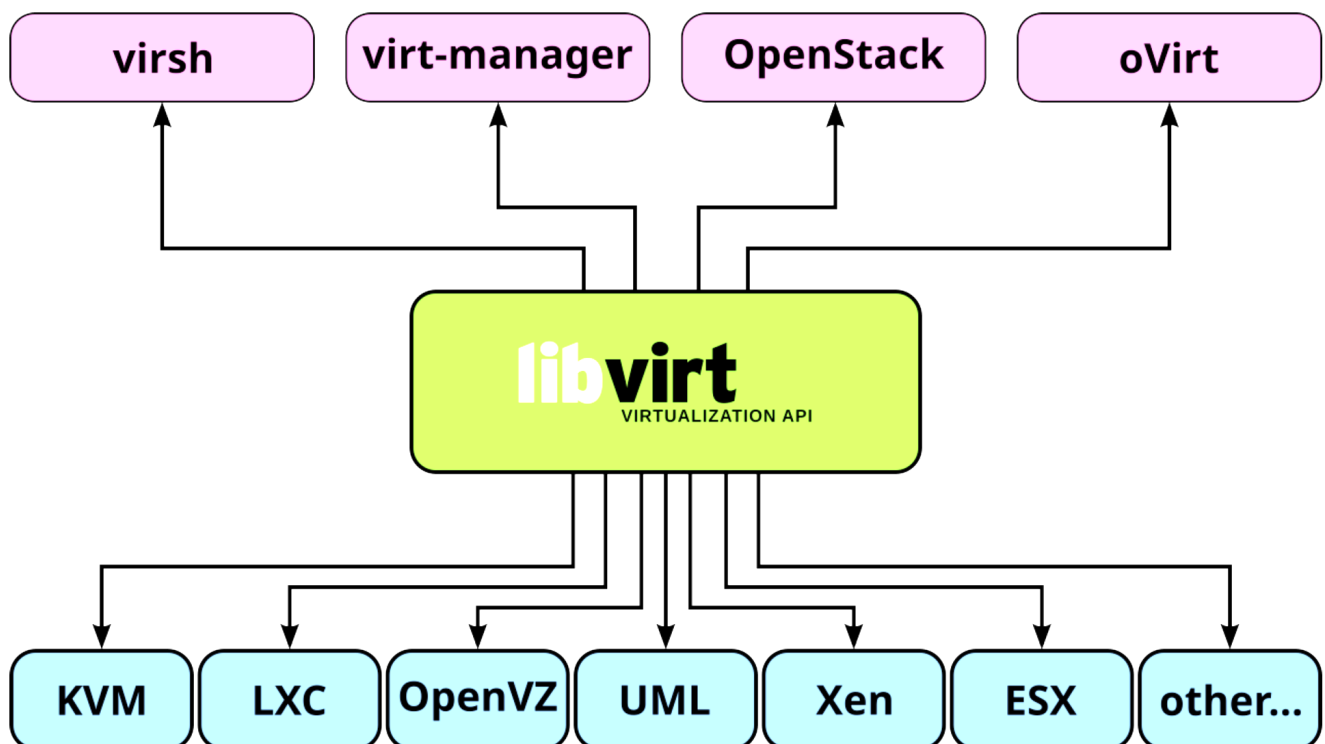


Guest Agent

For enable, use:

```
qemu-system-x86_64 \
-chardev socket,path=/tmp/qga.sock,server=on,wait=off,id=qga0 \
-device virtio-serial \
-device virtserialport,chardev=qga0,name=org.qemu.guest_agent.0
```

351.4 Libvirt Virtual Machine Management



Weight: 9

Description: Candidates should be able to manage virtualization hosts and virtual machines ('libvirt domains') using libvirt and related tools.

Key Knowledge Areas:

- Understand the architecture of libvirt
- Manage libvirt connections and nodes
- Create and manage QEMU and Xen domains, including snapshots
- Manage and analyze resource consumption of domains
- Create and manage storage pools and volumes
- Create and manage virtual networks
- Migrate domains between nodes
- Understand how libvirt interacts with Xen and QEMU
- Understand how libvirt interacts with network services such as dnsmasq and radvd
- Understand libvirt XML configuration files
- Awareness of virtlogd and virtlockd

351.4 Cited Objects

```
libvirtd  
/etc/libvirt/  
/var/lib/libvirt  
/var/log/libvirt  
virsh (including relevant subcommands)
```

351.4 Important Commands

virsh

```
# using env variable for set virsh uri (local or remotely)
export LIBVIRT_DEFAULT_URI=qemu:///system
export LIBVIRT_DEFAULT_URI=xen+ssh://vagrant@192.168.0.130
export LIBVIRT_DEFAULT_URI='xen+ssh://vagrant@192.168.0.130?keyfile=/home/vagrant/.ssh/skynet-key-ecdsa'

# COMMONS

# get helps
virsh help
virsh help pool-create

# view version
virsh version

# view system info
sudo virsh sysinfo

# view node info
virsh nodeinfo

# hostname
virsh hostname

# check vcn allocated port
virsh vncdisplay <domain_id>
virsh vncdisplay <domain_name>
virsh vncdisplay rocky9-server01

# HYPERVISORS

# view libvirt hypervisor connection
virsh uri

# list valid hypervisors
virt-host-validate
virt-host-validate qemu

# test connection uri(vm test)
virsh -c test:///default list

# connect remotely
virsh -c xen+ssh://vagrant@192.168.0.130
virsh -c xen+ssh://vagrant@192.168.0.130 list
virsh -c qemu+ssh://vagrant@192.168.0.130/system list

# connect remotely without enter password
virsh -c 'xen+ssh://vagrant@192.168.0.130?keyfile=/home/vagrant/.ssh/skynet-key-ecdsa'

# STORAGE

# list storage pools
virsh pool-list --details

# list all storage pool
virsh pool-list --all --details
```

```

# get a pool configuration
virsh pool-dumpxml default

# get pool info
virsh pool-info default

# create a storage pool
virsh pool-define-as --name default --type dir --target /var/lib/libvirt/images

# create a storage pool with dumpxml
virsh pool-create --overwrite --file configs/kvm/libvirt/pool.xml

# start storage pool
virsh pool-start default

# set storage pool for autostart
virsh pool-autostart default

# stop storage pool
virsh pool-destroy linux

# delete xml storage pool file
virsh pool-undefine linux

# edit storage pool
virsh pool-edit linux

# list volumes
virsh vol-list linux

# get volume infos
virsh vol-info Debian_12.0.0.qcow2 os-images
virsh vol-info --pool os-images Debian_12.0.0.qcow2

# get volume xml
virsh vol-dumpxml rocky9-disk1 default

# create volume
virsh vol-create-as default --format qcow2 disk1 10G

# delete volume
virsh vol-delete disk1 default

# DOMAINS \ INSTANCES \ VIRTUAL MACHINES

# list domain\instance\vm
virsh list
virsh list --all

# create domain\instance\vm
virsh create configs/kvm/libvirt/rocky9-server03.xml

# view domain\instance\vm info
virsh dominfo rocky9-server01

# view domain\instance\vm xml
virsh dumpxml rocky9-server01

# edit domain\instance\vm xml
virsh edit rocky9-server01

```

```

# stop domain\instance\vm
virsh shutdown rocky9-server01 # gracefully
virsh destroy 1
virsh destroy rocky9-server01

# suspend domain\instance\vm
virsh suspend rocky9-server01

# resume domain\instance\vm
virsh resume rocky9-server01

# start domain\instance\vm
virsh start rocky9-server01

# remove domain\instance\vm
virsh undefine rocky9-server01

# remove domain\instance\vm and storage volumes
virsh undefine rocky9-server01 --remove-all-storage

# save domain\instance\vm
virsh save rocky9-server01 rocky9-server01.qcow2

# restore domain\instance\vm
virsh restore rocky9-server01.rocky9-server01.qcow2

# list snapshots
virsh snapshot-list rocky9-server01

# create snapshot
virsh snapshot-create rocky9-server01

# restore snapshot
virsh snapshot-revert rocky9-server01 1748983520

# view snapshot xml
virsh snapshot-info rocky9-server01 1748983520

# dumpxml snapshot
virsh snapshot-dumpxml rocky9-server01 1748983520

# xml snapshot path
/var/lib/libvirt/qemu/snapshot/rocky9-server01/

# view snapshot info
virsh snapshot-info rocky9-server01 1748983671

# edit snapshot
virsh snapshot-edit rocky9-server01 1748983520

# delete snapshot
virsh snapshot-delete rocky9-server01 1748983520

# DEVICES

# list block devices
virsh domblklist rocky9-server01 --details

# add cdrom media
virsh change-media rocky9-server01 sda /home/vagrant/isos/rocky/Rocky-9.5-x86_64-minimal.iso
virsh attach-disk rocky9-server01 /home/vagrant/isos/rocky/Rocky-9.5-x86_64-minimal.iso sda --
type cdrom --mode readonly

```

```

# remove cdrom media
virsh change-media rocky9-server01 sda --eject

# add new disk
virsh attach-disk rocky9-server01 /var/lib/libvirt/images/rocky9-disk2 vdb --persistent

# remove disk
virsh detach-disk rocky9-server01 vdb --persistent

# RESOURCES (CPU and Memory)

# get cpu infos
virsh vcpuinfo rocky9-server01 --pretty
virsh dominfo rocky9-server01 | grep 'CPU'

# get vcpu count
virsh vcpucount rocky9-server01

# set vcpus maximum config
virsh setvcpus rocky9-server01 --count 4 --maximum --config
virsh shutdown rocky9-server01
virsh start rocky9-server01

# set vcpu current config
virsh setvcpus rocky9-server01 --count 4 --config

# set vcpu current live
virsh setvcpus rocky9-server01 --count 3 --current
virsh setvcpus rocky9-server01 --count 3 --live

# configure vcpu affinity config
virsh vcpupin rocky9-server01 0 7 --config
virsh vcpupin rocky9-server01 1 5-6 --config

# configure vcpu affinity current
virsh vcpupin rocky9-server01 0 7
virsh vcpupin rocky9-server01 1 5-6

# set maximum memory config
virsh setmaxmem rocky9-server01 3000000 --config
virsh shutdown rocky9-server01
virsh start rocky9-server01

# set current memory config
virsh setmem rocky9-server01 2500000 --current

# NETWORK

# get network bridges
brctl show

# get iptables rules for libvirt
sudo iptables -L -n -t nat

# list network
virsh net-list --all

# set default network
virsh net-define /etc/libvirt/qemu/networks/default.xml

# get network infos

```

```

virsh net-info default

# get xml network
virsh net-dumpxml default

# xml file
cat /etc/libvirt/qemu/networks/default.xml

# dhcp config
sudo cat /etc/libvirt/qemu/networks/default.xml | grep -A 10 dhcp
sudo cat /var/lib/libvirt/dnsmasq/default.conf

# get domain ip address
virsh net-dhcp-leases default
virsh net-dhcp-leases default --mac 52\:54\:00\:89\:19\:86

# edit network
virsh net-edit default

# get domain network details
virsh domiflist debian-server01

# path for network filter files
/etc/libvirt/nwfilter/

# list network filters
virsh nwfilter-list

# create network filter - block icmp traffic
virsh nwfilter-define block-icmp.xml
# virsh edit Debian-Server
#   <interface type='network'>
#       ...
#       <filterref filter='block-icmp' />
#       ...
#   </interface>
# virsh destroy debian-server01
# virsh start debian-server01

# delete network filter
virsh nwfilter-undefine block-icmp

# get xml network filter
virsh nwfilter-dumpxml block-icmp

```

virt-install

```
# list os variants
virt-install --os-variant list
osinfo-query os

# create domain\instance\vm with iso file
virsh vol-create-as default --format qcow2 rocky9-disk1 20G
virt-install --name rocky9-server01 \
--vcpus 2 \
--cpu host \
--memory 2048 \
--disk vol=default/rocky9-disk1 \
--cdrom /home/vagrant/isos/rocky/Rocky-9.5-x86_64-minimal.iso \
--os-variant=rocky9 \
--graphics vnc,listen=0.0.0.0,port=5905

# create debian domain\instance\vm with qcow2 file
virt-install --name debian-server01 \
--vcpus 2 \
--ram 2048 \
--disk vol=os-images/Debian_12.0.0.qcow2 \
--import \
--osinfo detect=on \
--graphics vnc,listen=0.0.0.0,port=5906 \
--network network=default \
--noautoconsole

# create rocky9 domain\instance\vm with qcow2 file
virt-install --name rocky9-server02 \
--vcpus 2 \
--ram 2048 \
--disk path=os-images/RockyLinux_9.4_VMG/RockyLinux_9.4.qcow2,format=qcow2,bus=virtio \
--import \
--osinfo detect=on \
--graphics vnc,listen=0.0.0.0,port=5907 \
--network bridge=qemubr0,model=virtio \
--noautoconsole

# open domain\instance\vm gui console
virt-viewer debian-server01

# check metadata domain\instance\vm file (if uri is qemu:///system)
less /etc/libvirt/qemu/debian-server01.xml
```

Virtual Machine Disk Images

Format	Hypervisor	Size	Compression	Encryption	Snapshots
IMG	Generic	Static	No	No	No
RAW	Generic	Static	Yes	No	No
VHD	Hyper-V (VMWare, VirtualBox)	Dynamic	Yes	No	Sim
VMDK	Hyper-V	Dynamic	Yes	Yes	Sim
VDI	VirtualBox	Dynamic	Yes	Yes	Sim
QCOW2	QEMU, KVM	Dynamic	Yes	Yes	Sim

Weight: 3

Description: Candidates should be able to manage virtual machines disk images. This includes converting disk images between various formats and hypervisors and accessing data stored within an image.

Key Knowledge Areas:

- Understand features of various virtual disk image formats, such as raw images, qcow2 and VMDK
- Manage virtual machine disk images using qemu-img
- Mount partitions and access files contained in virtual machine disk images using libguestfish
- Copy physical disk content to a virtual machine disk image
- Migrate disk content between various virtual machine disk image formats
- Awareness of Open Virtualization Format (OVF)

351.5 Cited Objects

```
qemu-img
guestfish (including relevant subcommands)
guestmount
guestumount
virt-cat
virt-copy-in
virt-copy-out
virt-diff
virt-inspector
virt-filesystems
virt-rescue
virt-df
virt-sparsify
virt-p2v
virt-p2v-make-disk
virt-v2v
```

351.5 Important Commands

351.5.1 qemu-img

```
# Display detailed information about a disk image
qemu-img info UbuntuServer_24.04.qcow2

# Create a new 22G raw disk image (default format is raw)
qemu-img create new-disk 22G

# Create a new 22G disk image in qcow2 format
qemu-img create -f qcow2 new-disk2 22G

# Convert a VDI image to raw format using 5 threads and show progress
qemu-img convert -f vdi -O raw Ubuntu-Server.vdk new-Ubuntu.raw -m5 -p

# Convert vmdk to qcow2 image
qemu-img convert \
-f vmdk \
-O qcow2 os-images/UbuntuServer_24.04_VM/UbuntuServer_24.04_VM_LinuxVMImages.COM.vmdk \
os-images/UbuntuServer_24.04_VM/UbuntuServer_24.04.qcow2 \
-p \
-m16

# Resize a raw image to 30G
qemu-img resize -f raw new-disk 30G

# Resize a qcow2 image to 15G(actual size 30Gdisk 30G)
qemu-img resize -f raw --shrink new-disk 15G

# Snapshots

# List all snapshots in the image
qemu-img snapshot -l new-disk2.qcow2

# Create a snapshot named SNAP1
qemu-img snapshot -c SNAP1 disk

# Apply a snapshot by ID or name
qemu-img snapshot -a 123456789 disk

# Delete the snapshot named SNAP1
qemu-img snapshot -d SNAP1 disk
```

```
# set environment variables for guestfish
export LIBGUESTFS_BACKEND_SETTINGS=force_tcg

# Launch guestfish with a disk image
guestfish -a UbuntuServer_24.04.qcow2
#run
#list-partitions

# Run the commands in a script file
guestfish -a UbuntuServer_24.04.qcow2 -m /dev/sda -i < script.ssh

# Interactively run commands
guestfish --rw -a UbuntuServer_24.04.qcow2 <<'EOF'
run
list-filesystems
EOF

# Copy a file from the guest image to the host
export LIBGUESTFS_BACKEND_SETTINGS=force_tcg
sudo guestfish --rw -a UbuntuServer_24.04.qcow2 -i <<'EOF'
copy-out /etc/hostname /tmp/
EOF

# Copy a file from the host into the guest image
echo "new-hostname" > /tmp/hostname
export LIBGUESTFS_BACKEND_SETTINGS=force_tcg
sudo guestfish --rw -a UbuntuServer_24.04.qcow2 -i <<'EOF'
copy-in /tmp/hostname /etc/
EOF

# View contents of a file in the guest image
guestfish --ro -a UbuntuServer_24.04.qcow2 -i <<'EOF'
cat /etc/hostname
EOF

# List files in the guest image
export LIBGUESTFS_BACKEND_SETTINGS=force_tcg
guestfish --rw -a UbuntuServer_24.04.qcow2 -i <<'EOF'
ls /home/ubuntu
EOF

# Edit a file in the guest image
export LIBGUESTFS_BACKEND_SETTINGS=force_tcg
guestfish --rw -a UbuntuServer_24.04.qcow2 -i <<'EOF'
edit /etc/hosts
EOF
```

guestmount

```
# Mount a disk image to a directory
guestmount -a UbuntuServer_24.04.qcow2 -m /dev/ubuntuv/ubuntuv-lv /mnt/ubuntuv
# domain
guestmount -d rocky9-server02 -m /dev/ubuntuv/ubuntuv-lv /mnt/ubuntuv

# Mount a specific partition from a disk image
guestmount -a UbuntuServer_24.04.qcow2 -m /dev/sda2 /mnt/ubuntuv
# domain
guestmount -d debian-server01 --ro -m /dev/debianvg/root /mnt/debian
```

guestumount

```
# Umount a disk image to a directory
sudo guestumount /mnt/ubuntuv
```

virt-df

```
# Show free and used space on virtual machine filesystems
virt-df UbuntuServer_24.04.qcow2 -h
virt-df -d rocky9-server02 -h
```

virt-filesystems

```
# List filesystems, partitions, and logical volumes in a VM disk image (disk image)
virt-filesystems -a UbuntuServer_24.04.qcow2 --all --long -h

# List filesystems, partitions, and logical volumes in a VM disk image (domain)
virt-filesystems -d debian-server01 --all --long -h
```

virt-inspector

```
# Inspect and report on the operating system in a VM disk image
virt-inspector -a UbuntuServer_24.04.qcow2 #(disk)
virt-inspector -d debian-server01 #(domain)
```

virt-cat

```
# Display the contents of a file inside a VM disk image
virt-cat -a UbuntuServer_24.04.qcow2 /etc/hosts
virt-cat -d debian-server01 /etc/hosts #(domain)
```

virt-diff

```
# Show differences between two VM disk images
virt-diff -a UbuntuServer_24.04.qcow2 -A Rocky-Linux.qcow2
```

virt-sparsify

```
# Make a VM disk image smaller by removing unused space
virt-sparsify UbuntuServer_24.04.qcow2 UbuntuServer_24.04-sparse.qcow2
```

virt-resize

```
# Resize a VM disk image or its partitions
virt-filesystems -a UbuntuServer_24.04.qcow2 --all --long -h #(check size of partitions)
qemu-img create -f qcow2 UbuntuServer_24.04-expanded.qcow2 100G #(create new disk image with 100G)
virt-resize --expand /dev/ubuntu-vg/ubuntu-lv \
UbuntuServer_24.04.qcow2 UbuntuServer_24.04-expanded.qcow2
```

virt-copy-in

```
# Copy files from the host into a VM disk image

virt-copy-in -a UbuntuServer_24.04.qcow2 ~vagrant/test-virt-copy-in.txt /home/ubuntu
```

virt-copy-out

```
# Copy files from a VM disk image to the host
virt-copy-out -a UbuntuServer_24.04.qcow2 /home/ubuntu/.bashrc /tmp
```

virt-ls

```
# List files and directories inside a VM disk image
virt-ls -a UbuntuServer_24.04.qcow2 /home/ubuntu
```

virt-rescue

```
# Launch a rescue shell on a VM disk image for recovery
virt-rescue -a UbuntuServer_24.04.qcow2
```

virt-sysprep

```
# Prepare a VM disk image for cloning by removing system-specific data
virt-sysprep -a UbuntuServer_24.04.qcow2
```

virt-v2v

```
# Convert a VM from a foreign hypervisor to run on KVM
virt-v2v -i disk input-disk.img -o local -os /var/tmp
```

virt-p2v

```
# Convert a physical machine to use KVM
```

virt-p2v-make-disk

```
# Create a bootable disk image for physical to virtual conversion
sudo virt-p2v-make-disk -o output.img
```

351.5 Notes

OVF: Open Virtualization Format

OVF: An open format that defines a standard for packaging and distributing virtual machines across different environments.

The generated package has the .ova extension and contains the following files:

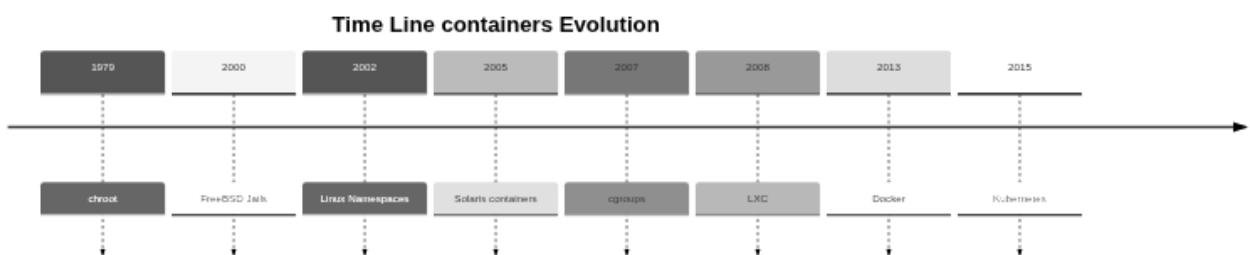
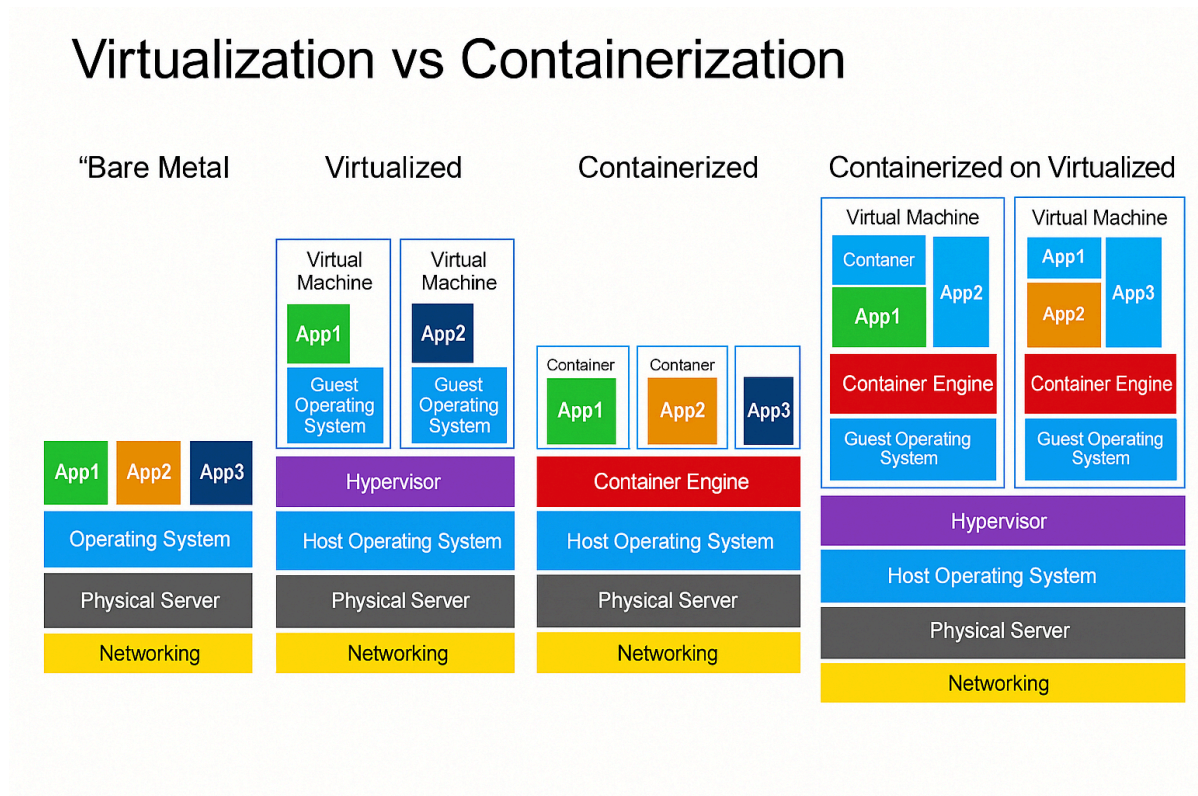
- .ovf: XML file with metadata defining the virtual machine environment
- Image files: .vmdk, .vhd, .vhdx, .qcow2, .raw
- Additional files: metadata, snapshots, configuration, hash



Topic 352: container Virtualization



352.1 container Virtualization Concepts



Weight: 7

Description: Candidates should understand the concept of container virtualization. This includes understanding the Linux components used to implement container virtualization as well as using standard Linux tools to troubleshoot these components.

Key Knowledge Areas:

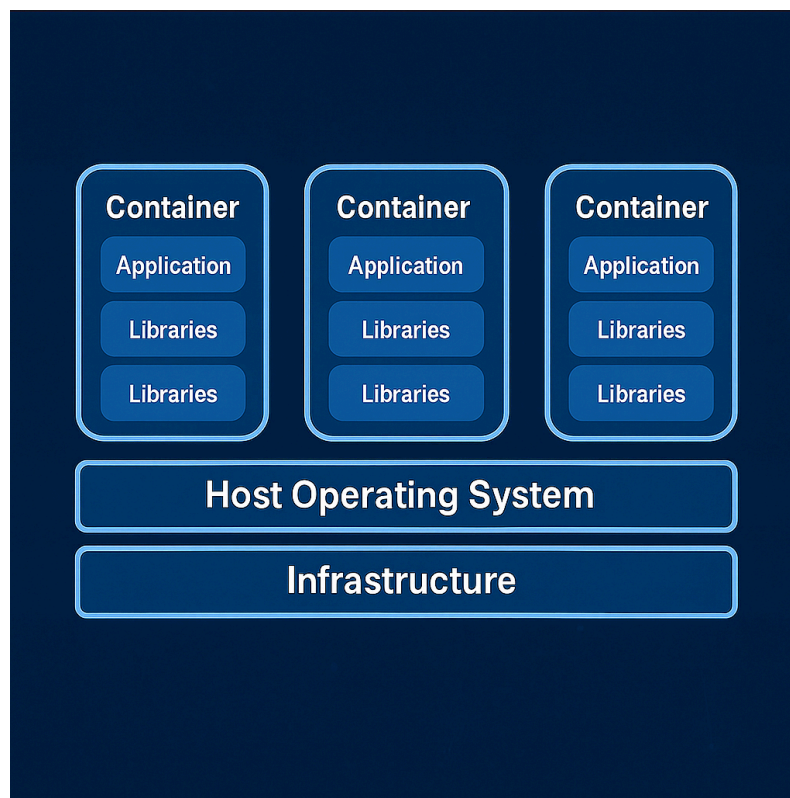
- Understand the concepts of system and application container

- Understand and analyze kernel namespaces
- Understand and analyze control groups
- Understand and analyze capabilities
- Understand the role of seccomp, SELinux and AppArmor for container virtualization
- Understand how LXC and Docker leverage namespaces, cgroups, capabilities, seccomp and MAC(Mandatory Access Control [SELinux,AppArmor])
- Understand the principle of runc
- Understand the principle of CRI-O and containerd
- Awareness of the OCI runtime and image specifications
- Awareness of the Kubernetes container Runtime Interface (CRI)
- Awareness of podman, buildah and skopeo
- Awareness of other container virtualization approaches in Linux and other free operating systems, such as rkt, OpenVZ, systemd-nspawn or BSD Jails

352.1 Cited Objects

```
nsenter
unshare
ip (including relevant subcommands)
capsh
/sys/fs/cgroups
/proc/[0-9]+/ns
/proc/[0-9]+/status
```

Understanding containers



containers are a lightweight virtualization technology that package applications along with their required dependencies — code, libraries, environment variables, and configuration files — into isolated, portable, and reproducible units.

In simple terms: a container is a self-contained box that runs your application the same way, anywhere.

What Is a container?

Unlike Virtual Machines (VMs), containers do not virtualize hardware. Instead, they virtualize the operating system. containers share the same Linux kernel with the host, but each one operates in a fully isolated user space.

containers vs Virtual Machines:

Feature	containers	Virtual Machines
OS Kernel	Shared with host	Each VM has its own OS
Startup time	Fast (seconds or less)	Slow (minutes)
Image size	Lightweight (MBs)	Heavy (GBs)
Resource efficiency	High	Lower
Isolation mechanism	Kernel features (namespaces)	Hypervisor

Key Characteristics of containers

- ♦ **Lightweight:** Share the host OS kernel, reducing overhead and enabling fast startup.
- ♦ **Portable:** Run consistently across different environments (dev, staging, prod, cloud, on-prem).
- ♦ **Isolated:** Use namespaces for process, network, and filesystem isolation.
- ♦ **Efficient:** Enable higher density and better resource utilization than traditional VMs.
- ♦ **Scalable:** Perfect fit for microservices and cloud-native architecture.

Types of containers

1. System containers

- Designed to run the entire OS, Resemble virtual machines.
- Support multiple processes and system services (init, syslog).
- Ideal for legacy or monolithic applications.
- Example: LXC, libvirt-lxc.

2. Application containers

- Designed to run a single process.
- Stateless, ephemeral, and horizontally scalable.
- Used widely in modern DevOps and Kubernetes environments.
- Example: Docker, containerd, CRI-O.

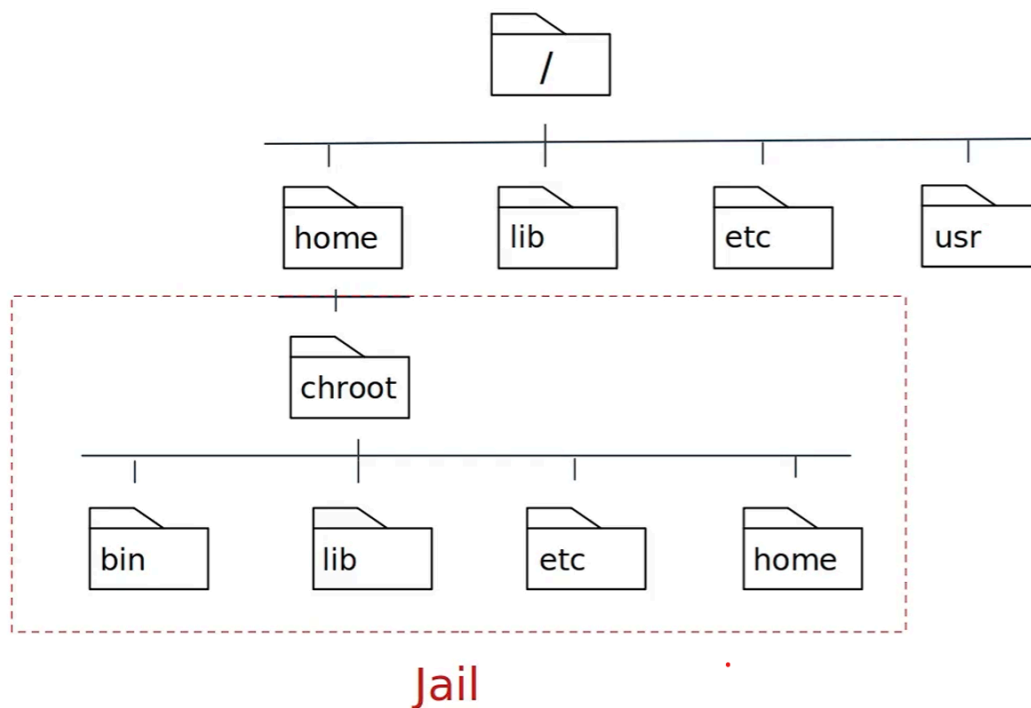
Popular container Runtimes

Runtime	Description
Docker	Most widely adopted CLI/daemon for building and running containers.
containerd	Lightweight runtime powering Docker and Kubernetes.
CRI-O	Kubernetes-native runtime for OCI containers.
LXC	Traditional Linux system containers, closer to full OS.
RKT	Security-focused runtime (deprecated).

🔒 container Internals and Security Elements

Component	Role
Namespaces	Isolate processes, users, mounts, networks.
cgroups	Control and limit resource usage (CPU, memory, IO).
Capabilities	Fine-grained privilege control inside containers.
seccomp	Restricts allowed syscalls to reduce attack surface.
AppArmor / SELinux	Mandatory Access Control enforcement at kernel level.

🧠 Understanding chroot - Change Root Directory in Unix/Linux



What is chroot?

chroot (short for change root) is a system call and command on Unix-like operating systems that changes the apparent root directory (/) for the current running process and its children. This creates an isolated environment, commonly referred to as a chroot jail.

🧱 Purpose and Use Cases

- 🔒 Isolate applications for security (jailing).
- 🛠 Create testing environments without impacting the rest of the system.
- 🛠 System recovery (e.g., boot into LiveCD and chroot into installed system).
- 📦 Building software packages in a controlled environment.

Minimum Required Structure

The chroot environment must have its own essential files and structure:

```
/mnt/myenv/  
├── bin/  
│   └── bash  
├── etc/  
├── lib/  
├── lib64/  
├── usr/  
├── dev/  
├── proc/  
└── tmp/
```

Use ldd to identify required libraries:

```
ldd /bin/bash
```

Limitations and Security Considerations

- chroot is not a security boundary like containers or VMs.
- A privileged user (root) inside the jail can potentially break out.
- No isolation of process namespaces, devices, or kernel-level resources.

For stronger isolation, consider alternatives like:

- Linux containers (LXC, Docker)
- Virtual machines (KVM, QEMU)
- Kernel namespaces and cgroups

Test chroot bash

```
# create chroot environment
sudo mkdir -p /mnt/chroot/{bin,lib,lib64,usr,dev,proc,sys,run,tmp}
sudo chmod 1777 /mnt/chroot/tmp

# copy bash and its dependencies
sudo cp /bin/bash /mnt/chroot/bin/

# find dependencies of bash and copy them to chroot
ldd /bin/bash

# copy dependencies to chroot
# ldd /bin/bash | awk '{print $3}' | grep '^/' | xargs -I '{}' sudo cp -v '{}'
# /mnt/chroot/{}'
sudo cp /lib/x86_64-linux-gnu/libtinfo.so.6 /mnt/chroot/lib/x86_64-linux-gnu/
sudo cp /lib/x86_64-linux-gnu/libc.so.6 /mnt/chroot/lib/x86_64-linux-gnu/
sudo cp /lib64/ld-linux-x86-64.so.2 /mnt/chroot/lib64/

# mount necessary filesystems
sudo mount --bind /dev /mnt/chroot/dev
sudo mount -t proc proc /mnt/chroot/proc
sudo mount -t sysfs sys /mnt/chroot/sys
sudo mount -t tmpfs tmpfs /mnt/chroot/run

# enter chroot environment
sudo chroot /mnt/chroot /bin/bash

# test chroot
echo "Hello from chroot!"
pwd

# umount filesystems after exit
sudo umount /mnt/chroot/run
sudo umount /mnt/chroot/sys
sudo umount /mnt/chroot/proc
sudo umount /mnt/chroot/dev

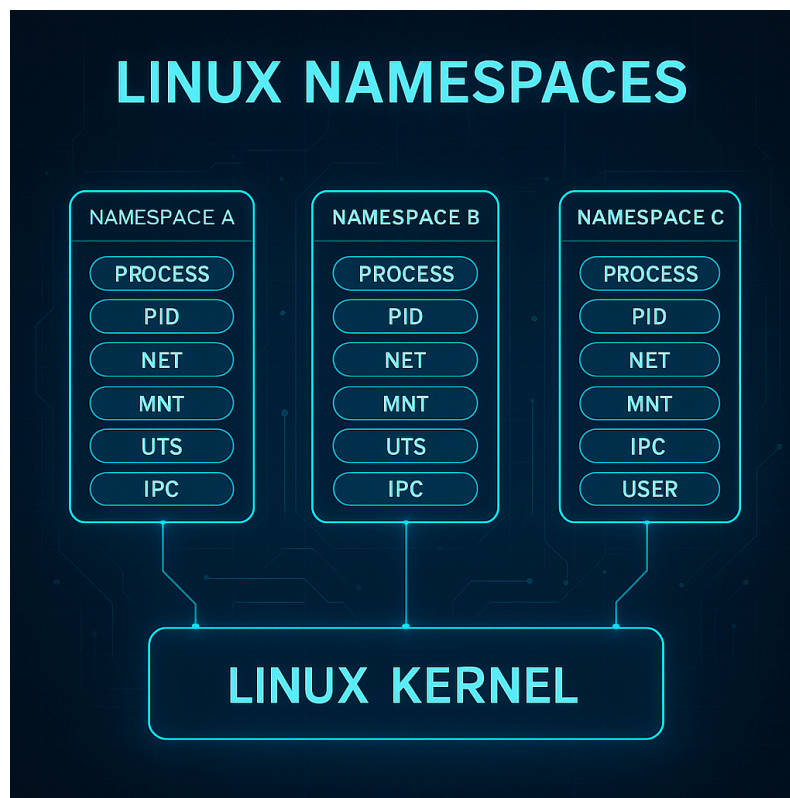
# delete chroot environment
sudo rm -rf /mnt/chroot
```

Test chroot with debootstrap

```
# download debian files
sudo debootstrap stable ~vagrant/debian http://deb.debian.org/debian
sudo chroot ~vagrant/debian bash
```

Lab chroot

Use this script for lab: chroot.sh



Namespaces are a core Linux kernel feature that enable process-level isolation. They create separate “views” of global system resources — such as process IDs, networking, filesystems, and users — so that each process group believes it is running in its own system.

In simple terms: namespaces trick a process into thinking it owns the machine, even though it’s just sharing it.

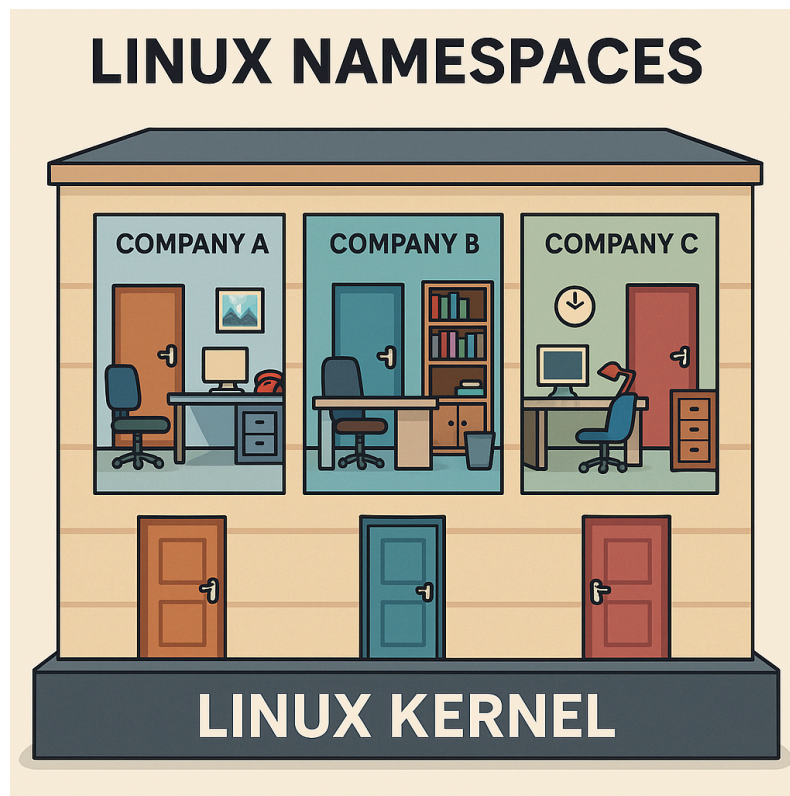
This is the foundation for container isolation.

What Do Namespaces Isolate?

Each namespace type isolates a specific system resource. Together, they make up the sandbox that a container operates in:

Namespace	Isolates...	Real-world example
PID	Process IDs	Processes inside a container see a different PID space
Mount	Filesystem mount points	Each container sees its own root filesystem
Network	Network stack	containers have isolated IPs, interfaces, and routes
UTS	Hostname and domain name	Each container sets its own hostname
IPC	Shared memory and semaphores	Prevents inter-process communication between containers
User	User and group IDs	Enables fake root (UID 0) inside the container
Cgroup (v2)	Control group membership	Ties into resource controls like CPU and memory limits

Visual Analogy



Imagine a shared office building:

- All tenants share the same foundation (Linux kernel).
- Each company has its own office (namespace): different locks, furniture, phone lines, and company name.
- To each tenant, it feels like their own building.

That's exactly how containers experience the system — isolated, yet efficient.

How containers Use Namespaces

When you run a container (e.g., with Docker or Podman), the runtime creates a new set of namespaces:

```
docker run -it --rm alpine sh
```

This command gives the process:

- A new PID namespace → it's process 1 inside the container.
- A new network namespace → its own virtual Ethernet.
- A mount namespace → a container-specific root filesystem.
- Other namespaces depending on configuration (user, IPC, etc.)

The result: a lightweight, isolated runtime environment that behaves like a separate system.

Complementary Kernel Features

Namespaces hide resources from containers. But to control how much they can use and what they can do, we need additional mechanisms:

Cgroups (Control Groups)

Cgroups allow the kernel to limit, prioritize, and monitor resource usage across process groups.

Resource	Use case examples
CPU	Limit CPU time per container
Memory	Cap RAM usage
Disk I/O	Throttle read/write operations
Network (v2)	Bandwidth restrictions

 Prevents the “noisy neighbor” problem by stopping one container from consuming all system resources.

Capabilities

Traditional Linux uses a binary privilege model: root (UID 0) can do everything, everyone else is limited.

Capability	Allows...
<code>CAP_NET_BIND_SERVICE</code>	Binding to privileged ports (e.g. 80, 443)
<code>CAP_SYS_ADMIN</code>	A powerful catch-all for system admin tasks
<code>CAP_KILL</code>	Sending signals to arbitrary processes

By dropping unnecessary capabilities, containers can run with only what they need — reducing risk.

Security Mechanisms

Used in conjunction with namespaces and cgroups to lock down what a containerized process can do:

Feature	Main Role	Container Security Purpose
seccomp	Filters Linux syscalls	Limits what kernel operations a containerized process can request
AppArmor	Path-based Mandatory Access Control	Restricts what files, capabilities and resources a process profile can access
SELinux	Label-based Mandatory Access Control	Enforces policy decisions based on security labels assigned to processes and objects

Summary for Beginners

- ✓ Namespaces isolate what a container can see
- ✓ Cgroups control what it can use
- ✓ Capabilities and security modules define what it can do

Together, these kernel features form the technical backbone of container isolation — enabling high-density, secure, and efficient application deployment without full VMs.

Test namespaces with unshare

```
# create new namespaces and run bash
sudo unshare \
  --fork \
  --pid \
  --mount \
  --uts \
  --ipc \
  --net \
  --mount-proc \
  /bin/bash

# check hostname (UTS namespace)
hostname ns-lab
hostname

# check PID namespace
ps auxf

# check network namespace
ip addr
ip link set lo up
ip addr

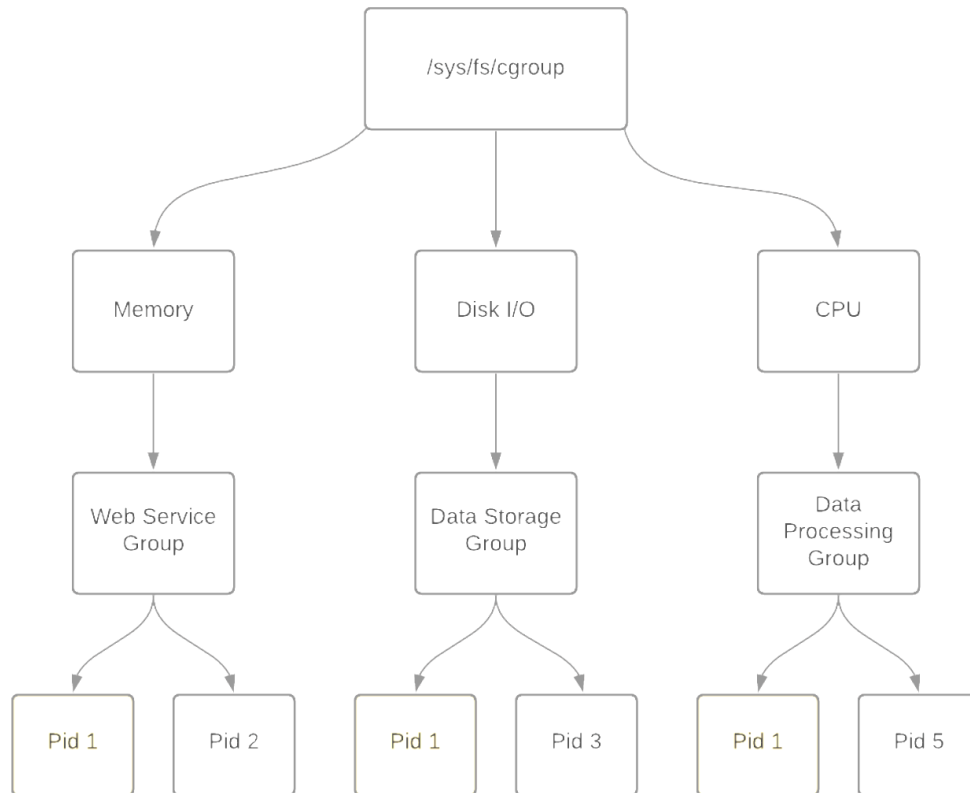
# test mount namespace
mkdir /tmp/ns-test
mount -t tmpfs tmpfs /tmp/ns-test
touch /tmp/ns-test/file
ls /tmp/ns-test

# exit namespace shell
exit

# summary of namespaces used
--pid: processes see their own PID space (PID 1 inside the namespace)
--mount: processes have their own filesystem view (can mount/unmount without affecting host)
--uts: processes have their own hostname/domainname (can set hostname without affecting host)
--ipc: processes have their own inter-process communication (shared memory, semaphores)
--net: processes have their own network stack (interfaces, IPs, routing)
```

Lab namespaces

Understanding Cgroups (Control Groups)



Definition

Control Groups (cgroups) are a Linux kernel feature introduced in 2007 that allow you to limit, account for, and isolate the resource usage (CPU, memory, disk I/O, etc.) of groups of processes.

cgroups are heavily used by low-level container runtimes such as runc and crun, and leveraged by container engines like Docker, Podman, and LXC to enforce resource boundaries and provide isolation between containers.

Namespaces isolate, cgroups control.

Namespaces create separate environments for processes (like PID, network, or mounts), while cgroups limit and monitor resource usage (CPU, memory, I/O) for those processes.

Key Capabilities

Feature	Description
Resource Limiting	Impose limits on how much of a resource a group can use
Prioritization	Allocate more CPU/IO priority to some groups over others
Accounting	Track usage of resources per group
Control	Suspend, resume, or kill processes in bulk
Isolation	Prevent resource starvation between groups

Subsystems (Controllers)

cgroups operate through controllers, each responsible for managing one type of resource:

Subsystem	Description
<code>cpu</code>	Controls CPU scheduling
<code>cpuacct</code>	Generates CPU usage reports
<code>memory</code>	Limits and accounts memory usage
<code>blkio</code>	Limits block device I/O
<code>devices</code>	Controls access to devices
<code>freezer</code>	Suspends/resumes execution of tasks
<code>net_cls</code>	Tags packets for traffic shaping
<code>ns</code>	Manages namespace access (rare)

Filesystem Layout

cgroups are exposed through the virtual filesystem under `/sys/fs/cgroup`.

Depending on the version:

- **cgroups v1:** separate hierarchies for each controller (e.g., memory, cpu, etc.)
- **cgroups v2:** unified hierarchy under a single mount point

Mounted under:

```
/sys/fs/cgroup/
```

Typical cgroups v1 hierarchy:

```

/sys/fs/cgroup/
├─ memory/
│   └─ mygroup/
│       ├── tasks
│       └─ memory.limit_in_bytes
├─ cpu/
│   └─ mygroup/
└─ ...

```

In cgroups v2, all resources are managed under a unified hierarchy:

```

/sys/fs/cgroup/
├─ cgroup.procs
├─ cgroup.controllers
├─ memory.max
├─ cpu.max
└─ ...

```

Common Usage (v1 and v2 examples)

v1 – Create and assign memory limit:

```

# Mount memory controller (if needed)
mount -t cgroup -o memory none /sys/fs/cgroup/memory

# Create group
mkdir /sys/fs/cgroup/memory/mygroup

# Set memory limit (100 MB)
echo 104857600 | tee /sys/fs/cgroup/memory/mygroup/memory.limit_in_bytes

# Assign a process (e.g., current shell)
echo $$ | tee /sys/fs/cgroup/memory/mygroup/tasks

```

v2 – Unified hierarchy:

```

# Create subgroup
mkdir /sys/fs/cgroup/mygroup

# Enable controllers
echo +memory +cpu > /sys/fs/cgroup/cgroup.subtree_control

# Move shell into group
echo $$ > /sys/fs/cgroup/mygroup/cgroup.procs

# Set limits
echo 104857600 > /sys/fs/cgroup/mygroup/memory.max
echo "50000 100000" > /sys/fs/cgroup/mygroup/cpu.max # 50ms quota per 100ms period

```

Process & Group Inspection

Command	Description
<code>cat /proc/self/cgroup</code>	Shows current cgroup membership
<code>cat /proc/PID/cgroup</code>	cgroup of another process
<code>cat /proc/PID/status</code>	Memory and cgroup info
<code>ps -o pid,cmd,cgroup</code>	Show process-to-cgroup mapping

Usage in containers

container engines like Docker, Podman, and containerd delegate resource control to cgroups (via runc or crun), allowing:

- Per-container CPU and memory limits
- Fine-grained control over blkio and devices
- Real-time resource accounting

Docker example:

```
docker run --memory=256m --cpus=1 busybox
```

Behind the scenes, this creates cgroup rules for memory and CPU limits for the container process.

Concepts Summary

Concept	Explanation
Controllers	Modules like <code>cpu</code> , <code>memory</code> , <code>blkio</code> , etc. apply limits and rules
Tasks	PIDs (processes) assigned to the control group
Hierarchy	Cgroups are structured in a parent-child tree
Delegation	Systemd and user services may manage subtrees of cgroups

Test cgroups

Example using cgroups v2 to limit memory usage:

```
# create a new cgroup for memory control
sudo mkdir -p /sys/fs/cgroup/lab-memory
echo 50M | sudo tee /sys/fs/cgroup/lab-memory/memory.max
echo 0 | sudo tee /sys/fs/cgroup/lab-memory/memory.swap.max

# move current shell into the cgroup
bash -c '
    echo $$ | sudo tee /sys/fs/cgroup/lab-memory/cgroup.procs
    python3 -c '''
import mmap,time
blocks=[]
for i in range(200):
    m=mmap.mmap(-1, 1024*1024)
    m.write(b"x" * 1024*1024)
    blocks.append(m)
    time.sleep(0.05)
'''
'

# view memory usage and cgroup status
cat /sys/fs/cgroup/lab-memory/memory.events
```

CPU limit example:

```
# create a group
sudo mkdir /sys/fs/cgroup/lab-cpu

# set CPU limit to 50ms per 100ms (50% of CPU)
echo "50000 100000" | sudo tee /sys/fs/cgroup/lab-cpu/cpu.max

# open terminal inside the cgroup
echo $$ | sudo tee /sys/fs/cgroup/lab-cpu/cgroup.procs

# run a CPU-intensive process
yes > /dev/null

# monitor CPU usage with top or htop in another terminal
top -p "$(paste -sd, /sys/fs/cgroup/lab-cpu/cgroup.procs)"
htop -p "$(paste -sd, /sys/fs/cgroup/lab-cpu/cgroup.procs)"

# check CPU usage and cgroup status
cat /sys/fs/cgroup/lab-cpu/cpu.stat
```

Lab Cgroups

Use this script for lab: cgroups.sh

Understanding Capabilities

What Are Linux Capabilities?

Traditionally in Linux, the root user has unrestricted access to the system. Linux capabilities were introduced to break down these all-powerful privileges into smaller, discrete permissions, allowing processes to perform specific privileged operations without requiring full root access.

This enhances system security by enforcing the principle of least privilege.

 Capability	 Description
CAP_CHOWN	Change file owner regardless of permissions
CAP_NET_BIND_SERVICE	Bind to ports below 1024 (e.g., 80, 443)
CAP_SYS_TIME	Set system clock
CAP_SYS_ADMIN	 Very powerful – includes mount, BPF, and more
CAP_NET_RAW	Use raw sockets (e.g., ping, traceroute)
CAP_SYS_PTRACE	Trace other processes (debugging)
CAP_KILL	Send signals to any process
CAP_DAC_OVERRIDE	Modify files and directories without permission
CAP_SETUID	Change user ID (UID) of the process
CAP_NET_ADMIN	Manage network interfaces, routing, etc.

Some Linux Capabilities Types

Capability Type	Description
CapInh (Inherited)	Capabilities inherited from the parent process.
CapPrm (Permitted)	Capabilities that the process is allowed to have.
CapEff (Effective)	Capabilities that the process is currently using.
CapBnd (Bounding)	Restricts the maximum set of effective capabilities a process can obtain.
CapAmb (Ambient)	Allows a process to explicitly define its own effective capabilities.

 Capabilities in containers and Pods containers typically do not run as full root, but instead receive a limited set of capabilities by default depending on the runtime.

Capabilities can be added or dropped in Kubernetes using the securityContext.

 Kubernetes example:

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]
```

 This ensures the container starts with zero privileges and receives only what is needed.

Test capabilities

```
# try to create a http server
python3 -m http.server 80
# error: Permission denied (because binding to port 80 requires CAP_NET_BIND_SERVICE)

# check python path
which python3

# add capability to allow binding to port 80
sudo setcap 'cap_net_bind_service=+ep' $(readlink -f $(which python3))

# check capabilities of python3
getcap $(readlink -f $(which python3))

# try to create a http server again
python3 -m http.server 80

# remove capability
sudo setcap -r $(readlink -f $(which python3))
```

Lab Capabilities

Use this script for lab: capabilities.sh

Seccomp (Secure Computing Mode)

What is it?

- A Linux kernel feature for restricting which syscalls (system calls) a process can use.
- Commonly used in containers (like Docker), browsers, sandboxes, etc.

How does it work?

- A process enables a seccomp profile/filter.
- The kernel blocks, logs, or kills the process if it tries forbidden syscalls.
- Filters are written in BPF (Berkeley Packet Filter) format.

Quick commands

```
# Check support
docker info | grep Seccomp

# Disable for a container:
docker run --security-opt seccomp=unconfined ...

# Inspect running process:
grep Seccomp /proc/$$/status
```

Tools

```
# for analyzing
seccomp-tools

# Profiles
/etc/docker/seccomp.json
```

Test seccomp

```
# check if seccomp is supported
grep SECCOMP /boot/config-$(uname -r)

# create a seccomp profile to block chmod syscalls
cat > /tmp/seccomp-deny-chmod.json <<'EOF'
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "syscalls": [
    {
      "names": ["chmod", "fchmod", "fchmodat"],
      "action": "SCMP_ACT_ERRNO"
    }
  ]
}
EOF

# test without seccomp
docker run --rm alpine sh -c 'touch /tmp/a && chmod 600 /tmp/a && echo OK'

# test with seccomp profile
docker run --rm \
  --security-opt seccomp=/tmp/seccomp-deny-chmod.json \
  alpine sh -c 'touch /tmp/a && chmod 600 /tmp/a && echo OK'
# expected: chmod: /tmp/a: Operation not permitted
```

AppArmor

What is it?

- A Mandatory Access Control (MAC) system for restricting what specific programs can access.
- Profiles are text-based, path-oriented, easy to read and edit.

How does it work?

- Each binary can have a profile that defines its allowed files, network, and capabilities—even as root!
- Easy to switch between complain, enforce, and disabled modes.

Quick commands:

```
#Status
aa-status

# Put a program in enforce mode
sudo aa-enforce /etc/apparmor.d/usr.bin.foo

# Profiles
location: /etc/apparmor.d/
```

Tools:

aa-genprof, aa-logprof for generating/updating profiles

Logs

```
/var/log/syslog (search for apparmor)
```

Test AppArmor

```
# check if AppArmor is supported
aa-status
cat /sys/module/apparmor/parameters/enabled

# create a simple AppArmor profile to block access to /tmp/secret
cat > /tmp/apparmor-block-tmp-secret <<'EOF'
#include <tunables/global>

profile block-tmp-secret flags=(attach_disconnected,mediate_deleted) {
    #include <abstractions/base>

    file,
    capability,

    /tmp/secret w,
    deny /tmp/secret r,
}
EOF

# load the profile
sudo apparmor_parser -r /tmp/apparmor-block-tmp-secret

# confirm profile is loaded
sudo aa-status | grep block-tmp-secret

# test without AppArmor
docker run --rm alpine sh -c 'echo secret > /tmp/secret && cat /tmp/secret'

# test with AppArmor profile
docker run --rm \
    --security-opt apparmor=block-tmp-secret \
    alpine sh -c 'echo secret > /tmp/secret && cat /tmp/secret'
# expected: cat: /tmp/secret: Permission denied
```

SELinux (Security-Enhanced Linux)

What is it?

- A very powerful MAC system for controlling access to everything: files, processes, users, ports, networks, and more.
- Uses labels (contexts) and detailed policies.

How does it work?

- Everything (process, file, port, etc.) gets a security context.
- Kernel checks every action against policy rules.

Quick commands:

```
#Status
sestatus

#Set to enforcing/permissive:
setenforce 1 # Enforcing
setenforce 0 # Permissive

#List security contexts:
ls -Z # Files
ps -eZ # Processes
```

Tools:

- audit2allow, semanage, chcon (for managing policies/labels)
- Logs: /var/log/audit/audit.log
- Policies: /etc/selinux/

Summary Table for Common Security Systems

System	Focus	Complexity	Policy Location	Typical Use
Seccomp	Kernel syscalls	Medium	Per-process (via code/config)	Docker, sandboxes
AppArmor	Per-program access	Easy	/etc/apparmor.d/	Ubuntu, Snap, SUSE
SELinux	Full-system MAC	Advanced	/etc/selinux/ + labels	RHEL, Fedora, CentOS

Linux container Isolation & Security Comparison

Technology	Purpose / What It Does	Main Differences	Example in containers
chroot 	Changes the apparent root directory for a process. Isolates filesystem.	Simple filesystem isolation; does not restrict resources, privileges, or system calls.	Docker uses chroot internally for building minimal images, but not for strong isolation.
namespaces 	Isolate system resources: PID, mount, UTS, network, user, IPC, time.	Kernel feature; provides different kinds of isolation.	Each container runs in its own set of namespaces (PID, net, mount, etc).
cgroups 	Controls and limits resource usage (CPU, memory, disk I/O, etc.) per group of processes.	Kernel feature; fine-grained resource control, not isolation.	Docker and Kubernetes use cgroups to limit CPU/mem per container/pod.
capabilities 	Split root privileges into fine-grained units (e.g., net_admin, sys_admin).	More granular than all-or-nothing root/non-root; can drop or grant specific privileges.	Docker containers usually run with reduced capabilities (drop dangerous ones).
seccomp 	Filter/restrict which syscalls a process can make (whitelisting/blacklisting).	Very focused: blocks kernel syscalls; cannot block all actions.	Docker's default profile blocks dangerous syscalls (e.g., ptrace , mount).
AppArmor 	Mandatory Access Control (MAC) framework: restricts programs' file/network access via profiles.	Profile-based, easier to manage than SELinux; less fine-grained in some cases.	Ubuntu-based containers often use AppArmor for container process profiles.
SELinux 	More complex MAC framework, label-based, very fine-grained. Can confine users, processes, and files.	More powerful and complex than AppArmor; enforced on Fedora/RHEL/CentOS.	On OpenShift/Kubernetes with RHEL, SELinux labels are used to keep pods separate.

Summary

- **chroot**: Basic isolation, no resource/security guarantees.
- **cgroups**: Resource control, not isolation.
- **namespaces**: Isolate “views” of kernel resources.
- **capabilities**: Fine-tune process privileges.
- **seccomp**: Restrict system call surface.
- **AppArmor/SELinux**: Limit what processes can touch, even as root (MAC).

OCI, runc, containerd, CRI, CRI-O — What They Are in the container Ecosystem

Overview and Roles

- **OCI (Open container Initiative)**  A foundation creating open standards for **container images** and **runtimes**. *Defines how images are formatted, stored, and how containers are started/stopped (runtime spec).*

- **runc**  A universal, low-level, lightweight CLI tool that can run containers according to the OCI runtime specification.
“The engine” that turns an image + configuration into an actual running Linux container.
 Responsible for creating namespaces, cgroups, capabilities, seccomp, and executing the container process.
- **containerd**  A core container runtime daemon for managing the complete container lifecycle: **pulling images, managing storage, running containers** (calls runc), networking plugins, etc.
Used by Docker, Kubernetes, nerdctl, and other tools as their main container runtime backend.
- **CRI (container Runtime Interface)**  A Kubernetes-specific gRPC API to connect Kubernetes with container runtimes.
Not used outside Kubernetes, but enables K8s to talk to containerd, CRI-O, etc.
- **CRI-O**  A lightweight, Kubernetes-focused runtime that **only** runs OCI containers, using runc under the hood.
Mostly used in Kubernetes, but demonstrates how to build a minimal container runtime focused on open standards.

Comparison Table: OCI, runc, containerd, CRI, CRI-O

Component	What Is It?	Who Uses It?	Example Usage
OCI	Standards/specifications	Docker, Podman, CRI-O, containerd, runc	Ensures images/containers are compatible across tools
runc	container runtime (CLI)	containerd, CRI-O, Docker, Podman	Directly running a container from a bundle (e.g. runc run)
containerd	container runtime daemon	Docker, Kubernetes, nerdctl	Handles pulling images, managing storage/network, starts containers via runc
CRI	K8s runtime interface (API)	Kubernetes only	Lets kubelet talk to containerd/CRI-O
CRI-O	Lightweight container runtime for K8s	Kubernetes, OpenShift	Used as K8s container engine

Practical Examples (General container World)

- **Building images:**

Any tool (Docker, Podman, Buildah) can produce images following the **OCI Image Spec** so they’re compatible everywhere.

- **Running containers:**

Both Podman and Docker ultimately use **runc** (via containerd or directly) to create containers.

- **Managing many containers:**

containerd can be used on its own (via **ctr** or **nerdctl**) or as a backend for Docker and Kubernetes.

- **Plug-and-play runtimes:**

Thanks to **OCI**, you could swap runc for another OCI-compliant runtime (like Kata containers for VMs, gVisor for sandboxing) without changing how you build or manage images.

Typical Stack

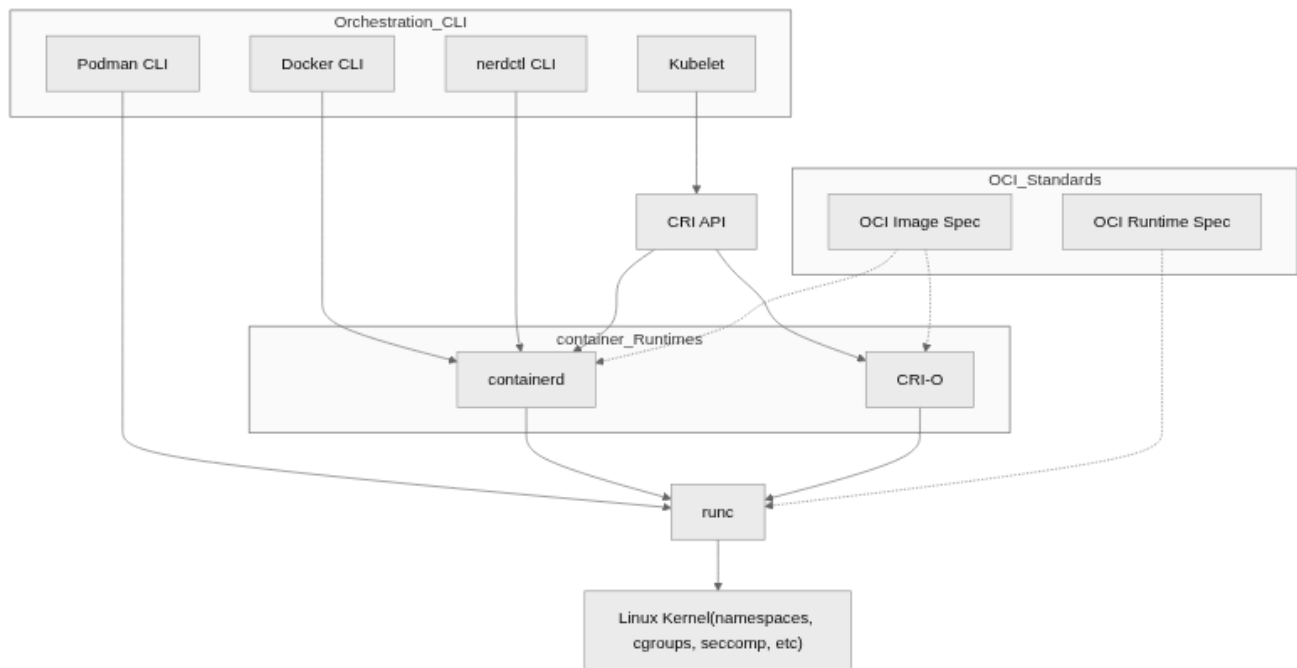
```
[User CLI / Orchestration]
    |
  [containerd / CRI-O]
    |
   [runc]
    |
[Linux Kernel: namespaces, cgroups, etc]
```

- **Docker** : User CLI → containerd → runc
- **Podman** : User CLI → runc
- **Kubernetes** : kubelet (CRI) → containerd or CRI-O → runc

Summary

- **OCI** = Common language for images/runtimes (standards/specs)
- **runc** = Actual tool that creates and manages container processes
- **containerd** = Full-featured daemon that manages images, containers, lifecycle
- **CRI** = Only for Kubernetes, to make runtimes pluggable
- **CRI-O** = Lightweight runtime focused on Kubernetes, built on OCI standards and runc

🧩 Diagram: container Ecosystem



🚀 lab runc

For runc lab, you can use this script: `runc.sh`

🚀 lab containerd

For containerd, you can use this script: `containerd.sh`

🚀 Podman, Buildah, Skopeo, OpenVZ, crun & Kata containers

🐳 Podman

- **What is it?** A container manager compatible with Docker CLI, but **daemonless** and can run **rootless**.
- **Use:** Create, run, stop, and inspect containers and pods.
- **Highlights:** No central daemon, safer for multi-user, integrates with systemd.
- More info

📦 Buildah

- **What is it?** Tool to **build and manipulate container images** (OCI/Docker) without a daemon.
- **Use:** Building images in CI/CD pipelines or scripting.
- **Highlights:** Lightweight, rootless support, used by Podman under the hood.
- More info

🛸 Skopeo

- **What is it?** Utility to **inspect, copy, and move container images** between registries **without pulling or running** them.

- **Use:** Move images, check signatures and metadata.
- **Highlights:** No daemon, ideal for automation and security.
- More info

OpenVZ

- **What is it?** **container-based virtualization** solution for Linux (pre-dating modern container tools).
- **Use:** Lightweight VPS (virtual private servers) sharing the same kernel.
- **Highlights:** Very efficient, but less isolated than VM (shares kernel).
- More info

crun

- **What is it?** Ultra-fast, minimal OCI runtime for containers, written in C (not Go).
- **Use:** Executes containers with minimal overhead.
- **Highlights:** Faster and lighter than runc, default for Podman on some systems.
- More info

Kata containers

- **What is it?** Open source project combining containers and VMs: each container runs in a lightweight micro-VM.
- **Use:** Strong isolation for sensitive workloads or multi-tenant environments.
- **Highlights:** VM-grade security, near-container performance.
- More info

Comparison Table

Project	Category	Isolation	Daemon?	Main Use	Rootless	Notes
Podman	Orchestration	container	No	Manage containers	Yes	Docker-like CLI
Buildah	Build	N/A	No	Build images	Yes	For CI/CD, no container run
Skopeo	Image transfer	N/A	No	Move/check images	Yes	No container execution
OpenVZ	Virtualization	container/VPS	Yes	Lightweight VPS	No	Kernel shared, legacy tech
crun	OCI Runtime	container	No	Fast container runtime	Yes	Faster than runc
Kata containers	Runtime/VM	MicroVM per container	No	Strong isolation	Yes	VM-level security

✓ Quick Recap

- **Podman:** Modern, daemonless Docker alternative.
- **Buildah:** Build images, doesn't run containers.
- **Skopeo:** Moves/inspects images, never runs them.
- **OpenVZ:** Legacy container-based VPS.
- **crun:** Super fast, lightweight OCI runtime.
- **Kata:** containers with VM-level isolation.

🔧 352.1 Important Commands

🔗 unshare

```
# create rootfs with debootstrap
sudo apt install -y debootstrap
sudo debootstrap stable ~vagrant/debian http://deb.debian.org/debian
# create a new namespaces and run a command in it
unshare --mount --uts --ipc --user --pid --net --map-root-user --mount-proc --fork chroot
~vagrant/debian bash
# mount /proc for test
mount -t proc proc /proc
ps -aux
ip addr show
umount /proc
```

🔍 lsns

```
# show all namespaces
lsns

# show only pid namespace
lsns -p <pid>
lsns -p 3669

ls -l /proc/<pid>/ns
ls -l /proc/3669/ns

ps -o pid,pidns,netns,ipcns,utsns,userns,args -p <PID>
ps -o pid,pidns,netns,ipcns,utsns,userns,args -p 3669
```

nsenter

```
# get <PID_CRIO_CONTAINER>
crictl ps
CID=$(sudo crictl ps -q --name <CONTAINER>)
PID=$(sudo crictl inspect -o go-template --template '{{.info.pid}}' "$CID")

# get <PID_DOCKER_CONTAINER> with docker
docker ps
CID=$(docker ps -q --filter "name=<CONTAINER>")
PID=$(docker inspect -f '{{.State.Pid}}' <CONTAINER>)

# execute a command in namespace Network
sudo nsenter -t <PID_DOCKER_CONTAINER> -n ip link show

# execute a command in namespace UTS
sudo nsenter -t <PID_DOCKER_CONTAINER> -u hostname

# execute a command in namespace mount
nsenter -t <PID_DOCKER_CONTAINER> -m ls

# execute a command in multiple namespaces
sudo nsenter -t <PID_DOCKER_CONTAINER> -m -u -i -n -p -- bash

# execute a command in all namespaces
sudo nsenter -t <PID_DOCKER_CONTAINER> -a -- bash
```

252.1 ip

```
# create a new network namespace
sudo ip netns add lxc1

# list network list
ip netns list

# exec command in network namespace
sudo ip netns exec lxc1 ip addr show
```

stat

```
# get cgroup version
stat -fc %T /sys/fs/cgroup
```

systemctl and systemd

```
# get cgroups of system
systemctl status
systemd-cgls
```

cgcreate

```
cgcreate -g memory,cpu:lsf
```

cgclassify

```
cgclassify -g memory,cpu:lsf <PID>
```

pscap - List Process Capabilities

```
# List capabilities of all process  
pscap
```

getcap /usr/bin/tcpdump

```
getcap /usr/bin/tcpdump
```

setcap cap_net_raw=ep /usr/bin/tcpdump

```
# add capabilities to tcpdump  
sudo setcap cap_net_raw=ep /usr/bin/tcpdump  
  
# remove capabilities from tcpdump  
sudo setcap -r /usr/bin/tcpdump  
sudo setcap '' /usr/bin/tcpdump
```

check capabilities by process

```
grep Cap /proc/<PID>/status
```

capsh - capability shell wrapper

```
# use grep Cap /proc/<PID>/status for get hexadecimal value(Example CApEff=0000000000002000)  
capsh --decode=0000000000002000
```

AppArmor - kernel enhancement to confine programs to a limited set of resources

```
# check AppArmor status
sudo aa-status

# unload all AppArmor profiles
aa-teardown

# loads AppArmor profiles into the kernel
aaparmor_parser
```

SELinux - Security-Enhanced Linux

```
# check SELinux status
sudo sestatus

# check SELinux mode
sudo getenforce

# set SELinux to enforcing mode
sudo setenforce 1
```

runc

```
#create a spec file for runc
runc spec

# run a container using runc
sudo runc run mycontainer
```

352.2 LXC

Weight: 6

Description: Candidates should be able to use system containers using LXC and LXD. The version of LXC covered is 3.0 or higher.

Key Knowledge Areas:

- Understand the architecture of LXC and LXD
- Manage LXC containers based on existing images using LXD, including networking and storage
- Configure LXC container properties
- Limit LXC container resource usage
- Use LXD profiles
- Understand LXC images
- Awareness of traditional LXC tools

352.2 Cited Objects

```
lxd
lxc (including relevant subcommands)
/etc/lxc/
/etc/default/lxc
/var/log/lxc/
/usr/share/lxc/templates
```

LXC & LXD – The Linux System containers Suite

LXC (Linux containers)

- **What is it?**

The *core* userspace toolset for managing application and system containers on Linux. Think of LXC as “**chroot on steroids**” – it provides lightweight process isolation using kernel features (namespaces, cgroups, AppArmor, seccomp, etc).

- **Use:**

- Run full Linux distributions as containers (not just single apps).
- Useful for testing, legacy apps, or simulating servers.

- **Highlights:**

- CLI-focused: `lxc-create`, `lxc-start`, `lxc-attach`, etc.
- Fine-grained control over container resources.
- No daemon – runs per-container processes.

- **Best for:**

Linux experts who want total control and “bare-metal” feel for containers.

lab LXC

For LXC lab, you can use this script: `lxc.sh`

LXD

- **What is it?**

LXD is a *next-generation* container and VM manager, **built on top of LXC**. It offers a powerful but user-friendly experience to manage containers and virtual machines via REST API, CLI, or even a Web UI.

- **Use:**

- Manage system containers and virtual machines at scale.
- Networked “container as a service” with easy orchestration.

- **Highlights:**

- **REST API** : manage containers/VMs over the network.
- **Images**: Instant deployment of many Linux distros.
- **Snapshots, storage pools, clustering, live migration.**
- Supports running unprivileged containers by default.
- CLI: `lxc launch`, `lxc exec`, `lxc snapshot`, etc. (Yes, same prefix as LXC, but different backend!)

- **Best for:**

DevOps, sysadmins, cloud-native setups, lab environments.

LXD Storage: Feature Table (per backend)

Feature	dir	zfs	btrfs	lvm/lvmthin	ceph/cephfs
Snapshots	✗	✓	✓	✓	✓
Thin Provisioning	✗	✓	✓	✓ (lvmthin)	✓
Resizing	✗	✓	✓	✓	✓
Quotas	✗	✓	✓	✓ (lvmthin)	✓
Live Migration	✗	✓	✓	✓	✓
Deduplication	✗	✓	✗	✗	✓ (Ceph)
Compression	✗	✓	✓	✗	✓ (Ceph)
Encryption	✗	✓	✗	✓ (LUKS)	✓
Cluster/Remote	✗	✗	✗	✗	✓
Best Use Case	Dev	Labs/Prod	Labs/Prod	Labs/Prod	Clusters, Enterprise

Quick LXD Storage Summary

- **Storage Pools**: Abstracts the backend—multiple pools, different drivers per pool.
- **Available Drivers**: dir, zfs, btrfs, lvm, lvmthin, ceph, cephfs (more via plugins).
- **Custom Volumes**: Create, mount, unmount for containers/VMs.
- **Snapshots & Clones**: Native, fast, supports backup/restore, copy-on-write migration.
- **Quotas & Resize**: Easy live management for pools, containers, or volumes.
- **Live Migration**: Move containers/VMs across hosts without downtime.
- **Security**: Built-in encryption (ZFS, LVM, Ceph), ACLs, backup/restore, etc.
- **Enterprise-ready**: Suits clustered and high-availability setups.

LXC vs LXD Comparison Table

Feature	 LXC	 LXD
Type	Low-level userspace container manager	High-level manager (containers + VMs)
Interface	CLI only	REST API, CLI, Web UI
Daemon?	No (runs as processes)	Yes (central daemon/service)
Orchestration	Manual, scriptable	Built-in clustering & API
Images	Template-based	Full image repository, many OSes
Snapshots	Manual	Native, integrated
VM support	No	Yes (QEMU/KVM)
Use-case	Fine-grained control, “bare-metal”	Scalable, user-friendly, multi-host
Security	Can be unprivileged, but DIY	Default unprivileged, more isolation
Best for	Linux pros, advanced scripting	DevOps, cloud, teams, self-service

Quick Recap

- **LXC** = The low-level building blocks. Power and flexibility for *container purists*.
- **LXD** = Modern, API-driven, scalable platform on top of LXC for *easy* container and VM management (single node or clusters).

LXC vs LXD - Storage Support (Summary)

Feature	LXC	LXD
Storage Backends	Local filesystem (default only)	dir (filesystem), zfs , btrfs , lvm , ceph , cephfs , lvmthin
Storage Pools	✗ (just local paths, no native pools)	✓ Multiple storage pools, each with different drivers
Snapshots	Manual/FS dependent	✓ Native, fast, automatic, scheduled, consistent snapshots
Thin Provisioning	✗ (not supported natively)	✓ Supported in ZFS, Btrfs, LVM thin, Ceph
Quotas	✗	✓ Supported per container/volume (in ZFS, Btrfs, Ceph, LVMthin)
Live Migration	Limited	✓ Live storage migration between hosts, copy-on-write
Encryption	✗	✓ (ZFS, LVM, Ceph)
Custom Volumes	✗	✓ Create, attach/detach custom storage volumes for containers/VMs
Remote Storage	✗	✓ Ceph, CephFS, NFS, SMB support
Filesystem Features	Host dependent	ZFS: dedup, compress, snapshots, send/receive, cache, quotas. LVM: thin, snapshots, etc.
Resizing	Manual (via host)	✓ Volumes and pools can be resized live
Storage Drivers	Basic/local only	Extensible plugins, multiple enterprise-ready drivers

Final Storage Comparison Table

	LXC	LXD
Storage Backend	Local only	dir, zfs, btrfs, lvm, lvmthin, ceph, cephfs
Storage Pools	✗	✓ Multiple, independent, hot-pluggable
Snapshots	Limited/manual	✓ Fast, automatic, consistent
Thin Provisioning	✗	✓ (ZFS, Btrfs, LVMthin, Ceph)
Quotas	✗	✓
Resizing	Manual	✓
Remote Storage	✗	✓ (Ceph, NFS, SMB)
Custom Volumes	✗	✓
Cluster Ready	✗	✓
Enterprise	No	Yes—HA, backup, migration, security, production ready

LXC vs LXD - Network Support (Summary)

Feature	LXC	LXD
Network Types	bridge, veth, macvlan, phys, vlan	bridge, ovn, macvlan, sriov, physical, vlan, fan, tunnels
Managed Networks	✗ Manual (host config)	✓ Natively managed via API/CLI, easy to create and edit
Network API	✗ CLI commands only	✓ REST API, CLI, integration with external tools
Bridge Support	✓ Manual	✓ Automatic and advanced (L2, Open vSwitch, native bridge)
NAT & DHCP	✗ Manual (iptables/dnsmasq)	✓ Integrated NAT, DHCP, DNS, per-network configurable
DNS	✗ Manual	✓ Integrated DNS, custom domains, systemd-resolved integration
IPv6	✓ (manual, limited)	✓ Full support, auto, DHCPv6, NAT6, routing
VLAN	✓ (manual, host)	✓ Native VLANs, easy configuration
SR-IOV	✗	✓ Native support
Network ACLs	✗	✓ ACLs, forwards, zones, peerings, firewall rules
Clustering	✗	✓ Replicated and managed networks in clusters
Attach/Detach	Manual (host)	✓ CLI/API, hotplug, easy for containers/VMs
Security	Manual (host)	✓ Isolation, firewall, ACL, firewalld integration, per-network rules
Custom Routes	Manual	✓ Custom routes support, multiple gateways
Network Profiles	✗	✓ Reusable network profiles
Monitoring	Manual	✓ Status, IPAM, logs, detailed info via CLI/API
Enterprise	No	Yes—multi-tenant, ACL, clustering, cloud integration

Final Network Comparison Table

	LXC	LXD
Network Types	bridge, veth, vlan	bridge, ovn, macvlan, sriov, physical, vlan, fan, tunnels
Managed	✗	✓
NAT/DHCP/DNS	Manual	✓ Integrated
VLAN	Manual	✓
SR-IOV	✗	✓
API	✗	✓
Clustering	✗	✓
Security/ACL	Manual	✓
Profiles	✗	✓
Enterprise	No	Yes

lab LXD

For LXD lab, you can use this script: `lxd.sh`

352.2 Important Commands

lxc

```
# lxc configuration
/etc/default/lxc
/etc/default/lxc-net
/etc/lxc/default.conf
/usr/share/lxc/

# lxc container configuration
/var/lib/lxc/

# check lxc version
lxc-create --version

# list containers
sudo lxc-ls --fancy
sudo lxc-ls -f

# create a priveleged container
sudo lxc-create -n busybox -t busybox

# create a priveleged container with template
sudo lxc-create -n debian01 -t download
sudo lxc-create --name server2 --template download -- --dist alpine --release 3.19 --arch amd64

# get container info
sudo lxc-info -n debian01

# get container PID
sudo lxc-info -n debian01 -pH

# get container config
sudo lxc-checkconfig -n debian01

# start container
sudo lxc-start -n debian01

# stop container
sudo lxc-stop -n debian01

# connect to container
sudo lxc-attach -n debian01

# execute a command in container
sudo lxc-attach -n debian01 -- echo "Hello from"
sudo lxc-attach -n debian01 -- bash -c ls

# delete container
sudo lxc-destroy -n debian01

# delete container and snapshot
sudo lxc-destroy -n -s debian01

# rootfs of a container
sudo ls -l /var/lib/lxc/server1/rootfs
```

```

# modify rootfs of a container
sudo touch /var/lib/lxc/server1/rootfs/tmp/test_roofs_file
sudo lxc-attach server1
ls /tmp

# get lxc namespaces
sudo lsns -p <LXC_container_PID>
sudo lsns -p $(sudo lxc-info server2 -pH)
sudo lsns -p $(sudo lxc-info -n server1 | awk '/PID:/ { print $2 }')

# unprivileged container namespaces
lsns -p $(lxc-info -n ubuntu | awk '/PID:/ { print $2 }')

# get container resource
sudo lxc-top

# create a container snapshot
sudo lxc-stop -k -n debian01
sudo lxc-snapshot -n debian01

# list snapshots
sudo lxc-snapshot -n debian01 -L

# restore snapshot
sudo lxc-stop -n debian01
sudo lxc-snapshot -n debian01 -r snap0

# delete snapshot
sudo lxc-snapshot -n debian01 -d snap0

# create a new container with snapshot
sudo lxc-snapshot -n debian01 -r snap0 -N debian02

# create container checkpoint (privileged container)
sudo lxc-checkpoint -n debian01 -s -D /home/vagrant/.config/lxc/checkpoints/debian01-checkpoint01.file

# define memory container limits with cgroups
sudo lxc-cgroup -n debian01 memory.max 262144000 #(250 MB × 1.048.576 bytes = 262144000 bytes)

# define CPU cores of container with cgroups
sudo lxc-cgroup -n debian01 cpuset.cpus 0-2

# get container cgroup limits
sudo cgget -g :lxc.payload.debian01 -a |grep memory.max
sudo cgget -g :lxc.payload.debian01 -a |grep cpuset

# set container cgroup vcpus range in file
sudo vim /var/lib/lxc/debian01/config
# add the following lines
lxc.cgroup2.cpuset.cpus = "5-6"

##### create unprivileged container #####

## create directory for unprivileged container
mkdir -p /home/vagrant/.config/lxc

## copy default config
cp /etc/lxc/default.conf /home/vagrant/.config/lxc/

## get subordinate user and group IDs
cat /etc/subuid

```

```
## configure subordinate user and group IDs
vim /home/vagrant/.config/lxc/default.conf

## add the following lines
lxc.idmap = u 0 100000 65536
lxc.idmap = g 0 100000 65536

## configure lxc-usernet
sudo vim /etc/lxc/lxc-usernet

## add the following line
vagrant veth lxcbr0 10

## create unprivileged container
lxc-create -n unprivileged -t download -- -d ubuntu -r jammy -a amd64

## set permissions for unprivileged container
sudo setfacl -m u:100000:--x /home/vagrant
sudo setfacl -m u:100000:--x /home/vagrant/.config
sudo setfacl -m u:100000:--x /home/vagrant/.local
sudo setfacl -m u:100000:--x /home/vagrant/.local/share

## start unprivileged container
lxc-start -n unprivileged --logpriority=DEBUG --logfile=lxc.log

## check container status
lxc-ls -f

## unprivileged container files
ls .local/share/lxc/unprivileged/
```

```
# lxd configuration files
/var/lib/lxd
/var/log/lxd

# initialize lxd
sudo lxd init
sudo lxd init --auto
sudo cat lxd-init.yaml | lxd init --preseed

# check lxd version
lxd --version

# check lxd status
systemctl status lxd

#### LXD STORAGE MANAGEMENT ####

# lxd list storage
lxc storage list

# show lxd storage pools
lxc storage show default

# lxd storage info
lxc storage info default

# create a new storage pool dir
lxc storage create lpic3-dir dir

# create a new storage pool lvm
lxc storage create lpic3-lvm lvm source=/dev/sdb1

# create a new storage pool btrfs
lxc storage create lpic3-btrfs btrfs
lxc storage create lpic3-btrfs btrfs size=10GB
lxc storage create lpic3-btrfs btrfs source=/dev/sdb2

# create a new storage pool zfs
lxc storage create lpic3-zfs zfs source=/dev/sdb3

# delete storage pool
lxc storage delete lpic3-btrfs

# edit storage pool
lxc storage edit lpic3-btrfs

# get storage pool properties
lxc storage get lpic3-btrfs size

# set storage pool properties
lxc storage set lpic3-btrfs size 20GB

# list storage volumes
lxc storage volume list lpic3-btrfs

# create a new storage volume
lxc storage volume create lpic3-btrfs vol-lpic3-btrfs

# delete storage volume
```

```

lxc storage volume delete lpic3-btrfs vol-lpic3-btrfs

### Management lxd storage buckets ###

# create lxd bucket
lxc storage bucket create lpic3-btrfs bucket-lpic3-btrfs
lxc storage bucket create lpic3-zfs bucket-lpic3-zfs

# list lxd buckets
lxc storage bucket list lpic3-btrfs

# set lxd bucket properties
lxc storage bucket set lpic3-btrfs bucket-lpic3-btrfs size 10GB

# edit lxd bucket
lxc storage bucket edit lpic3-btrfs bucket-lpic3-btrfs

# delete lxd bucket
lxc storage bucket delete lpic3-btrfs bucket-lpic3-btrfs

# show lxd storage bucket
lxc storage bucket show lpic3-btrfs bucket-lpic3-btrfs

# create storage bucket keys
lxc storage bucket key create lpic3-btrfs bucket-lpic3-btrfs key-bucket-lpic3-btrfs

# edit storage bucket keys
lxc storage bucket key edit lpic3-btrfs bucket-lpic3-btrfs key-bucket-lpic3-btrfs

# list storage bucket keys
lxc storage bucket key list lpic3-btrfs bucket-lpic3-btrfs

# show storage bucket keys
lxc storage bucket key show lpic3-btrfs bucket-lpic3-btrfs key-bucket-lpic3-btrfs

# delete storage bucket keys
lxc storage bucket key delete lpic3-btrfs bucket-lpic3-btrfs key-bucket-lpic3-btrfs

### LXD IMAGE MANAGEMENT ###

# list lxd repositories
lxc remote list

# add lxd remote repository
lxc remote add lpic3-images https://images.lxd.canonical.com --protocol=simplestreams

# remove lxd remote repository
lxc remote remove lpic3-images

# list lxd images
lxc image list

# list lxd images from remote repository
lxc image list images:
lxc image list images: os=Ubuntu
lxc image list images: os=Ubuntu release=jammy
lxc image list images: os=Ubuntu release=jammy architecture=amd64
lxc image list images: architecture=amd64 type=container
lxc image list images: d kal

# download lxd image to local
lxc image copy images:centos/9-Stream local: --alias centos-9

```

```

# export lxd remote image
lxc image export aed8a3749942 ./lxd-images/centos-9

# export lxd remote image
lxc image export images:f8fadb0d1b28 ./lxd-images/alma-9

# remove lxd image
lxc image delete centos-9

# mount lxd rootfs
mkdir -p /mnt/lxd-rootfs/centos-9
sudo mount lxd-images/centos-9/aed8a374994230243aaa82e979ac7d23f379e511556d35af051b1638662d47ae.squashfs /mnt/lxd-rootfs/centos-9/
ls /mnt/lxd-rootfs/centos-9/

### LXD INSTANCES MANAGEMENT ###

# create a new container from image
lxc launch ubuntu:24.04 ubuntu-lxd
lxc launch images:debian/12 debian12lxc
lxc launch images:fedora/44/cloud fedora44
lxc launch images:opensuse/16.0 opensuse16

# create a new container from image with storage pool
lxc launch images:alpine/3.24/cloud alpine --storage lpic3-lvm
lxc launch images:kali kali --storage lpic3-zfs

# create a new container from image local
lxc launch 757b2a721e9d kali-local-image

# create new vm
lxc launch --vm images:debian/13 debian13 --storage lpic3-zfs
lxc launch --vm images:e44d713a71b6 rocky9 --storage lpic3-btrfs

# list container\instances
lxc list

# stop container\instance
lxc stop alpine

# start container\instance
lxc start alpine

# delete container\instance
lxc delete alpine --force

# show container\instance
lxc info alpine

# show container\instance config
lxc config show alpine

# edit container\instance config
lxc config edit alpine

# view container\instance config
lxc config get alpine boot.autostart

# set container\instance config
lxc config set alpine boot.autostart=false

```

```

# set limit for container\instance
lxc config set alpine limits.cpu 2
lxc config set alpine limits.memory 10%

# unset limit for container\instance
lxc config unset alpine limits.cpu
lxc config unset alpine limits.memory

# execute command in container\instance
lxc --exec alpine -- /bin/bash
lxc exec alpine -- uname -a || dhclient
lxc exec alpine -- sh -c "echo 'Hello from Alpine'"

# lxd copy file to container\instance
lxc file push /etc/hosts alpine/etc/hosts

# lxd edit file in container\instance
lxc file edit alpine/etc/hosts

# download file from container\instance
lxc file pull alpine/etc/hosts /tmp/alpine-hosts

### LXD NETWORK MANAGEMENT ###

# list networks
lxc network list

# show network details
lxc network show lxdbr0

# create a new network
lxc network create lxdbr1

# delete a network
lxc network delete lxdbr0

# show network details
lxc network show lxdbr0

# set ipv4.dhcp.ranges
lxc network set lxdbr0 ipv4.dhcp.ranges=10.119.220.100-10.119.220.200

# attach a network to a container
lxc network attach lxdbr0 alpine

# detach a network from a container
lxc network detach lxdbr0 alpine

### LXD SNAPSHOT MANAGEMENT ###

# snapshot files
/var/lib/lxd/snapshots/
/var/snap/lxd/common/lxd/snapshots

# create a snapshot
lxc snapshot debian12

# create a snapshot
lxc snapshot debian12 nome-snapshot

# restore a snapshot

```

```
lxc restore debian12 nome-snapshot

# delete a snapshot
lxc delete debian12/snap0

# show snapshot info
lxc info debian12

# copy a snapshot
lxc copy debian12/snap0 debian12-2

### LXD PROFILES MANAGEMENT ###

# list profiles
lxc profile list

# show profile details
lxc profile show default

# copy profile
lxc profile copy default production

# edit profile
lxc profile edit production

#set environment variables
lxc profile set production environment.EDITOR vim

# unset memory limit
lxc profile unset production limits.memory

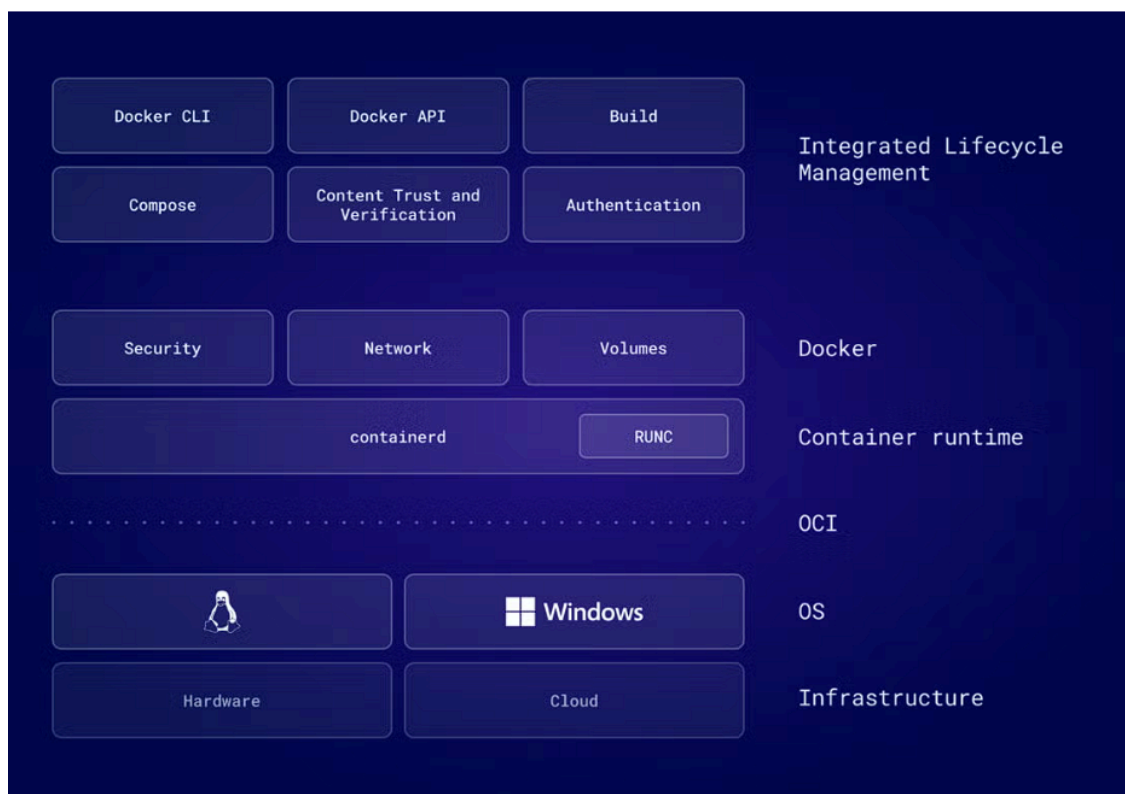
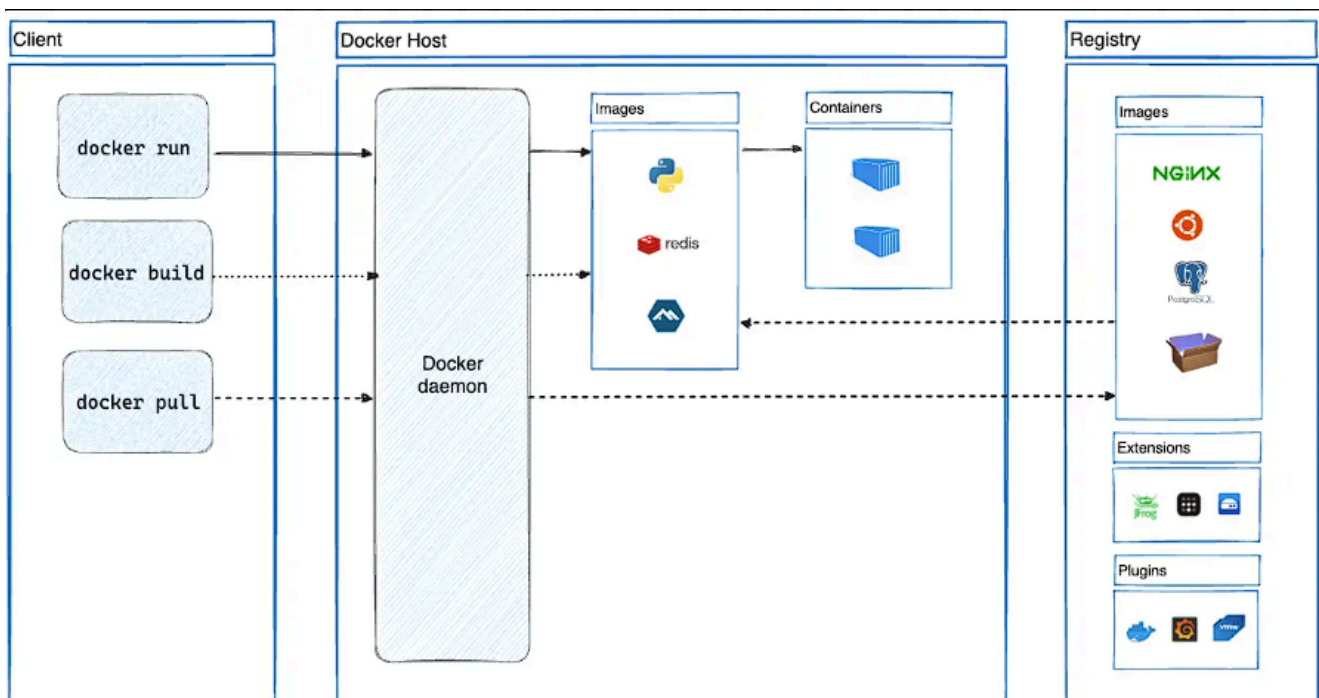
# set boot autostart
lxc profile set production boot.autostart true

# add profile to container
lxc profile add debian12 production

# remove profile from container
lxc profile remove debian12 production

# launch container with profile
lxc launch lululululul rockylinux9-2 -p production
```

352.3 Docker



Weight: 9

Description: Candidate should be able to manage Docker nodes and Docker containers. This include understand the architecture of Docker as well as understanding how Docker interacts with the node's Linux system.

Key Knowledge Areas:

- Understand the architecture and components of Docker
- Manage Docker containers by using images from a Docker registry
- Understand and manage images and volumes for Docker containers
- Understand and manage logging for Docker containers
- Understand and manage networking for Docker
- Use Dockerfiles to create container images
- Run a Docker registry using the registry Docker image

352.3 Cited Objects

```
dockerd
/etc/docker/daemon.json
/var/lib/docker/
docker
Dockerfile
```

Definition

Docker is an **open-source container platform** that allows developers and operators to package applications and their dependencies into **containers**.

These containers ensure **consistency across environments**, speed up deployments, and reduce infrastructure complexity.

Key Concepts

-  **container** → Lightweight, isolated runtime sharing the host kernel.
-  **Image** → Read-only template containing the app and dependencies.
-  **Docker Engine (dockerd)** → Daemon managing containers, images, and volumes.
-  **Docker CLI** → Command-line tool (**docker**) communicating with the daemon.
-  **Docker Hub** → Default registry for storing and distributing images.

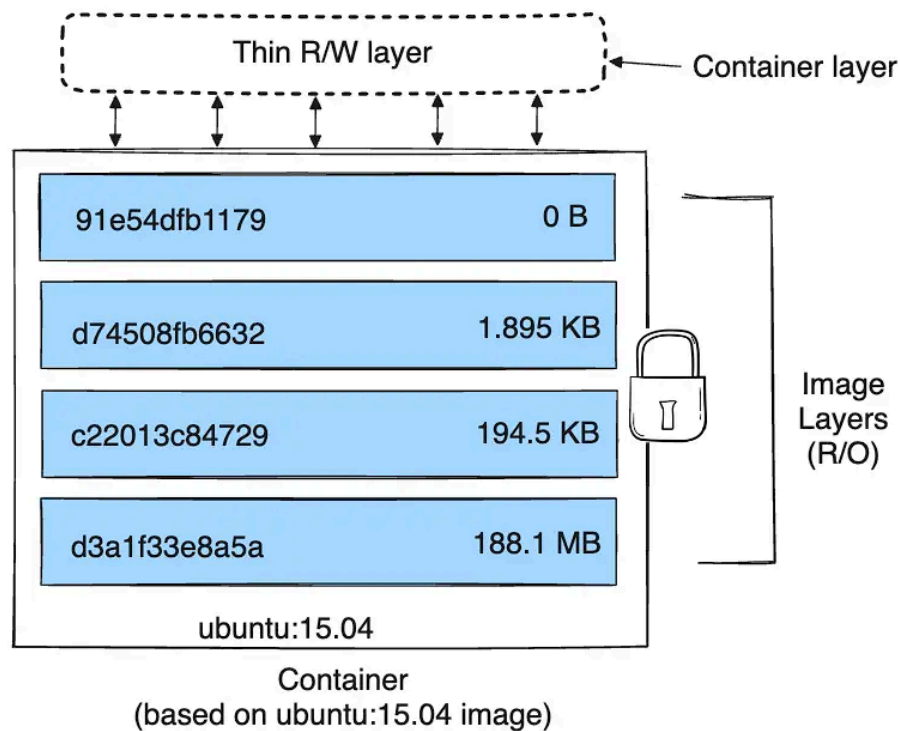
Advantages

-  **Lightweight & Fast** → Much faster than virtual machines.
-  **Portability** → Runs anywhere Docker is supported.
-  **Rich Ecosystem** → Compose, Swarm, Hub, Desktop UI, registries.
-  **DevOps Friendly** → CI/CD integration and IaC alignment.

Docker Registries

-  **Docker Hub** → Default, public registry.
-  **Private Registries** → Harbor, Artifactory, GitHub container Registry.
-  Use **docker login** to authenticate, push, and pull images.

Docker Images



- Concept: immutable package with app, dependencies, and metadata.
- Layers and cache: each Dockerfile instruction becomes a reusable layer
- Builds and pulls share layers.
- Naming: `registry/namespace/repo:tag` (e.g., `docker.io/library/nginx:1.27`).
- Digest: use `@sha256:...` to pin exact content (good for production).
- Image vs container: image is read-only; container is an instance with an ephemeral write layer.
- Basic commands: `docker image ls`, `docker pull`, `docker run`, `docker inspect`, `docker history`, `docker tag`, `docker push`, `docker rmi`, `docker image prune -a`, `docker save/docker load`.
- Best practices: minimal base (alpine/distroless), multi-stage builds, pin versions/tags, run as non-root `USER`.

Docker Image Layers

In this example, I demonstrate a docker image layers.

In the first image we have a base image of alpine and add one layer. Save file as `Dockerfile_base`.

```
# syntax=docker/dockerfile:1
FROM alpine
RUN apk add --no-cache bash
```

Build the image from the directory where the Dockerfile is saved. The build context is the current directory, so include `.` at the end:

```
docker build -t acme/my-base-image:1.0 -f Dockerfile_base .
```

To view the layers of the generated image, run:

```
docker history acme/my-base-image:1.0
```

```
[vagrant@lpic3-topic-352-docker cow-test]$docker image history acme/my-base-image:1.0 | lolcat
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
3a8a7794c791   10 minutes ago   RUN /bin/sh -c apk add --no-cache bash # bui...  2.04MB    buildkit.dockerfile.v0
<missing>      8 weeks ago     CMD ["/bin/sh"]     0B        buildkit.dockerfile.v0
<missing>      8 weeks ago     ADD alpine-minirootfs-3.22.1-x86_64.tar.gz /...  8.31MB    buildkit.dockerfile.v0
```

```
[vagrant@lpic3-topic-352-docker cow-test]$docker image inspect --format "{{json .RootFS.Layers}}" acme/my-base-image:1.0 | jq
[
  "sha256:418dccbd7d85a63a6aa574439840f7a6fa6fd2321b3e2394568a317735e867d35",
  "sha256:532c08503e63547ac7cdf4c1ac66db031da3344e26dce0bb03a37d86fc3f1d29"
]
```

The second image I have a my-base-image:1.0 and add two layers, generating a new image with name acme/my-final-image:1.0.

```
mkdir -p app
echo -e '#!/bin/bash\n\nnecho "Hello from my-final-image!"' > /app/hello.sh
```

```
# syntax=docker/dockerfile:1
FROM acme/my-base-image:1.0
COPY . app/
RUN chmod +x /app/hello.sh
CMD /app/hello.sh
```

Build the final image from the directory containing the Dockerfile and application files. Include the build context at the end:

```
docker build -t acme/my-final-image:1.0 -f Dockerfile .
```

To view the layers of the final image, run:

```
docker history acme/my-final-image:1.0
```

```
[vagrant@lpic3-topic-352-docker cow-test]$docker image history acme/my-final-image:1.0 | lolcat
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
7ed7f6f760eb   9 minutes ago    CMD ["/bin/sh" "-c" "/app/hello.sh"]         0B        buildkit.dockerfile.v0
<missing>      9 minutes ago    RUN /bin/sh -c chmod +x /app/hello.sh # buil...  40B        buildkit.dockerfile.v0
<missing>      9 minutes ago    COPY . /app # buildkit                        223B       buildkit.dockerfile.v0
<missing>      10 minutes ago   RUN /bin/sh -c apk add --no-cache bash # bui...  2.04MB     buildkit.dockerfile.v0
<missing>      8 weeks ago     CMD ["/bin/sh"]     0B        buildkit.dockerfile.v0
<missing>      8 weeks ago     ADD alpine-minirootfs-3.22.1-x86_64.tar.gz /...  8.31MB     buildkit.dockerfile.v0
```

```
[vagrant@lpic3-topic-352-docker cow-test]$docker image inspect --format "{{json .RootFS.Layers}}" acme/my-final-image:1.0 | jq
[
  "sha256:418dcc7d85a63a6aa574439840f7a6fa6fd2321b3e2394568a317735e867d35",
  "sha256:532c08503e63547ac7cdf4c1ac66db031da3344e26dce0bb03a37d86fc3f1d29",
  "sha256:0d3a4345bba0d87b8549f7b75171323aa1a65df03a576cf985cabd301e420b36",
  "sha256:0963b165a0503929ce59d53458db5bc2582e408f6d8e3d509b3ff7fe80fcbbb1"
]
```

Docker image Copy-on-Write (CoW)

In this example, I demonstrate a docker image Copy-on-Write (CoW).

Create a 5 containers from the same image.

```
docker run -dit --name my_container_1 acme/my-final-image:1.0 bash \
&& docker run -dit --name my_container_2 acme/my-final-image:1.0 bash \
&& docker run -dit --name my_container_3 acme/my-final-image:1.0 bash \
&& docker run -dit --name my_container_4 acme/my-final-image:1.0 bash \
&& docker run -dit --name my_container_5 acme/my-final-image:1.0 bash
```

See the size of the containers.

```
docker ps --size --format "table {{.ID}}\t{{.Image}}\t{{.Names}}\t{{.Size}}"
```

To demonstrate this, run the following command to write the word ‘hello’ to a file on the container’s writable layer in containers my_container_1, my_container_2, and my_container_3:

```
for i in {1..3}; do docker exec my_container_$i sh -c 'printf hello > /out.txt'; done
```

Check the size of the containers again.

```
docker ps --size --format "table {{.ID}}\t{{.Image}}\t{{.Names}}\t{{.Size}}"
```

```
[vagrant@lpic3-topic-352-docker cow-test]$docker ps --size --format "table {{.ID}}\t{{.Image}}\t{{.Names}}\t{{.Size}}" | lolcat
CONTAINER ID  IMAGE                                NAMES                SIZE
6d93bd19a72f  acme/my-final-image:1.0            my_container_5       0B (virtual 10.4MB)
f4b2fd871d6b  acme/my-final-image:1.0            my_container_4       0B (virtual 10.4MB)
03e698dbad9f  acme/my-final-image:1.0            my_container_3       0B (virtual 10.4MB)
0e273974e760  acme/my-final-image:1.0            my_container_2       0B (virtual 10.4MB)
f7943eb8c8b5  acme/my-final-image:1.0            my_container_1       0B (virtual 10.4MB)
[vagrant@lpic3-topic-352-docker cow-test]$for i in {1..3}; do docker exec my_container_$i sh -c 'printf hello > /out.txt'; done
[vagrant@lpic3-topic-352-docker cow-test]$docker ps --size --format "table {{.ID}}\t{{.Image}}\t{{.Names}}\t{{.Size}}" | lolcat
CONTAINER ID  IMAGE                                NAMES                SIZE
6d93bd19a72f  acme/my-final-image:1.0            my_container_5       0B (virtual 10.4MB)
f4b2fd871d6b  acme/my-final-image:1.0            my_container_4       0B (virtual 10.4MB)
03e698dbad9f  acme/my-final-image:1.0            my_container_3       5B (virtual 10.4MB) ←
0e273974e760  acme/my-final-image:1.0            my_container_2       5B (virtual 10.4MB) ←
f7943eb8c8b5  acme/my-final-image:1.0            my_container_1       5B (virtual 10.4MB) ←
[vagrant@lpic3-topic-352-docker cow-test]$
```

Dockerfile Image Instructions and Layers

Table: Instruction vs. Layer Generation

Instruction	Creates a Filesystem Layer?	Notes
FROM	✗ No	Sets the base image; underlying layers come from it.
RUN	✓ Yes	Executes filesystem changes; adds content that persists.
COPY	✓ Yes	Adds files from build context into the image filesystem.
ADD	✓ Yes	Similar to COPY, with additional features (URLs, tar extraction).
LABEL	✗ No	Only adds metadata; doesn't change filesystem content.
ENV	✗ No	Defines environment variables; stored as metadata.
ARG	✗ No	Build-time only; does not affect final image unless used later.
WORKDIR	✗ No	Changes working directory; metadata only.
USER	✗ No	Sets the user; metadata only.
EXPOSE	✗ No	Declares exposed port(s); metadata only.
ENTRYPOINT	✗ No	Defines how container starts; metadata configuration.
CMD	✗ No	Default command or args; metadata only.
VOLUME	✓ Yes / Partial	Declares mount points; metadata + volumes in runtime; has filesystem implications.
HEALTHCHECK	✗ No	Defines health check config; stored as metadata.
STOPSIGNAL	✗ No	Defines signal to stop container; metadata only.
SHELL	✗ No	Changes shell for later RUN; metadata only.
ONBUILD	✗ No	Triggers for future builds; metadata only.

Key Insights

- Most Dockerfile instructions **create a new image layer** — even metadata changes (**CMD**, **EXPOSE**, etc.) are stored as part of the final image configuration.
- **Heavyweight layers** come from instructions that **modify the filesystem** (**RUN**, **COPY**, **ADD**).
- **Lightweight/metadata layers** come from instructions like **ENV**, **CMD**, **LABEL**.
- **ARG is special** : it exists only at build-time and is discarded in the final image unless used in other instructions.
- To minimize image size:
 - Combine multiple **RUN** commands into one.
 - Use **.dockerignore** to avoid copying unnecessary files.
 - Order instructions to maximize Docker's **build cache efficiency** .

Dockerfile

A **Dockerfile** is a declarative text file that contains a sequence of instructions to build a Docker image. It's the blueprint for creating reproducible, portable, and automated containerized environments.

✨ Key Concepts

Concept	Description
 Declarative Script	A simple text file with line-by-line instructions for assembling an image.
 Layered Architecture	Each instruction in a Dockerfile creates a new layer in the image. Layers are stacked and are read-only.
 Build Cache	Docker caches the result of each layer. If a layer and its dependencies haven't changed, Docker reuses the cached layer, making builds significantly faster.
 Build Context	The set of files at a specified PATH or URL that are sent to the Docker daemon during a build. Use a .dockerignore file to exclude unnecessary files.
 Multi-Stage Builds	A powerful feature that allows you to use multiple FROM instructions in a single Dockerfile. This helps to separate build-time dependencies from runtime dependencies, resulting in smaller and more secure production images.

Core Instructions

The following table summarizes the most common Dockerfile instructions.

Instruction	Purpose	Example
 FROM	Specifies the base image for subsequent instructions. Must be the first instruction.	<code>FROM ubuntu:22.04</code>
 LABEL	Adds metadata to an image as key-value pairs.	<code>LABEL version="1.0" maintainer="me@example.com"</code>
 RUN	Executes any commands in a new layer on top of the current image and commits the results.	<code>RUN apt-get update && apt-get install -y nginx</code>
 CMD	Provides defaults for an executing container. There can only be one CMD .	<code>CMD ["nginx", "-g", "daemon off;"]</code>
 ENTRYPOINT	Configures a container that will run as an executable.	<code>ENTRYPOINT ["/usr/sbin/nginx"]</code>
 EXPOSE	Informs Docker that the container listens on the specified network ports at runtime.	<code>EXPOSE 80</code>
 ENV	Sets environment variables.	<code>ENV APP_VERSION=1.0</code>
 COPY	Copies new files or directories from the build context to the filesystem of the image.	<code>COPY ./app /app</code>
 ADD	Similar to COPY , but with more features like remote URL support and tar extraction.	<code>ADD http://example.com/big.tar.xz /usr/src</code>
 USER	Sets the user name (or UID) and optionally the user group (or GID) to use when running the image.	<code>USER appuser</code>
 WORKDIR	Sets the working directory for any RUN , CMD , ENTRYPOINT , COPY , and ADD instructions.	<code>WORKDIR /app</code>
 VOLUME	Creates a mount point with the specified name and marks it as holding externally mounted volumes.	<code>VOLUME /var/lib/mysql</code>
 ONBUILD	Adds to the image a trigger instruction to be executed at a later time, when the image is used as the base for another build.	<code>ONBUILD COPY . /app/src</code>
 HEALTHCHECK	Tells Docker how to test a container to check that it is still working.	<code>`HEALTHCHECK --interval=5m --timeout=3s CMD curl -f http://localhost/`</code>
 SHELL	Allows the default shell used for the shell form of commands to be overridden.	<code>SHELL ["/bin/bash", "-c"]</code>

★ Best Practices for Writing Dockerfiles

Following best practices is crucial for creating efficient, secure, and maintainable images.

Guideline	Description
 Keep it Small	Start with a minimal base image (like alpine or distroless). Don't install unnecessary packages to reduce size and attack surface.
 Leverage Build Cache	Order your Dockerfile instructions from least to most frequently changing. Place COPY and ADD instructions as late as possible to avoid cache invalidation.
 Use Multi-Stage Builds	Separate your build environment from your runtime environment. This dramatically reduces the size of your final image by excluding build tools and dependencies.
 Use .dockerignore	Exclude files and directories that are not necessary for the build (e.g., .git , node_modules , local test scripts) to keep the build context small and avoid sending sensitive data.
 Combine RUN Instructions	Chain related commands using && to create a single layer. For example, combine apt-get update with apt-get install and clean up afterwards (rm -rf /var/lib/apt/lists/*).
 Pin Versions	Pin versions for base images (ubuntu:22.04) and packages (nginx=1.21.6-1~bullseye) to ensure reproducible builds and avoid unexpected changes.
 Run as Non-Root	Create a dedicated user and group with RUN useradd , and use the USER instruction to switch to that user. This improves security by avoiding running containers with root privileges.
 CMD vs ENTRYPOINT	Use ENTRYPOINT for the main executable of the image and CMD to specify default arguments. This makes the image behave like a binary.
 Sort Multi-line Arguments	Sort multi-line arguments alphanumerically (e.g., in a long RUN apt-get install command) to make the Dockerfile easier to read and maintain.
 Be Explicit	Use COPY instead of ADD when the extra magic of ADD (like tar extraction or URL fetching) is not needed. It's more transparent.

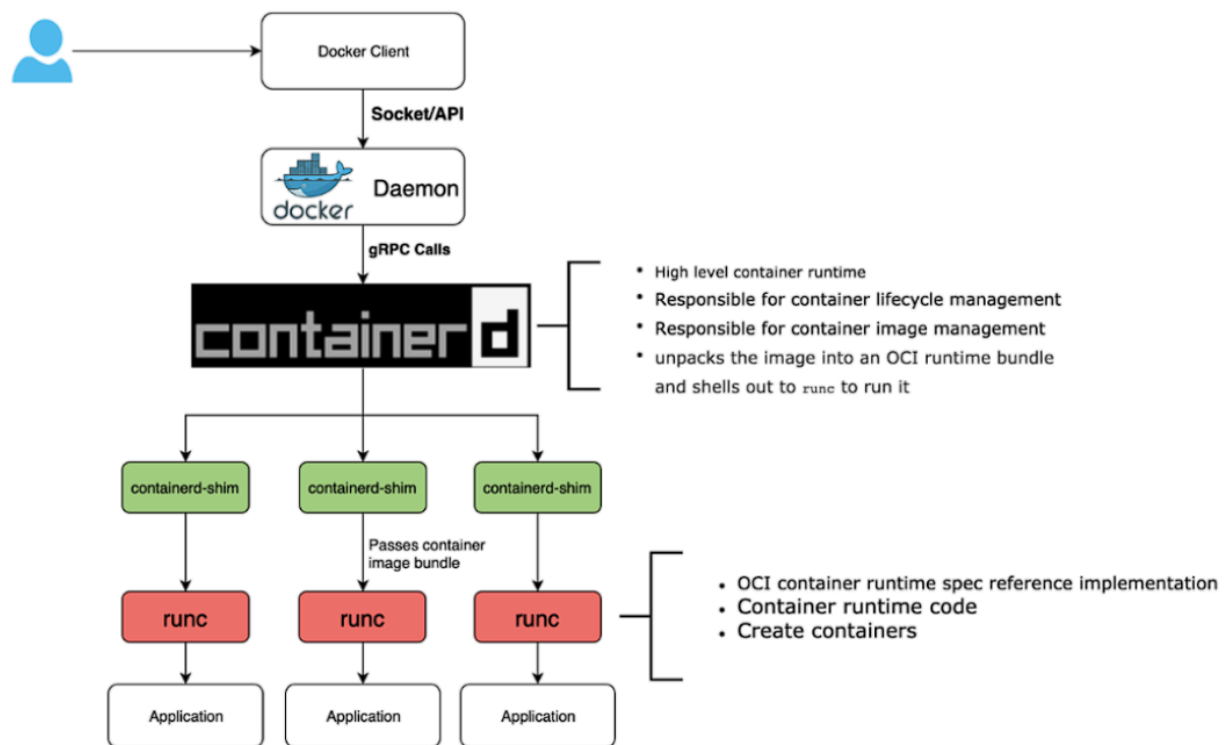
Dockerfile example

```
# syntax=docker/dockerfile:1

# ---- Base Stage ----
FROM ubuntu:22.04 AS base
RUN apt-get update && apt-get install -y --no-install-recommends nginx \
    && rm -rf /var/lib/apt/lists/*

# ---- Production Stage ----
FROM base AS production
COPY ./html /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Docker + containerd + shim + runc Architecture



♦ Main Components

- **Docker CLI / Docker Daemon (dockerd)**

The `docker` command communicates with the Docker daemon, which orchestrates container lifecycle, images, networks, and volumes.

- **containerd**

A high-level container runtime that manages the entire container lifecycle: pulling images, managing storage, networking, and execution.

- **containerd-shim**

- Acts as the *parent process* of each container once `runc` has done its job.
- Keeps `stdin/stdout/stderr` streams open, even if Docker or containerd restarts (so `docker logs` / `kubectl logs` still work).
- Collects the container exit code and reports it back to the manager.
- Prevents containers from becoming orphans if the daemon crashes or is restarted.

- **runc**

A low-level runtime (OCI-compliant) that creates containers using Linux namespaces and cgroups.

After launching the container, `runc` exits, and `containerd-shim` takes over as the parent process.

♦ Execution Flow

1. **User** runs `docker run ...` → the **Docker Daemon** is called.
2. **Docker Daemon** delegates to **containerd**.
3. **containerd** spawns **runc**, which sets up the container.
4. Once the container starts, **runc exits**.
5. **containerd-shim** remains as the **container's parent process**, handling logging and exit codes.

♦ Benefits of the Shim Layer

- **Resilience** → Containers continue running even if `dockerd` or `containerd` crash or restart.
- **Logging** → Maintains container log streams for `docker logs` or `kubectl logs`.
- **Isolation** → Each container has its own shim, simplifying lifecycle management.
- **Standards Compliance** → Works with the **OCI runtime spec**, ensuring compatibility.

Docker vs. containerd

♦ Feature / Component	 Docker (dockerd)	 containerd
Scope	Full platform (build, CLI, UI, Hub)	Core container runtime only
API	High-level Docker API	Low-level CRI/runtime API
Built upon	Uses containerd internally	Standalone runtime
Features	Build, Compose, Swarm, Hub, Desktop	Image lifecycle, pull/run, runtime
Use Cases	Dev workflows, local testing	Kubernetes, production runtimes
Footprint	Heavier, more tooling	Lightweight, efficient
Ecosystem	Rich developer tools	CNCF project, Kubernetes default

Docker Storage

Core Concepts

 Focus	Details
Union FS	Read-only image layers + the container's writable layer form a union filesystem; removing the container drops ephemeral changes.
Data Root	Storage drivers persist data under <code>/var/lib/docker/<driver>/</code> ; inspect the active driver via <code>docker info --format "{{.Driver}}"</code> .
Persistence	Move stateful data to volumes (persistent), bind mounts (host path), or tmpfs mounts (in-memory, ephemeral) to survive container recreation or optimize performance.

⚙️ Storage Drivers

Driver	When to use	Notes
overlay2	Default on modern Linux kernels.	Fast copy-on-write; backing filesystem must support <code>d_type</code> .
fuse-overlayfs	Rootless or user-namespace deployments.	Adds a thin FUSE layer; enables non-root workflows.
btrfs / zfs	Need native snapshots, quotas, compression.	Provision dedicated pools and use platform tooling for management.
devicemapper (direct-lvm) / aufs	Legacy setups only.	Maintenance mode; plan migrations to overlay2.
windowsfilter	Windows container images.	Use LCOW/WSL 2 to expose overlay2 for Linux workloads on Windows hosts.

🔍 Selecting the Driver

- Confirm kernel modules (`modprobe overlay`) and filesystem prerequisites before switching drivers.
- Match driver features to workloads: many small layers favor `overlay2`; filesystem-level snapshots may justify `btrfs` or `zfs`.
- Stick to provider defaults on Docker Desktop, EKS, GKE, etc., to stay within support boundaries.
- Keep `/var/lib/docker` on reliable, low-latency storage—copy-on-write drivers amplify slow disks.

For testing volume drivers use script: `docker-storage-driver.sh`.

📦 Docker Storage Types

Volumes:

- Managed by Docker, located outside the container's writable layer (`/var/lib/docker/volumes`).
- Persist after container removal, can be shared between containers.
- Used for data that must survive the container lifecycle.
- Examples:
 - Create volume: `docker volume create data`
 - Use volume: `docker run -v data:/app/data ...`

Bind Mounts:

- Mount a host directory/file directly into the container.
- Useful for development, code sync, or accessing existing host data.
- Less portable (absolute paths, host permissions).
- Examples:
 - `docker run -v /home/user/app:/app ...`

- `docker run --mount type=bind,source=/data,target=/app/data ...`

Tmpfs Mounts:

- In-memory mount (RAM), does not persist after container stops or restarts.
- Ideal for temporary data, caches, or sensitive information.
- Nothing is written to disk, maximum performance.
- Examples:
 - `docker run --mount type=tmpfs,target=/tmp/cache ...`
 - `docker run --tmpfs /tmp/cache ...`

Quick summary:

Type	Persistence	Location	Portability	Typical use
Volume	Yes	Docker	High	App data, databases
Bind mount	Optional	Host	Low	Dev, integration
Tmpfs	No	RAM	High	Cache, ephemeral

Storage Types Usage examples

```
# Persistent volume
docker run -d --name pg -v pgdata:/var/lib/postgresql/data postgres:16

# Bind mount
docker run -d -v /home/user/html:/usr/share/nginx/html nginx:latest

# Tmpfs mount
docker run -d --mount type=tmpfs,target=/tmp nginx:latest
docker run -d --tmpfs /tmp nginx:latest
```

Docker Storage Best practices

- Prefer volumes for persistent and backup data.
- Use tmpfs for sensitive or temporary data.
- Document volumes and mounts in Compose/Stack files.
- Monitor disk usage with `docker system df` and clean up orphaned volumes.
- Always check the official Docker Storage documentation and storage drivers.

For testing storage volumes use script: `docker-storage-volumes.sh`.

Docker Networking

Core Concepts

 Focus	Details
User-defined networks	Create isolated topologies (<code>docker network create</code>) and attach/detach containers on demand with <code>docker network connect</code> or the <code>--network</code> flag.
Network stack sharing	By default each container gets its own namespace; <code>--network container:<id></code> reuses another container's stack but disables flags like <code>--publish</code> , <code>--dns</code> , and <code>--hostname</code> .
Embedded DNS	Docker injects an internal DNS server per network; container names and <code>--network-alias</code> entries resolve automatically and fall back to the host resolver for external lookups.
Gateway priority	When a container joins multiple networks, Docker selects the default route via the highest <code>--gw-priority</code> ; override IPs with <code>--ip</code> / <code>--ip6</code> for deterministic addressing.

Default Drivers

Driver	Use when	Highlights
bridge	Standalone workloads on a single host need simple east-west traffic.	Default <code>bridge</code> network ships with Docker; create user-defined bridges for DNS, isolation, and per-project scoping.
host	You need native host networking with zero isolation.	Shares the host stack; no port mapping needed; ideal for high-throughput or port-dynamic workloads.
overlay	Services must span multiple Docker hosts or Swarm nodes.	VXLAN-backed; requires the Swarm control plane (or external KV store) to coordinate networks across engines.
macvlan	Containers must appear as physical devices on the LAN.	Assigns unique MAC/IP pairs from the parent interface; great for legacy integrations or strict VLAN segmentation.
ipvlan	Underlay restricts MAC addresses but permits L3 routing.	Provides per-container IPv4/IPv6 without extra MACs; supports L2 (<code>ipvlan -l2</code>) and L3 modes with VLAN tagging.
none	Full isolation is required.	Removes the network stack entirely; manual namespace wiring only (not supported for Swarm services).
Plugins	Built-in drivers fall short of SDN or vendor needs.	Install third-party network plugins from the Docker ecosystem to integrate with specialized fabrics.

Working with Networks

- Scope infrastructure with user-defined bridges for app components, overlays for distributed stacks, or L2-style macvlan/ipvlan for direct LAN presence.
- Combine frontend/backend networks per container; set `--internal` on a bridge to block egress while still allowing service-to-service traffic.

- Inspect connectivity with `docker network ls`, `docker network inspect`, and `docker exec <ctr> ip addr` to validate namespace wiring.
- Clean up unused networks regularly with `docker network prune` to avoid stale subnets and orphaned config.

Published Ports & Access

- Bridge networks keep ports private unless you publish them with `-p` / `--publish`; include `127.0.0.1:` (or `:::1:` for IPv6) to restrict exposure to the host only.
- Port publishing is not required for container-to-container access on the same user-defined bridge—DNS and the internal IP suffice.
- Overlay and macvlan drivers bypass the userland proxy; plan upstream firewalls or routing accordingly.

Addressing & DNS

- IPv4 is enabled by default on new networks; add `--ipv6` to provision dual-stack ranges and use `--ip` / `--ip6` to pin addresses.
- Each join operation can supply extra identities via `--alias`; Docker advertises them through the embedded DNS service.
- Override resolvers per container using `--dns`, `--dns-search`, or `--dns-option`, or import extra hosts with `-add-host`.
- Containers inherit a curated `/etc/hosts`; host-level entries are not synced automatically.

Docker Network Usage examples

```
# Create dedicated frontend and backend bridges
docker network create --driver bridge frontend_net
docker network create --driver bridge --internal --subnet 10.18.0.0/24 backend_net

# Launch API with deterministic addressing and backend alias
docker run -d --name api \
  --network backend_net --ip 10.18.0.10 \
  --network-alias api.internal \
  ghcr.io/mccutchen/go-httpbin:latest

# Launch web on the frontend network
docker run -d --name web \
  --network frontend_net \
  --network-alias web.public \
  -p 8080:80 \
  nginx:alpine

# Attach web to backend network with a backend-only alias
docker network connect --alias web-backend backend_net web

# Attach a troubleshooting container temporarily
docker run -it --rm \
  --network container:web \
  alpine:latest sh

# Tests
# Test backend API alias
wget -qO- http://api.internal:8080/get

# Test backend alias assigned to web itself
wget -qO- http://web-backend

# Test frontend alias assigned to web itself
wget -qO- http://web.public

# Show DNS resolution
getent hosts api.internal
getent hosts web-backend
getent hosts web.public
```

Docker Network Best practices

- Model network boundaries early; document which containers share bridges, overlays, or macvlan segments.
- Use **--internal** or firewalls to block unintended egress, and prefer network-level isolation over ad-hoc port publishing.
- When mixing drivers, verify default routes (**ip route**) to ensure the correct gateway won the **--gw-priority**.
- Monitor subnet allocation conflicts when multiple hosts create networks; explicitly set **--subnet** / **--gateway** for predictable CIDRs.
- Cross-check the official docs for updates: Networking overview and Network drivers.

For testing docker network use script: `docker-network.sh`.

Docker Registry

What is a Docker Registry?

A Docker Registry is a stateless, highly scalable server-side application that stores and lets you distribute Docker images. It's the central place where you can push your images after building them and pull them to run on other machines.

Key Concepts

- **Registry:** The storage system that contains repositories of images. Examples: Docker Hub, AWS ECR, a self-hosted registry.
- **Repository:** A collection of related Docker images, often different versions of the same application or service (e.g., the `nginx` repository).
- **Tag:** A label applied to an image within a repository to identify a specific version (e.g., `1.27`, `latest`).
- **Image Name:** The full name of an image follows the format: `[registry-host]/[username-or-org]/[repository]:[tag]`.
 - If `registry-host` is omitted, it defaults to Docker Hub (`docker.io`).
 - If `tag` is omitted, it defaults to `latest`.

Types of Registries

1. Public Registries:

- **Docker Hub:** The default and largest public registry.
- **Quay.io:** Another popular public and private registry by Red Hat.
- **GitHub Container Registry (GHCR):** Integrated with GitHub repositories and Actions.

2. Private Registries:

- **Self-Hosted:**
 - **Docker Registry Image:** A simple, official image to run your own basic registry.
 - **Harbor:** An enterprise-grade open-source registry with security scanning, access control, and replication.
 - **JFrog Artifactory:** A universal artifact manager that supports Docker images.
- **Cloud-Hosted:**
 - **Amazon Elastic Container Registry (ECR)**
 - **Google Artifact Registry (formerly GCR)**
 - **Azure Container Registry (ACR)**

Running a Local Registry

You can easily run a private registry locally for testing or development using Docker's official `registry` image.

1. Start the local registry container:

```
docker run -d -p 5000:5000 -v /var/lib/registry-data:/var/lib/registry --restart=always --name registry registry:2
```

This starts a registry listening on **localhost:5000**.

2. Tag an image to point to the local registry:

Before you can push an image to this registry, you need to tag it with the registry's host and port.

```
# Pull an image (e.g., alpine)
docker pull alpine

# Tag it for your local registry
docker tag alpine localhost:5000/my-alpine
```

3. Push the image to the local registry:

```
docker push localhost:5000/my-alpine
```

4. Pull the image from the local registry:

You can now pull this image on any machine that can access **localhost:5000**.

```
# First, remove the local copies to simulate pulling from scratch
docker image rm alpine
docker image rm localhost:5000/my-alpine

# Now, pull from your local registry
docker pull localhost:5000/my-alpine
```

5. Access the registry API:

You can interact with the registry using its HTTP API. For example, to list repositories:

```
curl -X GET http://localhost:5000/v2/_catalog
```

Core Commands

Command	Description	Example
<code>docker login</code>	Log in to a Docker registry.	<code>docker login myregistry.example.com</code>
<code>docker logout</code>	Log out from a Docker registry.	<code>docker logout</code>
<code>docker pull</code>	Pull an image or a repository from a registry.	<code>docker pull ubuntu:22.04</code>
<code>docker push</code>	Push an image or a repository to a registry.	<code>docker push myregistry.com/myapp:1.0</code>
<code>docker search</code>	Search Docker Hub for images.	<code>docker search nginx</code>

For testing docker registry use script: `docker-registry-lab.sh`.

352.3 Important Commands

docker

```
##### FILES #####
/var/lib/docker
/etc/docker/daemon.json

##### DAEMON #####
# get version
docker --version

# docker infos
docker info

##### MANAGE IMAGES #####
# pull image from docker hub
docker pull nginx:latest

# list images
docker image ls
docker images
docker images -a
docker images --format "Repository: {{.Repository}}\nTarg: {{.Tag}}\nSize: {{.Size}}\n"

# docker image inspect
docker image inspect nginx:latest
docker inspect nginx:latest
docker inspect --format '{{.Id}}' nginx:latest
docker image inspect --format "{{json .RootFS.Layers}}" acme/my-base-image:1.0

# remove image
docker image rm nginx:latest
docker rmi nginx:latest
docker rmi -f nginx:latest
docker image prune -a

# docker history
docker history nginx:latest

# docker push image to registry
docker push acme/my-final-image:1.0

# create image from dockerfile
docker build -t acme/my-base-image:1.0 .
docker build -t acme/my-final-image:1.0 -f Dockerfile.final .
docker build -t acme/my-final-image:1.0 --build-arg BASE_IMAGE=acme/my-base-image:1.0 .

# create a new tag for an image
docker tag nginx:latest acme/nginx:1.1

# send image to tar file
docker save -o nginx-latest.tar nginx:latest

# load image from tar file
docker load -i nginx-latest.tar

##### MANAGE CONTAINERS #####

# list containers running
```

```

docker container ls
docker ps

# list all containers
docker container ls -a
docker ps -a

# list containers id
docker container ls -q

# list last created container
docker container ls -l

# list containers with size
docker ps -s
docker ps --size --format "table {{.ID}}\t{{.Image}}\t{{.Names}}\t{{.Size}}"

# create container
docker container run hello-world

# create container as daemon
docker container run -d --name my-nginx -p 8080:80 nginx:latest

# create container and run interactively
docker container run -it ubuntu bash

# docker container port
docker container port my-nginx

# create container and expose port 80 to host port 8080
docker container run -d --name my-nginx -p 8080:80 nginx:latest

# create container and publish all exposed ports to random ports
docker container run -d --name my-nginx -P nginx:latest

# create container and expose tcp port 8080 and udp port 8080 to host
docker container run -d --name my-nginx -p 8080:80/tcp -p 8080:80/udp nginx:latest

# create a container and expose port 8888
docker container run -d --name my-nginx -p 9082:80 --expose 8888 nginx:latest

# create a container and define hostname, dns
docker container run -dit --name ubuntu --dns=8.8.8.8 --dns=1.1.1.1 --hostname ubuntu ubuntu

# create container in detached mode
docker container run -d -it --name alpine alpine

# pause container
docker container pause <container_id|name>

# unpause container
docker container unpause <container_id|name>

# stop container
docker container stop <container_id|name>

# start container
docker container start <container_id|name>

# remove container
docker container rm <container_id|name>

```

```

# remove container force
docker container rm -f <container_id|name>

# prune all stopped containers
docker container prune

# remove all containers
docker container rm -f $(docker container ps -a -q)

# inspect container
docker container inspect <container_id|name>

# get PID of container
docker container inspect --format '{{.State.Pid}}' <container_id|name>

# get ID of container
docker container inspect --format '{{.Id}}' <container_id|name>
docker container inspect --format '{{.Id}}' <container_id|name>

# execute command in container
docker container exec -it <container_id|name> bash
docker container exec -it <container_id|name> ls /
docker container exec -it <container_id|name> sh -c "echo 'Hello from container'"

# copy file to container
docker container cp /etc/hosts <container_id|name>:/etc/hosts

# copy file from container
docker container cp <container_id|name>:/etc/hosts /tmp/container-hosts

##### MANAGE STORAGE #####

# Docker Storage Files
/var/lib/docker/overlay/
/var/lib/docker/containers/
/var/lib/docker/volumes/

# list volumes
docker volume ls

# create volume
docker volume create my-volume

# inspect volume
docker volume inspect my-volume

# remove volume
docker volume rm my-volume

# prune all unused volumes
docker volume prune

# create container with bind mount
docker container run -d --name my-nginx -p 8080:80 --mount
type=bind,source=/home/vagrant/html,target=/usr/share/nginx/html nginx:latest
docker container run -d --name my-nginx -p 8080:80 --volume
/home/vagrant/html:/usr/share/nginx/html nginx:latest
docker container run -d --name my-nginx -p 8080:80 -v /home/vagrant/html:/usr/share/nginx/html
nginx:latest
docker container run -d --name my-nginx -p 8080:80 -v
/home/vagrant/html:/usr/share/nginx/html:ro nginx:latest

```

```

# create container with volume
docker container run -d --name my-nginx -p 8080:80 --mount type=volume,source=my-
volume,target=/usr/share/nginx/html nginx:latest
docker container run -d --name my-nginx -p 8080:80 --volume my-volume:/usr/share/nginx/html
nginx:latest

# create a container with volume-from another container
docker container run -d --name my-nginx2 -p 8086:80 --volumes-from my-nginx1 nginx:latest

# create container with tmpfs mount
docker container run -d --name my-nginx -p 8080:80 --mount
type=tmpfs,target=/usr/share/nginx/html nginx:latest
docker container run -d --name my-nginx -p 8080:80 --tmpfs /usr/share/nginx/html nginx:latest

# remove volume
docker volume rm my-volume

# remove volumes not used by any containers
docker volume prune

# remove all volumes
docker volume rm $(docker volume ls -q)

##### MANAGE LOGS #####

# view container logs
docker container logs <container_id|name>

# follow container logs
docker container logs -f <container_id|name>

# docker system events
docker system events --since "2h"
docker system events --since "20h" --filter 'container=<container_id|name>'
docker system events --since "20h" --filter type=container --filter 'event=start' --filter
'event=stop'

# docker stats
docker container stats
docker container stats <container_id|name>

# docker top
docker container top <container_id|name>

##### DOCKER NETWORKING #####

# list networks
docker network ls

# inspect network
docker network inspect bridge
docker network inspect --format '{{json .Containers}}' bridge | jq

# create network
docker network create my-bridge

# create network with specific driver
docker network create --driver bridge my-bridge

# create network with specific subnet
docker network create --subnet 192.168.1.0/24 my-bridge

```

```

# remove network
docker network rm my-bridge

# prune all unused networks
docker network prune

# connect container to network
docker network connect my-bridge <container_id|name>

# create a network and define: subnet,gateway,bridge name
docker network create \
  --driver bridge \
  --subnet 192.168.3.0/24 \
  --gateway 192.168.3.1 \
  --opt "com.docker.network.bridge.name="br-mybridge" \
  my-bridge3

# create a container and connect it to a specific network
docker container run -d --name my-nginx --network my-bridge3 -p

# disconnect container from network
docker network disconnect my-bridge <container_id|name>

# create a network with specific options
docker network create \
  --driver bridge \
  --opt com.docker.network.bridge.enable_icc=false \
  --opt com.docker.network.bridge.enable_ip_masquerade=true \
  --opt com.docker.network.bridge.host_binding_ipv4="192.168.1.1" \
  my-bridge

##### OTHERS COMMANDS #####

# inspect namespaces
ls -l /proc/<PID>/ns
sudo lsns -p <PID>
ps -o pid,ppid,cmd,netns,mntns,pidns,utsns <PID>

# inspect cgroups
lscgroup | grep <PID> # cgroup v1
cat /proc/<PID>/cgroup # cgroup v2
ls -l /sys/fs/cgroup/system.slice/docker-<FULL_ID_CONTAINER>.scope
cat /sys/fs/cgroup/system.slice/docker-<FULL_ID_CONTAINER>.scope/cgroup.procs

```

352.4 container Orchestration Platforms

Weight: 3

Description: Candidates should understand the importance of container orchestration and the key concepts Docker Swarm and Kubernetes provide to implement container orchestration.

Key Knowledge Areas:

- Understand the relevance of container orchestration
- Understand the key concepts of Docker Compose and Docker Swarm
- Understand the key concepts of Kubernetes and Helm

- Awareness of OpenShift, Rancher and Mesosphere DC/OS

Docker Compose

Docker Compose Command Reference

Docker Compose is a tool for defining and managing multi-container Docker applications using a YAML file (`docker-compose.yml`).

Below is a structured table of the main commands and their purposes.

Table: Docker Compose Commands

Command	Purpose	Example
 <code>docker compose up</code>	Build, (re)create, start, and attach to containers defined in <code>docker-compose.yml</code> .	<code>docker compose up -d</code>
 <code>docker compose down</code>	Stop and remove containers, networks, volumes, and images created by <code>up</code> .	<code>docker compose down --volumes</code>
 <code>docker compose restart</code>	Restart running services.	<code>docker compose restart web</code>
 <code>docker compose start</code>	Start existing containers without recreating them.	<code>docker compose start db</code>
 <code>docker compose stop</code>	Stop running containers without removing them.	<code>docker compose stop db</code>
 <code>docker compose rm</code>	Remove stopped service containers.	<code>docker compose rm -f</code>
 <code>docker compose build</code>	Build or rebuild service images.	<code>docker compose build web</code>
 <code>docker compose pull</code>	Pull service images from a registry.	<code>docker compose pull redis</code>
 <code>docker compose push</code>	Push service images to a registry.	<code>docker compose push api</code>
 <code>docker compose config</code>	Validate and view the Compose file.	<code>docker compose config</code>
 <code>docker compose ps</code>	List containers managed by Compose.	<code>docker compose ps</code>
 <code>docker compose top</code>	Display running processes of containers.	<code>docker compose top</code>
 <code>docker compose logs</code>	View output logs from services.	<code>docker compose logs -f api</code>
 <code>docker compose exec</code>	Run a command in a running service container.	<code>docker compose exec db psql -U postgres</code>
 <code>docker compose run</code>	Run one-off commands in a new container.	<code>docker compose run web sh</code>
 <code>docker compose override</code>	Use <code>-f</code> to specify multiple Compose files (overrides).	<code>docker compose -f docker-compose.yml -f docker-compose.override.yml up</code>

Key Notes

- **up vs start** : **up** builds/recreates containers, **start** only runs existing ones.
- **run vs exec** : **run** launches a *new* container, **exec** runs inside an existing one.
- **Config validation** : Always run **docker compose config** to check for syntax errors.
- **Detach mode** : Use **-d** to run services in background.

docker-compose.yml

```
services:
  web:
    image: nginx:latest
    container_name: my-nginx
    ports:
      - "8080:80"          # host:container
    volumes:
      - ./html:/usr/share/nginx/html:ro
    networks:
      - app-network

  api:
    build:
      context: ./api       # build from Dockerfile in ./api
      dockerfile: Dockerfile
    container_name: my-api
    environment:
      - NODE_ENV=production
      - API_KEY=${API_KEY} # read from .env file
    depends_on:
      - db
    ports:
      - "3000:3000"
    networks:
      - app-network

  db:
    image: postgres:15
    container_name: my-postgres
    restart: always
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network

volumes:
  db-data:

networks:
  app-network:
    driver: bridge
```

Explanation

- **services** : Defines containers (**web**, **api**, **db**) that make up the app.
- **ports** : Maps host ports to container ports (**8080:80**).
- **volumes** :
 - Named volume (**db-data**) for persistent DB data.
 - Bind mount (**./html:/usr/share/nginx/html**) to serve static content.
- **build** : Allows building a custom image from a Dockerfile.
- **depends_on** : Ensures service startup order (**api** waits for **db**).
- **networks** : Defines an isolated virtual network for communication.

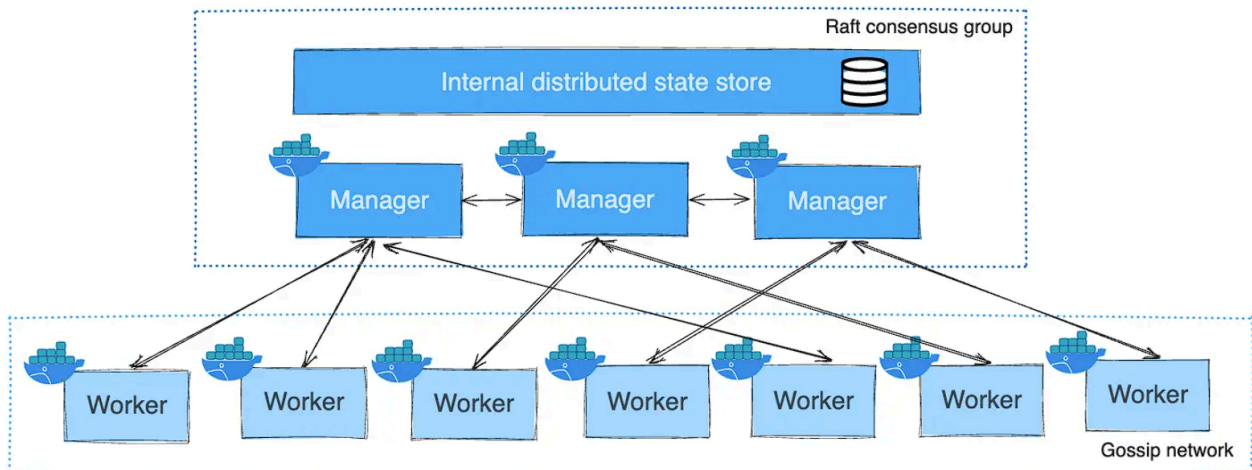
Usage

Start in detached mode

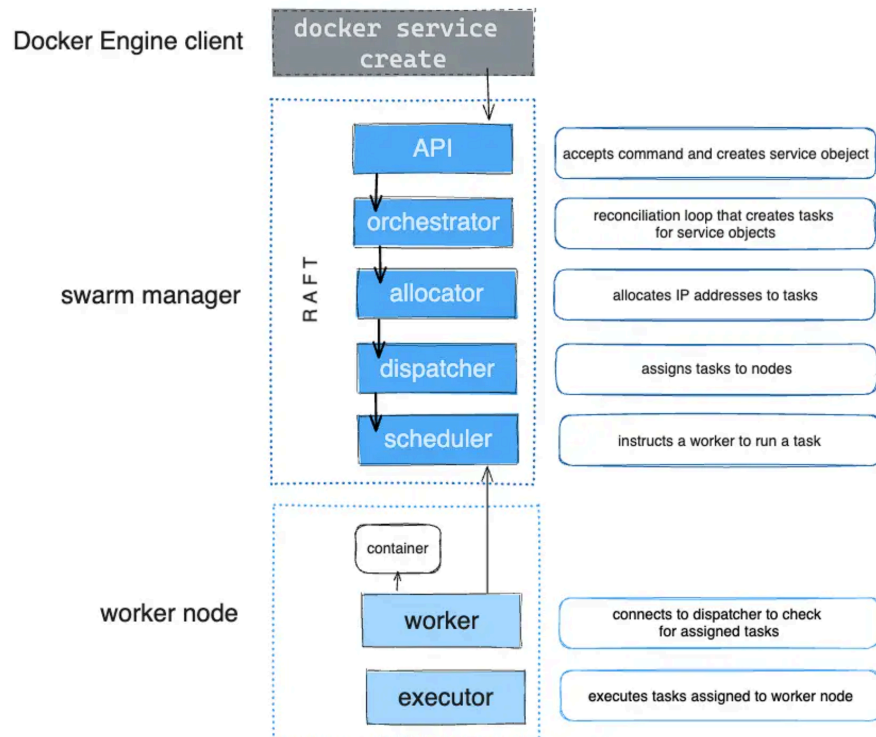
```
docker compose up -d
docker compose logs -f api
docker compose down -v
```

For testing docker compose use examples of services in apps.

Docker Swarm



Swarm architecture with manager and worker nodes



Swarm services with multiple replicas

Docker Swarm is Docker’s native orchestration tool that allows you to manage a cluster of Docker hosts as a single virtual system. It facilitates the deployment, management, and scaling of containerized applications across multiple machines.

Docker Swarm Key Concepts

Concept	Description
 Swarm	A cluster of Docker hosts running in swarm mode.
 Node	A Docker host participating in the swarm. Nodes can be either managers or workers.
 Manager Node	Responsible for managing the swarm’s state, scheduling tasks, and maintaining the desired state of the cluster.
 Worker Node	Executes tasks assigned by manager nodes, running the actual containers.
 Service	An abstract definition of a computational resource (e.g., an Nginx web server) that can be scaled and updated independently.
 Task	A running container that is part of a service.

🌟 Main characteristics

Feature	Description
📶 High availability	Distributes services across multiple nodes, ensuring applications remain available even if a node fails.
⚖️ Scalability	Easily scale services up or down to handle varying workloads.
🔄 Load Balancing	Built-in load balancing distributes requests evenly among service replicas.
🚀 Rolling Updates	Perform updates to services with zero downtime.
😊 Ease of use	Integrated directly into Docker Engine, making it relatively simple to set up and manage compared to other orchestrators.

🐳 Swarm Management Commands

Command	Description	Example
👑 <code>docker swarm init</code>	Initializes a new swarm on the current node, making it the manager.	<code>docker swarm init --advertise-addr 192.168.1.10</code>
👤 <code>docker swarm join</code>	Joins a node to an existing swarm as a worker or manager.	<code>docker swarm join --token <TOKEN> 192.168.1.10:2377</code>
👋 <code>docker swarm leave</code>	Removes the current node from the swarm.	<code>docker swarm leave --force</code>
📋 <code>docker node ls</code>	Lists all nodes in the swarm.	<code>docker node ls</code>

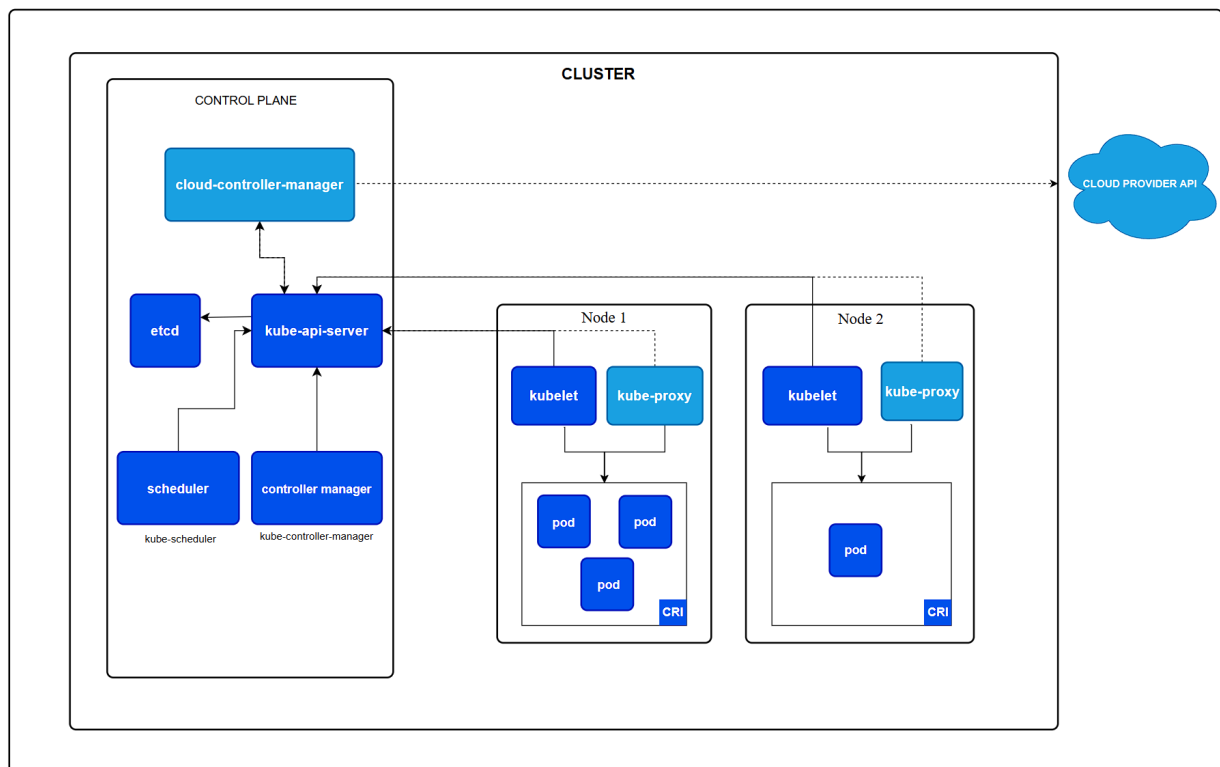
Service Management Commands

Command	Description	Example
 <code>docker service create</code>	Creates a new service in the swarm.	<code>docker service create --name web -p 80:80 --replicas 3 nginx</code>
 <code>docker service scale</code>	Scales one or more replicated services.	<code>docker service scale web=5</code>
 <code>docker service update</code>	Updates a service's configuration.	<code>docker service update --image nginx:latest web</code>
 <code>docker service rm</code>	Removes a service from the swarm.	<code>docker service rm web</code>
 <code>docker service ls</code>	Lists all services in the swarm.	<code>docker service ls</code>
 <code>docker service ps</code>	Lists the tasks of one or more services.	<code>docker service ps web</code>

For testing docker swarm use script: `docker-swarm.sh`.

Kubernetes

Kubernetes, also known as K8s, is an open-source platform for automating the deployment, scaling, and management of containerized applications. It groups containers that make up an application into logical units for easy management and discovery.



A Kubernetes cluster consists of a set of worker machines, called **nodes**, that run containerized applications. Every cluster has at least one worker node. The worker node(s) host the **Pods** which are the components of the application workload. The **control plane** manages the worker nodes and the Pods in the cluster.

Control Plane Components

The control plane's components make global decisions about the cluster (for example, scheduling), as well as detecting and responding to cluster events.

Component	Description
kube-apiserver	The API server is a component of the Kubernetes control plane that exposes the Kubernetes API. The API server is the front end for the Kubernetes control plane.
etcd	Consistent and highly-available key-value store used as Kubernetes' backing store for all cluster data.
kube-scheduler	Watches for newly created Pods with no assigned node, and selects a node for them to run on.
kube-controller-manager	Control Plane component that runs controller processes. To reduce complexity, dozens of separate controllers are compiled into this single binary and run within a single process. Each controller continuously monitors the cluster state (via kube-apiserver) and drives it toward the desired state. Key embedded controllers include: • Node Controller: Manages node health and downtime. • Deployment & ReplicaSet Controller: Maintains the correct number of pod replicas. • EndpointSlice Controller: Links Kubernetes Services to Pods. • Namespace Controller: Manages lifecycle and cleanup of namespaces. • ServiceAccount Controller: Creates default accounts for new namespaces.
cloud-controller-manager	A Kubernetes control plane component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster.

Node Components

Node components run on every node, maintaining running pods and providing the Kubernetes runtime environment.

Component	Description
kubelet	An agent that runs on each node in the cluster. It makes sure that containers are running in a Pod.
kube-proxy	A network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept.
Container runtime	The software that is responsible for running containers. Kubernetes supports several container runtimes: Docker, containerd, CRI-O, and any other implementation of the Kubernetes CRI (Container Runtime Interface).

Kubernetes Objects

Kubernetes objects are persistent entities in the Kubernetes system. Kubernetes uses these entities to represent the state of your cluster.

Object	Description
Pod	The smallest and simplest unit in the Kubernetes object model that you create or deploy. A Pod represents a set of running containers on your cluster.
Service	An abstract way to expose an application running on a set of Pods as a network service.
Volume	A directory containing data, accessible to the containers in a Pod.
Namespace	A way to divide cluster resources between multiple users.
Deployment	Provides declarative updates for Pods and ReplicaSets. You describe a desired state in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate.
ReplicaSet	Ensures that a specified number of pod replicas are running at any given time.
StatefulSet	Manages the deployment and scaling of a set of Pods, and provides guarantees about the ordering and uniqueness of these Pods.
DaemonSet	Ensures that all (or some) Nodes run a copy of a Pod.
Job	Creates one or more Pods and ensures that a specified number of them successfully terminate.
CronJob	Creates Jobs on a time-based schedule.

✳ Helm

Helm is a package manager for Kubernetes.

It helps you manage Kubernetes applications — Helm Charts help you define, install, and upgrade even the most complex Kubernetes application.

🎯 Key Concepts

Concept	Description
Chart	A Helm package. It contains all of the resource definitions necessary to run an application, tool, or service inside of a Kubernetes cluster.
Repository	A place where charts can be collected and shared.
Release	An instance of a chart running in a Kubernetes cluster. One chart can often be installed many times into the same cluster. And each time it is installed, a new <i>release</i> is created.

Core Commands Kubernetes

Command	Description	Example
<code>helm search</code>	Search for charts in a repository.	<code>helm search repo stable</code>
<code>helm install</code>	Install a chart.	<code>helm install my-release stable/mysql</code>
<code>helm upgrade</code>	Upgrade a release.	<code>helm upgrade my-release stable/mysql</code>
<code>helm uninstall</code>	Uninstall a release.	<code>helm uninstall my-release</code>
<code>helm list</code>	List releases.	<code>helm list</code>

Other Orchestration Platforms

OpenShift

OpenShift is a family of containerization software products developed by Red Hat. Its flagship product is the OpenShift Container Platform — an on-premises platform as a service built around Docker containers orchestrated and managed by Kubernetes on a foundation of Red Hat Enterprise Linux.

Rancher

Rancher is a complete software stack for teams adopting containers. It addresses the operational and security challenges of managing multiple Kubernetes clusters across any infrastructure, while providing DevOps teams with integrated tools for running containerized workloads.

Mesosphere DC/OS

Mesosphere DC/OS (the Datacenter Operating System) is a distributed operating system based on the Apache Mesos distributed systems kernel. It can manage multiple machines in a datacenter or cloud as if they're a single computer. It provides a highly elastic, and highly scalable way of deploying applications, services, and big data infrastructure on shared resources.

352.4 Important Commands

```
#----- SWARM MANAGEMENT COMMANDS -----
# initialize a new swarm
docker swarm init --advertise-addr 192.168.1.131

# list nodes in the swarm
docker node ls

# join a node to an existing swarm
docker swarm join --token <TOKEN> 192.168.1.131:2377

# leave the swarm
docker swarm leave --force

# promote a node to manager
docker node promote <NODE_ID_OR_NAME>

# demote a node to worker
docker node demote <NODE_ID_OR_NAME>

# inspect a node
docker node inspect <NODE_ID_OR_NAME>
docker node inspect --format '{{json .Spec.Role}}' lpic3-topic-352-docker-1
docker node inspect --format '{{json .Spec}}' lpic3-topic-352-docker-2 | jq
docker node inspect --format '{{json .Status}}' lpic3-topic-352-docker-1 | jq

# remove a node from the swarm
docker node rm <NODE_ID_OR_NAME>
# remove all nodes
docker node rm $(docker node ls -q)

#----- SERVICE MANAGEMENT COMMANDS -----
# create a service
docker service create --name registry --publish published=5000,target=5000 registry:2
docker service create --name web --replicas 3 --publish published=8080,target=80 nginx:latest

# list services
docker service ls

# logs of a service
docker service logs <SERVICE_ID_OR_NAME>
docker service logs -f <SERVICE_ID_OR_NAME>
docker service logs -f registry

# inspect a service
docker service inspect <SERVICE_ID_OR_NAME>
docker service inspect --format '{{json .Spec}}' registry

# remove a service
docker service rm <SERVICE_ID_OR_NAME>

# scale a service
docker service scale <SERVICE_ID_OR_NAME>=<NUMBER_OF_REPLICAS>
docker service scale registry=3

# update a service
docker service update --image registry:3 registry

# list tasks of a service
```

```

docker service ps <SERVICE_ID_OR_NAME>
docker service ps registry

# list all tasks
docker service ps $(docker service ls -q)

# inspect a task
docker inspect <TASK_ID>
docker inspect <TASK_ID> --format '{{json .Status}}' | jq
docker inspect <TASK_ID> --format '{{json .Spec}}' | jq

# remove all services
docker service rm $(docker service ls -q)

#----- STACK MANAGEMENT COMMANDS -----
# validate a stack file
docker stack config --compose-file docker-compose.yml

# deploy a stack
docker stack deploy -c docker-compose.yml mystack

# list stacks
docker stack ls

# list services in a stack
docker stack services mystack

# list tasks in a stack
docker stack ps mystack
docker stack ps mystack --filter "desired-state=running"

# scale a service in a stack
docker service scale mystack_web=5

# remove a stack
docker stack rm mystack

# remove all stacks
docker stack rm $(docker stack ls -q)

# ----- MINIKUBE -----
# start minikube
minikube start

# stop minikube
minikube stop

# delete minikube
minikube delete

# dashboard
minikube dashboard
minikube dashboard -p lpic3

#----- KUBERNETES COMMANDS -----
# get kubernetes cluster info
kubectl cluster-info

# Management nodes

# get nodes
kubectl get nodes

```

```

# Manage pods and deployments

# run a pod
kubectl run nginx-example --image=nginx:latest

# get pods
kubectl get pods

# get deployments
kubectl get deployments

# describe a pod
kubectl describe pod <POD_NAME>

# describe a deployment
kubectl describe deployment <DEPLOYMENT_NAME>

# generate a yaml file for a pod
kubectl run nginx-example-1 --image=nginx:latest --dry-run=client -o yaml >
configs/kubernetes/manifests/nginx-pod.yaml

# generate a yaml file for a deployment
kubectl create deployment nginx-example-2 --image=nginx:latest --dry-run=client -o yaml >
configs/kubernetes/manifests/nginx-deployment.yaml

# create a resource from a configuration file
kubectl create -f nginx-deployment.yaml

# apply a configuration file
kubectl apply -f nginx-deployment.yaml

# delete a resource from a configuration file
kubectl delete -f nginx-deployment.yaml

# scale a deployment
kubectl scale deployment nginx-example-2 --replicas=5

#----- HELM COMMANDS -----

# list repositories
helm repo list

# add a repository
helm repo add stable https://charts.helm.sh/stable

# update repositories
helm repo update

# delete a repository
helm repo remove stable

# search for charts in a repository
helm search repo stable

# install a chart
helm install my-release stable/mysql
helm install mysql -n mysql --create-namespace stable/mysql --values values.yaml

# upgrade a release
helm upgrade my-release stable/mysql

```

```
# uninstall a release
helm uninstall my-release

# list releases
helm list -A

# fetch chart
helm fetch stable/mysql

# fetch values of a release
helm get values my-release
```

Topic 353: VM Deployment and Provisioning

353.1 Cloud Management Tools

Weight: 2

Description: Candidates should understand common offerings in public clouds and have basic feature knowledge of commonly available cloud management tools.

Key Knowledge Areas:

- Understand common offerings in public clouds
- Basic feature knowledge of OpenStack
- Basic feature knowledge of Terraform
- Awareness of CloudStack, Eucalyptus and OpenNebula

353.1 Cited Objects

IaaS, PaaS, SaaS
OpenStack
Terraform

Cloud Definition

Cloud is a model for distributed computing that provides on-demand access to a shared pool of configurable computing resources (networks, servers, storage, applications, and services) via the internet, provisioned and released rapidly with minimal management effort.

Cloud Computing Definition

Cloud Computing is the paradigm for delivering IT services through the internet, where computing resources are dynamically allocated based on demand. Instead of investing in local infrastructure, users and organizations access computing power, storage, and applications through subscription or pay-per-use models, with automatic scalability and centralized management.

Cloud computing is the execution of workloads in clouds.

Clouds are IT environments that abstract, aggregate, and share scalable resources across a network.

Cloud computing and clouds are not technologies in themselves.

Cloud computing is an action, the execution of a workload in a cloud.

Clouds are environments, locations where applications run.

Technologies are things, such as software programs and hardware equipment used to create and use clouds.

Types of cloud

Public Cloud

Public cloud refers to cloud computing services offered by third-party providers over the public internet, making them available to anyone who wants to use or purchase them. Examples include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).

Private Cloud

Private cloud is a cloud computing environment dedicated to a single organization, either hosted on-premises or by a third-party provider. It offers enhanced security, control, and customization compared to public clouds.

Hybrid Cloud

Hybrid cloud combines public and private cloud environments, allowing data and applications to be shared between them. This approach provides greater flexibility, scalability, and cost-efficiency.

Multi-Cloud

Multi-cloud refers to the use of multiple cloud computing services from different providers within a single architecture. This strategy helps avoid vendor lock-in, enhances redundancy, and allows organizations to leverage the best features of each cloud provider.

IaaS, PaaS, SaaS

IaaS (Infrastructure as a Service)

IaaS provides virtualized computing resources on demand (virtual machines, block and object storage, networking, GPUs). By 2026, IaaS has evolved to include AI-ready infrastructure, automated bare-metal provisioning, and native integration with observability, security, and real-time compliance solutions.

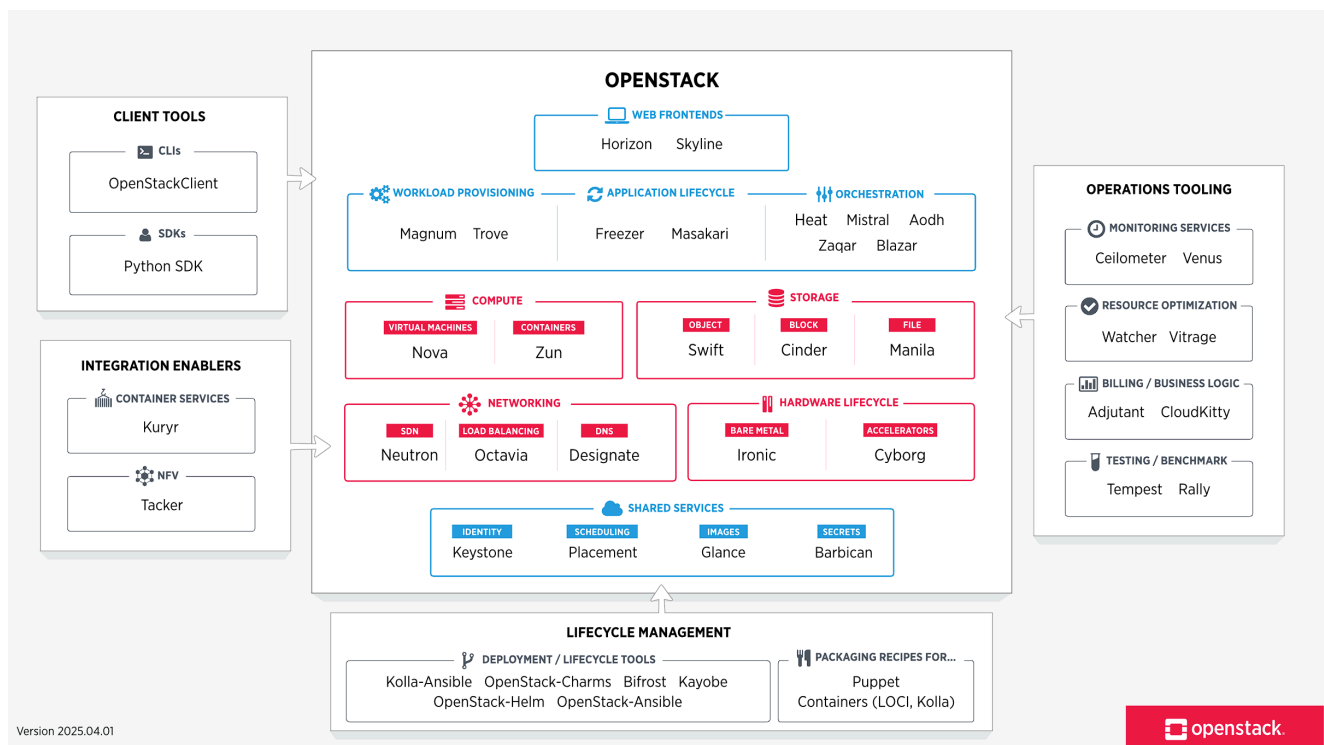
PaaS (Platform as a Service)

PaaS offers a managed platform for application development, deployment, and operation without managing underlying infrastructure. In 2026, PaaS includes serverless functions, managed databases, API gateways, native CI/CD, and event-driven and microservices architectures with automatic scaling.

SaaS (Software as a Service)

SaaS delivers software applications via web, accessible through browsers or native clients, with subscription-based models. In 2026, SaaS integrates artificial intelligence for automation and insights, provides extensible customization, open APIs for integration, and supports rigorous compliance with zero-trust security.

OpenStack



OpenStack is an open-source cloud computing platform that provides IaaS capabilities for private and hybrid cloud environments. In 2026, OpenStack maintains its relevance in corporate datacenters and private cloud providers, offering a complete service stack (Nova for compute, Cinder for block storage, Swift for object storage, Neutron for networking) with advanced support for Kubernetes containers, automatic load balancing, and integration with enterprise-grade security and compliance solutions.

Apache CloudStack

Apache CloudStack is an open-source cloud computing software platform that provides IaaS capabilities for building and managing public and private clouds.

CloudStack continues to be a reliable choice for service providers and enterprises, offering features such as multi-tenancy, advanced networking (SDN), and support for various hypervisors (KVM, XenServer, VMware) with enhanced integration for container orchestration and edge computing.

Eucalyptus

Eucalyptus is an open-source cloud computing platform that provides IaaS capabilities for building and managing private and hybrid clouds. It is designed to be compatible with Amazon Web Services (AWS) APIs, allowing users to run applications on Eucalyptus that are already deployed on AWS.

OpenNebula

OpenNebula is an open-source cloud computing platform that provides IaaS capabilities for building and managing private, public, and hybrid clouds. It focuses on simplicity and flexibility, allowing users to deploy and manage virtualized data centers with support for various hypervisors (KVM, VMware) and container technologies (LXC, Docker). OpenNebula continues to evolve with features such as edge computing support, enhanced networking capabilities, and integration with modern DevOps tools.

IaC

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable definition files, rather than through physical hardware configuration or interactive configuration tools. IaC has become a fundamental practice in DevOps and cloud-native environments, enabling automated, consistent, and repeatable infrastructure deployment across multiple cloud providers and on-premises environments.

Terraform

Terraform is an open-source Infrastructure as Code (IaC) tool that enables infrastructure definition and provisioning in a declarative and versioned manner. By 2026, Terraform is the de facto standard for infrastructure management in multicloud environments (AWS, Azure, GCP, OpenStack, etc.), with robust support for distributed state, policy as code, reusable modules, and native integration with CI/CD pipelines, GitOps, and compliance automation.

Terraform Key Concepts

Concept	Description
Provider	A plugin that enables interaction with cloud providers and services (e.g., AWS, Azure, GCP).
Resource	A component of your infrastructure (e.g., virtual machines, networks, storage).
Module	A container for multiple resources
State	A snapshot of your infrastructure, stored in a file or remote backend, used to track resource changes.
Plan	A preview of changes that Terraform will make to your infrastructure.
Apply	The command that executes the changes defined in the plan to your infrastructure.

Terraform Basic Workflow

- Write Configuration:** Define your infrastructure using HashiCorp Configuration Language (HCL) in `.tf` files.
- Initialize:** Run `terraform init` to initialize the working directory and download necessary provider plugins.

3. **Plan:** Execute **terraform plan** to create an execution plan, showing what actions Terraform will take to achieve the desired state.
4. **Apply:** Use **terraform apply** to apply the changes and create or modify resources in your infrastructure.
5. **Destroy:** Use **terraform destroy** to remove all resources managed by Terraform.

Terraform Example

```
provider "aws" {  
  region = "us-west-2"  
  profile = "my-profile"  
}  
resource "aws_instance" "web" {  
  ami           = "ami-12345678"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "MyWebServer"  
  }  
}
```

353.2 Packer

Weight: 2

Description: Candidates should be able to use Packer to create system images. This includes running Packer in various public and private cloud environments as well as building container images for LXC/LXD.

Key Knowledge Areas:

- Understand the functionality and features of Packer
- Create and maintain template files
- Build images from template files using different builders

353.2 Cited Objects

```
packer  
hcl2
```

About Packer

Packer is a tool for creating identical machine images for multiple platforms from a single source configuration.

It is designed to be a lightweight, fast, and flexible tool that can be used to build images for various cloud providers and virtualization platforms.

Packer Features

- **Multi-Platform Support:** Packer supports a wide range of platforms, including AWS, Azure, Google Cloud, VMware, VirtualBox, Docker, LXC/LXD, and more.

- **Declarative Configuration:** Packer uses JSON or HCL (HashiCorp Configuration Language) templates to define the image-building process, making it easy to version control and share configurations.
- **Provisioners:** Packer supports various provisioners, such as shell scripts, Ansible, Chef, Puppet, and Salt, allowing users to customize the images during the build process.
- **Parallel Builds:** Packer can build images for multiple platforms simultaneously, reducing the time required to create images for different environments.
- **Extensibility:** Packer has a plugin architecture that allows users to create custom builders and provisioners to extend its functionality.
- **Integration with CI/CD:** Packer can be integrated into continuous integration and continuous deployment (CI/CD) pipelines, enabling automated image creation and deployment.

Packer workflow functionality diagram

1. connect to a builder (e.g., AWS, Azure, Docker)
2. start a temporary instance or container
3. run provisioners to customize the instance
4. create an image from the customized instance
5. stop the instance or container
6. execute post-processors (optional)

Template Components

- **Builders:** Define the target platform and configuration for the machine image. Examples include `amazon-ebs`, `googlecompute`, `docker`, `lxc`, etc.
- **Provisioners:** Specify the steps to customize the image during the build process. Examples include `shell`, `ansible`, `chef`, `puppet`, etc.
- **Post-Processors:** Optional steps to modify or export the built image after the build process. Examples include `compress`, `docker-tag`, `vagrant`, etc.

Packer Example Templates

Template Docker

```
packer {
  required_plugins {
    docker = {
      version = ">= 1.0.0"
      source  = "github.com/hashicorp/docker"
    }
  }
}
source "docker" "nginx" {
  image = "nginx:latest"
  commit = true
}
build {
  sources = ["source.docker.nginx"]

  provisioner "shell" {
    inline = [
      "apt-get update",
      "apt-get install -y curl",
    ]
  }
}
post-processor "docker-tag" {
  repository = "my-nginx"
  tag        = "latest"
}
```

Template LXD

```
packer {
  required_plugins {
    lxd = {
      version = ">= 1.0.0"
      source  = "github.com/hashicorp/lxd"
    }
  }
}
source "lxd" "ubuntu" {
  image = "ubuntu/20.04"
  name  = "ubuntu-20.04-packer"
}
build {
  sources = ["source.lxd.ubuntu"]

  provisioner "shell" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nginx",
    ]
  }
}
post-processor "lxd-export" {
  output = "ubuntu-20.04-nginx.tar.gz"
}
```

Template Amazon

```
{
  "variables": {
    "aws_access_key": "F00",
    "aws_secret_key": "BAR"
  },
  "builders": [
    {
      "type": "amazon-ebs",
      "access_key": "{{user `aws_access_key`}}",
      "secret_key": "{{user `aws_secret_key`}}",
      "region": "sa-east-1",
      "instance_type": "t2.micro",
      "source_ami": "ami-0e7dc6a7bf702d57f",
      "ami_name": "packer-ami-{{timestamp}}",
      "ssh_username": "ubuntu"
    }
  ],
  "provisioners": [
    {
      "type": "file",
      "source": "index.html",
      "destination": "~/",
    },
    {
      "type": "shell",
      "inline": [
        "sudo apt update -y",
        "sudo apt install nginx -y",
        "sudo cp ~/index.html /var/www/html/"
      ]
    }
  ],
  "post-processors": [
    {
      "type": "vagrant"
    },
    {
      "type": "compress",
      "output": "vagrant.tgz"
    }
  ]
}
```

353.2 Important Commands

packer

```
# list available plugins
packer plugins installed

# install plugins
packer plugins install github.com/hashicorp/docker
packer init .
packer init template.pkr.hcl

# get a plugins required by a template
packer plugins required template.pkr.hcl

# validate a template file
packer validate template.pkr.hcl

# build an image from a template file
packer build template.pkr.hcl

# inspect a built image
packer inspect template.pkr.hcl

# list available builders and provisioners
packer plugins
```

353.3 cloud-init

Weight: 3

Description: Candidates should be able to use cloud-init to configure virtual machines created from standardized images. This includes adjusting virtual machines to match their available hardware resources, specifically, disk space and volumes. Additionally, candidates should be able to configure instances to allow secure SSH logins and install a specific set of software packages. Furthermore, candidates should be able to create new system images with cloud-init support.

Key Knowledge Areas:

- Understanding the features and concepts of cloud-init, including user-data, initializing and configuring cloud-init
- Use cloud-init to create, resize and mount file systems, configure user accounts, including login credentials such as SSH keys and install software packages from the distribution's repository
- Integrate cloud-init into system images
- Use config drive datasource for testing

353.3 Cited Objects

```
cloud-init
user-data
/var/lib/cloud/
```

Understanding cloud-init

Tool used for customizing cloud instances at boot time.

It is typically used to perform initial setup tasks such as configuring network interfaces, setting up user accounts, installing software packages, and running custom scripts.

Sources of cloud-init data

Source	Description
meta-data/vendor-data	Customizations provided by the platform managing the VM, such as the Cloud platform.
user-data	User-defined instructions primarily using YAML format or shell scripts.
Configuration Files	Instructions can also be provided through configuration files in YAML or Script format.
Datasource	Determines the origin of the information necessary for instance configuration (meta-data/user-data).

Modules in cloud-init

Below are some of the commonly used cloud-init modules, which are responsible for executing specific tasks during the instance initialization process:

Module	Description
bootcmd	Commands that run at every boot, before any other cloud-init modules.
runcmd	Commands that run at the end of the cloud-init process, after all other modules have completed.
users	Used to create and manage user accounts, including setting up SSH keys for secure logins.
packages	Used to install software packages from the distribution's repository.
disk_setup	Used to create, resize, and mount file systems on the instance's disks.
network	Used to configure network interfaces and settings for the instance.
write_files	Used to write files to the instance's file system, which can be used for configuration or other purposes.

Cloud-Init Configuration Files and Directories

- /etc/cloud/ – Main Configuration Files
 - cloud.cfg – The primary configuration file for Cloud-Init

- cloud.cfg.d/ – Directory containing additional configuration files that are merged with the cloud.cfg file
- /var/log/cloud-init.log – Log file containing detailed records of Cloud-Init activities during instance initialization
- /var/log/cloud-init-output.log – Logs the console output generated by initialization scripts executed by Cloud-Init
- /var/lib/cloud/ – Directory containing data and information generated by Cloud-Init during instance initialization
 - /var/lib/cloud/instance/ – Directory containing files and metadata related to the running instance

353.3 Important Commands

cloud-init

```
# clear cloud-init state and logs
sudo cloud-init clean
```

353.4 Vagrant

Weight: 3

Description: Candidate should be able to use Vagrant to manage virtual machines, including provisioning of the virtual machine.

Key Knowledge Areas:

- Understand Vagrant architecture and concepts, including storage and networking
- Retrieve and use boxes from Atlas
- Create and run Vagrantfiles
- Access Vagrant virtual machines
- Share and synchronize folder between a Vagrant virtual machine and the host system
- Understand Vagrant provisioning, i.e. File and Shell provisioners
- Understand multi-machine setup

353.4 Cited Objects

```
vagrant
Vagrantfile
```

About Vagrant

Vagrant is an open-source tool for building and managing virtual machine environments in a single workflow. It provides a simple and easy-to-use command-line interface for creating and configuring lightweight, reproducible, and portable development environments.

Vagrant Architecture and Concepts

Vagrant uses a declarative configuration file called **Vagrantfile** to define the virtual machine environment. The **Vagrantfile** specifies the base box to use, the virtual machine provider (e.g., VirtualBox, VMware, Hyper-V), and any additional configuration such as networking, shared folders, and provisioning scripts.

Vagrant Providers

A Vagrant provider is a plugin that allows Vagrant to manage virtual machines using a specific virtualization technology.

Common providers include:

- **VirtualBox:** A free and open-source virtualization platform that is widely used for local development environments.
- **VMware:** A commercial virtualization platform that offers advanced features and performance for enterprise environments.
- **Hyper-V:** A virtualization platform developed by Microsoft, available on Windows operating systems.
- **Docker:** A containerization platform that allows Vagrant to manage containerized environments.

Vagrant Plugins

Vagrant plugins are extensions that add functionality to Vagrant.

Examples of Vagrant plugins include:

- **vagrant-vbguest:** Automatically installs the VirtualBox Guest Additions on the guest machine, improving performance and enabling features such as shared folders and clipboard sharing.
- **vagrant-hostmanager:** Manages the host's **/etc/hosts** file to allow easy access to Vagrant machines by name.
- **vagrant-disksize:** Allows resizing of the virtual machine's disk size during provisioning.

Vagrant Provisioning

Vagrant provisioning is the process of configuring the virtual machine after it has been created.

Provisioning can be done using various provisioners, such as:

- **File Provisioner:** Used to copy files from the host machine to the guest machine during provisioning.
- **Shell Provisioner:** Used to run shell scripts on the guest machine during provisioning, allowing for tasks such as installing software packages, configuring services, and setting up the environment.
- **Ansible Provisioner:** Used to run Ansible playbooks on the guest machine during provisioning, allowing for more complex configuration management and automation.
- **Puppet Provisioner:** Used to run Puppet manifests on the guest machine during provisioning, allowing for configuration management and automation using Puppet.
- **Chef Provisioner:** Used to run Chef recipes on the guest machine during provisioning, allowing for configuration management and automation using Chef.

Vagrant Boxes

A Vagrant box is a pre-packaged virtual machine image that serves as the base for creating new virtual machine instances.

Boxes can be retrieved from the Vagrant Cloud (formerly known as Atlas).

Vagrant Cloud is a platform for sharing and distributing Vagrant boxes, allowing users to easily find and use pre-configured environments for their development needs.

Vagrantfile Example

```
Vagrant.configure("2") do |config|
  # Define the base box to use
  config.vm.box = "ubuntu/bionic64"
  # Configure the virtual machine provider
  config.vm.provider "virtualbox" do |vb|
    vb.memory = "1024"
    vb.cpus = 2
  end
  # Configure a private network
  config.vm.network "private_network", ip: "192.168.33.10", netmask: "255.255.255.0"

  # Configure a shared folder
  config.vm.synced_folder "./data", "/vagrant_data"
  # Provision the virtual machine using a shell script
  config.vm.provision "shell", inline: <<-SHELL
    apt-get update
    apt-get install -y nginx
  SHELL
end
```

Install vagrant

```
# on Debian/Ubuntu
wget -q https://releases.hashicorp.com/vagrant/2.4.9/vagrant_2.4.9-1_amd64.deb
sudo dpkg -i vagrant_2.4.9-1_amd64.deb
sudo apt-get install -f
```

Install Vagrant Plugins

```
vagrant plugin install vagrant-vbguest
vagrant plugin install vagrant-hostmanager
vagrant plugin install vagrant-disksize
```

353.4 Important Commands

vagrant

```
# vagrant plugins
vagrant plugin list
vagrant plugin install vagrant-vbguest
vagrant plugin uninstall vagrant-vbguest

# vagrant box
vagrant box list
vagrant box add ubuntu/jammy64
vagrant box remove ubuntu/jammy64

# init a new vagrant environment
vagrant init ubuntu/jammy64

# init a new vagrant environment with minimum output
vagrant init -m ubuntu/jammy64

# vagrant validate the Vagrantfile
vagrant validate

# status of the vagrant environment
vagrant status
vagrant global-status

# vagrant up the machine
vagrant up
vagrant up --provider=virtualbox
vagrant up <machine-name>
vagrant up --provision

# vagrant suspends the machine
vagrant suspend

# vagrant resumes the machine
vagrant resume

# vagrant halts the machine
vagrant halt
vagrant halt <machine-name>

# vagrant ssh into the machine
vagrant ssh
vagrant ssh <machine-name>

# vagrant destroys the machine
vagrant destroy
vagrant destroy -f
vagrant destroy <machine-name>

# vagrant provision the machine
vagrant provision
vagrant provision <machine-name>

# vagrant reload the machine
vagrant reload --provision
vagrant reload <machine-name>
```

```
# list ports forwarded by vagrant
vagrant port
vagrant port <machine-name>
```



Contributing

Contributions are what make the open source community such an amazing place to learn, inspire, and create. Any contributions you make are **greatly appreciated**.

If you have a suggestion that would make this better, please fork the repo and create a pull request. You can also simply open an issue with the tag “enhancement”. Don’t forget to give the project a star! Thanks again!

1. Fork the Project
2. Create your Feature Branch (`git checkout -b feature/AmazingFeature`)
3. Commit your Changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the Branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request



License

- This project is licensed under the MIT License * see the LICENSE.md file for details

Contact

Marcos Silvestrini - marcos.silvestrini@gmail.com

Project Link: <https://github.com/marcossilvestrini/learning-lpic-3-305-300>



Acknowledgments

- Richard Stallman's
 - GNU
 - GNU/Linux FAQ by Richard Stallman
 - GNU
 - GNU Operating System
 - GCC Compiler
 - GNU Tar
 - GNU Make
 - GNU Emacs
 - GNU Packages
 - GNU/Linux Collection
 - GNU Grub Bootloader
 - GNU Hurd
- Kernel
 - Kernel
 - Linux Kernel Man Pages
 - Compile Your Kernel
- Linux Standard Base
 - Linux Standard Base
 - Filesystem Hierarchy Standard
 - File Hierarchy Structure
- Free Software
 - FSF
 - Free Software Directory
- License
 - Free Software
 - Copyleft

- GPL
- GNU Lesser General Public License
- BSD
- Open Source Initiative
- Creative Commons
- License LTS
- Distros
 - Debian Free Software Guidelines
 - List Linux Distribution
 - Distro Watch
 - Comparison Linux Distributions
- Desktop Environments
 - X11 Org
 - Wayland
 - GNU GNOME
 - GNOME
 - XFCE
 - KDE Plasma
 - Harmony
- Protocols
 - HTTP
 - W3Techs
 - Apache
 - Apache Directives
 - HTTP Status Codes
 - Strong Ciphers for Apache, nginx and Lighttpd
 - SSL Tutorials
 - SSL Config Mozilla
 - xRDP
 - NTP
- DNS
 - Bind

- Bind Logging
 - List of DNS record types
 - List of DNS record types
- Package Manager
 - Download Packages
 - Install Packages
 - Guide Install Packages
- Shell Script
 - Bourne Again Shell
 - Shebang
 - Environment Variables
 - GNU Globbing
 - Globbing
 - Quoting
 - Regular Expressions
 - Command Not Found
 - Bash Prompt Generator
 - Explainshell
 - Vim Tutorial
 - Linux Shell Scripting Tutorial
 - Commands Examples
- Others Tools
 - Bugzilla
 - Github Badges
- Virtualization Definitions
 - Red Hat
 - AWS
 - IBM
 - OpenSource.com
- Xen
 - XenServer
 - Wiki XenProject

- Network Interfaces
- Xen Tools
- LPI Blog: Xen Virtualization and Cloud Computing #01: Introduction
- LPI Blog: Xen Virtualization and Cloud Computing #02: How Xen Does the Job
- LPI Blog: Xen Virtualization and Cloud Computing #04: containers, OpenStack, and Other Related Platforms
- Xen Virtualization and Cloud Computing #05: The Xen Project, Unikernels, and the Future
- Xen Project Beginners Guide
- Crazy Book
- Unikernels
 - Unikraft
 - MirageOS
 - HaLVM
 - Unik
- KVM
 - Official Doc
 - KVM(Kernel Virtual Machines by RedHat)
 - KVM Management Tools
 - KVM Network
- QEMU
 - Official Doc
 - Download Images osboxes
 - Download Images linuximages
 - Virtio
 - Guest Agent
- Libvirt
 - Official Doc
 - System Socket Activation
 - Conections
 - Storage
 - Network
 - VirtualNetwork
 - virtlogd

- virtlockd
- virt-manager
- Disk Management
 - Disk Images
 - copy-on-write
 - RAM x QCOW2
 - Libguestfs
- containers
 - AWS Doc containers
 - GCP Doc containers
 - IBM Doc container
 - Red Hat Docs containers
 - Namespaces
 - Most important Namespaces
 - Cgroups Classes
 - Man Cgroups
 - Capabilities Doc
 - Man Capabilities
 - Seccomp Profiles in Docker
 - AppArmor Profiles in Docker
 - SELinux
 - Apparmor SELinux comparator
 - runC
 - runc Github
 - OCI
 - CRI
 - CRI-O
 - containerd
 - Podman
 - Skopeo
 - Buildah
 - OpenVZ
 - crun
 - kata

- LXC - Linux containers
 - LXC
 - Linux container Images
- LXD
 - LXD Canonical
 - LXD Canonical Github
 - LXD Documentation
 - LXD Installation
 - LXD Images
 - LXD Storage
 - LXD Storage pools, volumes, and buckets
 - LXD Network Types
 - LXD Network Parameters
 - LXD Network Configuration
 - LXD Profiles
 - LXD Instances
- Docker
 - Docker Overview
 - Containerd x Docker
 - Install
 - Daemon Configuration
 - Docker Images
 - Docker Hub
 - Storage on Docker
 - Volumes
 - Bind Mounts
 - Storage Drivers
 - Testcontainers
 - Docker Networking
 - Docker Network Drivers
 - Dockerfile
 - Dockerfile Best Practices
 - The Twelve-Factor App - Processes

- Container Orchestration
 - Container Orchestration by IBM
 - Container Orchestration by AWS
 - Container Orchestration by RedHat
 - Container Orchestration by Google Cloud
 - Docker Compose
 - Docker Swarm
 - Keys Concepts
 - Nodes
 - Services
 - Docker Swarm Stack
 - Play with Docker Swarm
 - Kubernetes
 - Kubernetes
 - Components
 - Architecture
 - Red Hat Site
 - Minikube
 - Kind
 - Play with Kubernetes
- Cloud
 - Red Hat - Cloud Computing
- Openstack Docs
 - OpenStack
 - RedHat
 - The OpenStack Landscape
 - Componentes do OpenStack
 - Apache CloudStack
 - Eucalyptus
 - Eucalyptus - Componentes
 - Open Nebula
- IaC - Infrastructure as Code

- Terraform
 - AWS/Terraform Get Started
 - Terraform Examples
- Packer
 - Packer Getting Started
 - Packer Templates
- cloud-init
 - cloud-init Documentation
 - cloud-init Modules
 - cloud-init Examples
- Vagrant
 - Vagrant Documentation
 - Vagrant Boxes
 - Vagrant Providers
 - Vagrant Plugins
 - Available Vagrant Plugins
 - Vagrant Boxes Search
- Open vSwitch
 - OVS Doc 4Linux
- LPIC-3 305-300 Exam
 - LPIC-3 305-300 Objectives
 - LPIC-3 305-300 Wiki
 - LPIC-3 305-300 Learning Material
 - LPIC-3 305-300 Simulated Exam By ITexams